

មូលដ្ឋានគ្រឹះនៃ



Arduino Programming

រៀបរៀងលើកទី



Thol Chanthorn

អារម្ភកថា

សៀវភៅ **មូលដ្ឋានគ្រឹះនៃ** Arduino ដែលលោកគ្រូ អ្នកគ្រូ ប្អូនៗ និង អ្នកសិក្សា ស្រាវជ្រាវកំពុងកាន់កាប់ក្នុងដៃនេះ ជាស្នាដៃដែលខ្ញុំបានរៀបរៀងជាជំនួយមូលដ្ឋាន ដល់ប្អូនៗ ដែលសិក្សានៅថ្នាក់ចំណេះដឹងទូទៅ ដែលនឹងជ្រើសរើសយកជំនាញ មេកានិច អេឡិចត្រូនិច និង ព័ត៌មានវិទ្យាវិស្វកម្ម នៅក្រោយពីបញ្ចប់ថ្នាក់វិទ្យាល័យ ។

នៅក្នុងសៀវភៅនេះ នឹងបង្ហាញពីប្រើប្រាស់មុខវិជ្ជា វិទ្យាសាស្ត្រ និងគណិតវិទ្យា មកប្រើ ដែលរួមមាន ទ្រឹស្តី ការផ្គុំឧបករណ៍អេឡិចត្រូនិច និង ការសរសេរកូដ ពោលគឺមុខវិជ្ជាស្វែម(STEM) ។ ខ្ញុំសង្ឃឹមថាសៀវភៅនេះនឹងជួយជំរុញឱ្យប្អូនៗខិតខំនិងត្រៀមខ្លួនមុន និង ចាប់យកនូវជំនាញដែលខ្លួនចូលចិត្ត នៅថ្ងៃអនាគត ដើម្បីកសាងប្រទេសរបស់យើង ឱ្យមានរីកចម្រើនក្នុងសម័យបដិវត្តន៍ឧស្សាហកម្មជំនាន់ទី៤នេះ ។

ខ្ញុំសូមអរគុណចំពោះការគាំទ្រ និងរង់ចាំទទួលនូវការរិះគន់រាល់កំហុសឆ្គង ពីគ្រប់មជ្ឈដ្ឋានដោយក្តីរីករាយ ។

ថ្ងៃ ចន្ទ ៦ រោច ខែស្រាពណ៍ ឆ្នាំច សំរឹទ្ធស័ក ព.ស ៤៥៦២

ភ្នំពេញ ថ្ងៃទី ០២ ខែ កញ្ញា ឆ្នាំ ២០១៨

រៀបរៀងដោយ **ថុល ចាន់ថន**

បញ្ជីអត្ថបទ

ជំពូកទី១

មូលដ្ឋានគ្រឹះនៃអេឡិចត្រូនិច

I. អាំងតង់ស៊ីតេ តង់ស្យុង ថាមពល និង ការបង្កើតចរន្តអគ្គិសនី	1
1. វេស៊ីស្តង់	3
1.1. ការជ្រើសរើសវេស៊ីស្តង់	4
2. តង់ស្យុង	6
3. អានុភាពអគ្គិសនី	6
4. សៀគ្វីអគ្គិសនី	6
5. កុងដង់សាទ័រ	7
6. អាំងឌុចតង់	10
7. ឌីយ៉ូត និង LED	11
7.1. សុវត្ថិភាពភ្លើងបញ្ជ្រាស់	15
7.2. Flyback Diode ការពារទល់នឹង Voltage Spike	16

ជំពូកទី២

ការអនុវត្តអេឡិចត្រូនិច និងកម្មវិធី Arduino

I. Digital Output	18
1. Blinking តាមវិធី: ICs NE555	18
II. Arduino	19
1. ការបង្ហាញអំពី IDE Arduino	21
2. លក្ខណៈរបស់បន្ទុះ Arduino Uno	22
3. Blinking តាមវិធី: Arduino	25
3.1. អញ្ញាត (Variable)	25
3.2. ទិសដៅដឹងរបស់ Arduino Uno	25
3.3. Delay Time	26
3.4. ការបញ្ចេញសញ្ញាឌីជីថល (Digital Write)	26
3.5. LED Blink	26
3.6. for loop Statement	27
III. ប្រព័ន្ធរាប់	31
IV. អង្កត់លេខ 7-segment	32
V. អារេ (Array)	37
1. ការបង្ហាញ 7-segment លេខមួយខ្ទង់ តាម bitRead	38
2. ការបង្ហាញ 7-segment លេខពីរខ្ទង់តាម bitRead	40
3. ការបង្កើត function	43
VI. Driver for 7 segments more than 2 digits (Max7219)	44
1. 7-segment 0-9	49
2. 7-segment 00-99	50

3.	7-segment 000-999	51
4.	7-segment 0000-9999.....	52
5.	7-segment 0000-99999.....	52
6.	7-segment 00000-999999.....	53
7.	ការបង្កើតភ្លើងសញ្ញាចរាចរណ៍	55

ជំពូកទី៣ ការប្រើប្រាស់ថាមពលខ្ពស់ជាថាមពល Input របស់ Arduino

I.ត្រង់ស៊ីស្ទ័រ (Transistor)	60
II.រឺឡេ (Relay)	67
III.Digital Input	68
1.Comparision Operation	68
2.Bitwise Operator	69
3.Boolean Operation	70
4. Compound Operator.....	71
5.Control structure	71
5.1. if/else.....	72
5.2.switch/case Stement	72
5.3.while loops	73
5.4.do-while	73
6.Digital Input arduino	74
IV.Filter.....	74
1.Hardware Filter	74
2.Softeware Filter.....	77
V.Push Button.....	78
1.ការប្រើប្រាស់ប៊ូតុងមួយ.....	78
2.Two buttons LED on and off	78
3. One button LED on and off.....	79
4.One digit count up 0-9	79
5.ប៊ូតុងពីរ មួយចុចពី0-9 និងមួយទៀត 9-0(មួយខ្ទង់).....	80
6.ប៊ូតុងមួយ ចុចពី 00-99.....	81
7.ប៊ូតុង2 មួយចុចពី 00-99 មួយទៀត 99-00	82

ជំពូកទី៤ Serial Monitor

I.Serial Protocol ជាអ្វី?	85
II. Serial of Arduino	85
1.Function of Syntax និងឧទាហរណ៍.....	85
2.String Objects.....	92

ជំពូកទី៥ ការបង្ហាញបត្រាស្ទើរអេក្រង់ LCD និង OLED

I.Liquid Crystal Display (LCD)	100
1.Function of Syntax និងឧទាហរណ៍.....	101

2.ការដាក់ម៉ោង digital	107
II.Graphic LED OLED.....	109
1.ការប្រើប្រាស់	109
2.Function of Syntax	111
3.ការគូរក្រាហ្វិក	114
4.ការបង្កើតវិទ្យុ	122
5.Real Time Clock.....	123
6.Keypad	126
III.EEPROM	129
1.Function of Syntax និង ឧទាហរណ៍	129

ជំពូកទី៦ មូលដ្ឋានគ្រឹះនៃទំនាក់ទំនងតាមប្រព័ន្ធ WIRELESS

I.ការគ្រប់គ្រងការបញ្ជាតាមប្រព័ន្ធ IR	132
1.Function of Syntax	133
2.ការបង្ហាញតាមវិស័យៈ Serial Monitor	135
II. RF Tx/Rx module	138
1.Function of Syntax	138
III.UART Tx/Rx Tranceiver module(HC-12)	140
1.Function of Syntax	140
2.របៀប Transmite និង Receive Data	143
3.របៀបបញ្ជូនសញ្ញាសំឡេងហើយធ្វើឱ្យអំពូលភ្លឺ.....	145
4.HC-12 with Button.....	147

ជំពូកទី៧ មូលដ្ឋានគ្រឹះនៃការបញ្ជាម៉ូទ័រ DC

I.Analog Input / Output	148
1.Analog Read and Syntax.....	148
2.រៀបបញ្ជាអំពូល និង ម៉ូទ័រឱ្យដំណើរការ.....	150
3.ការប្រើអំពូល RGB LED.....	151
4.ការបញ្ជា car តាម IR remote.....	153
5.ការបញ្ជា car តាម HC-12	154
II.ការបញ្ជា servo motor	134
1.Function of Syntax និង ឧទាហរណ៍.....	159
III.SD Card.....	163

ជំពូកទី៨ Sensors

1.Thermistor sensor.....	167
2.Light Sensor	170
3.Distance Sensor.....	171
4.Security Sensor	172
5.Argriculture Sensor	174
6.Switch sensor	175

ឯកសារពិគ្រោះ..... 176

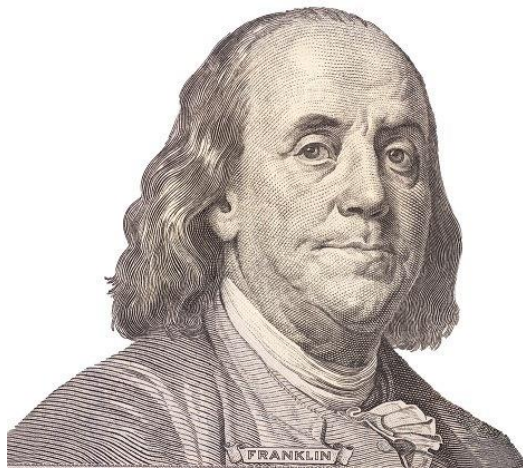
ជំពូកទី១

មូលដ្ឋានគ្រឹះអេឡិចត្រូនិច

នៅក្នុងជំពូកនេះ អ្នកនឹងរៀនពីគោលការណ៍របស់អេឡិចត្រូនិច ឧបករណ៍អេឡិចត្រូនិច គោលការណ៍បង្កើតស្មុំស្មើគ្នា ការប្រើប្រាស់ និងការជ្រើសរើសស៊ីស្ទង់ឱ្យបានត្រឹមត្រូវ ។

I. អំពីតង់ស៊ីតេ តង់ស្យុង ថាមពល និង ការបង្កើតចរន្តអគ្គិសនី

តើអ្វីទៅជាអគ្គិសនី ? ទម្រង់នៃថាមពលសំរេចមួយដែលកើតចេញពីការបន្ថែមអេឡិចត្រុង ពីចំណុចអវិជ្ជមាន ទៅចំណុចវិជ្ជមានតាមរយៈអង្គធាតុចំលងអគ្គិសនី ។ ដោយសារតែការបង្ហោះ ខ្លួនរបស់ Benjamin Franklin លោកបានរកឃើញថា រន្ទះបាញ់ជាទម្រង់នៃការបន្ថែមថាមពល អគ្គិសនី ។



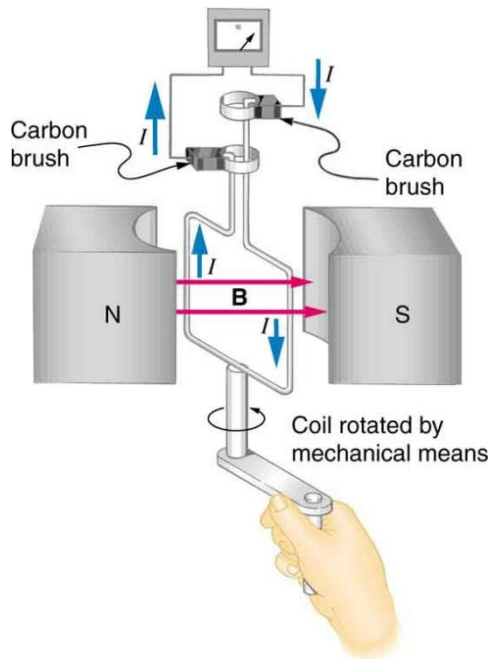
តើអ្វីទៅជាចរន្តអគ្គិសនី ? ជាបរិមាណបន្ទុកអគ្គិសនីដែលផ្លាស់ទីលើខ្សែចំលងក្នុងរយៈពេលមួយវិនាទី ។ ចុះអ្វីទៅជាតង់ស្យុង ? វាជារង្វាស់កំរិតខុសគ្នារវាង ប៉ូតង់ស្យែលពីរផ្សេងគ្នាគឺ ប៉ូលវិជ្ជមាន និងអវិជ្ជមាន ។ ហើយចុះថាមពលវិញ ? ជាការវាស់នូវកំរិតមួយដែលគ្រឿងអគ្គិសនី បានបំប្លែងថាមពលអគ្គិសនីទៅជាថាមពលមួយផ្សេងទៀត ។ ឧទាហរណ៍ដូចជាអំពូលបំប្លែងពី ថាមពលអគ្គិសនីទៅជាពន្លឺដែលអំពូល 100W ភ្លឺជាអំពូល 60W ។

ក្នុងអតីតកាលអគ្គិសនីត្រូវបានគេបង្កើតក្រោមទម្រង់ជាការត្រជុសជ័រឈើ ដែលក្រោយមកត្រូវបានរកឃើញតាមរយៈ ការបង្កើតថ្មពិលគីមី ។ ចរន្តអគ្គិសនីមានពីរប្រភេទ គឺចរន្តជាប់ និងចរន្តឆ្លាស់ ៖

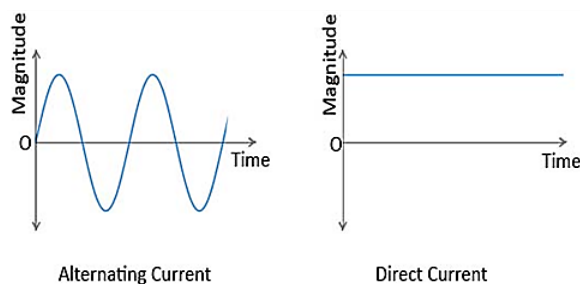
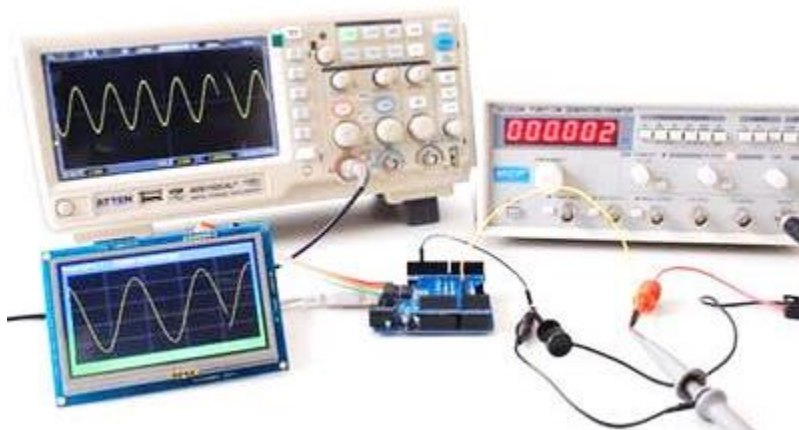
ថ្មពិល ឬអាគុយជាប្រភពបង្កើតចរន្តជាប់ វាផ្ទុកថាមពលអគ្គិសនីបានយូរឬឆាប់អាស្រ័យលើសារធាតុគីមីរបស់វា ។ បច្ចុប្បន្នថ្មពិលអាចត្រូវបានគេធ្វើឡើងដើម្បីសាកប្រើប្រាស់ឡើងវិញបាន ។ ប្រភពចរន្តជាប់មួយដែលមិនចេះវិស្វតគឺ ថាមពល សូឡា (Solar Panel) បច្ចុប្បន្នកំពុងពេញនិយម ។



ឱណាម៉ូរ ម៉ាស៊ីនភ្លើង វាវិអគ្គិសនី ឬថាមពលខ្យល់ សុទ្ធតែបង្កើតចរន្តឆ្លាស់ ។ ចរន្តឆ្លាស់គឺជាចរន្តដែលប្តូរទិសដៅរត់របស់វាជានិច្ចកាលរៀងរាល់វិនាទី រយៈពេលនោះត្រូវបានគេហៅថាខួប ។ ការប្តូរទិសចរន្តនេះកើតឡើង ដោយសារឥទ្ធិពលនៃម៉ាញ៉េទិចរិលក្បែរខ្សែចំលងរំដាប ។



អ្នកនឹងបានពិសោធន៍ជាមួយចរន្តជាប់ និងចរន្តផ្លាស់ ។ អ្នកនឹងឃើញភាពខុសគ្នា រវាងចរន្តទាំងពីរតាមរយៈម៉ាស៊ីន Oscilloscope ។



1. រេស៊ីស្តង់ (Resistor)

រេស៊ីស្តង់គឺជាឧបករណ៍អេឡិចត្រូនិចដែលមានតួនាទីជាអ្នកកំណត់ ឬកម្រិតនូវតម្លៃរបស់ចរន្តអគ្គិសនីដែលហូរឆ្លងកាត់ក្នុងសៀគ្វីអគ្គិសនីមួយ ។ វាក៏អាចត្រូវបានគេប្រើសម្រាប់កំណត់តម្លៃតង់ស្យុងបានដែរ ។

ច្បាប់រូបវិទ្យាអគ្គិសនីមួយដែលត្រូវបានគេប្រើសម្រាប់សិក្សាលើរេស៊ីស្តង់គឺ ច្បាប់អូម ។ រូបមន្តនេះត្រូវបានកំណត់ដោយលោក Georg Simon Ohm ដែលមានរូបមន្ត៖

$$\boxed{I = \frac{U}{R}} \text{ ដែល } \begin{cases} I: \text{ចរន្តអគ្គិសនី ឬអាំងតង់ស៊ីតេចរន្ត គិតជាអំពែ (A)} \\ U: \text{តង់ស្យុង គិតជាវ៉ុល (V)} \\ R: \text{រេស៊ីស្តង់គិតជាអូម (\Omega)} \end{cases}$$

វេស៊ីស្តង់មាន Power Dissipation របស់វាខុសគ្នាៗ អាស្រ័យទៅលើទំហំ និងប្រភេទរូបរាងដែលគេបង្កើតវា ។



1/8 Watt Resistors



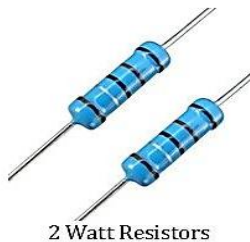
1/4 Watt Resistors



1/2 Watt Resistors



1 Watt Resistors



2 Watt Resistors



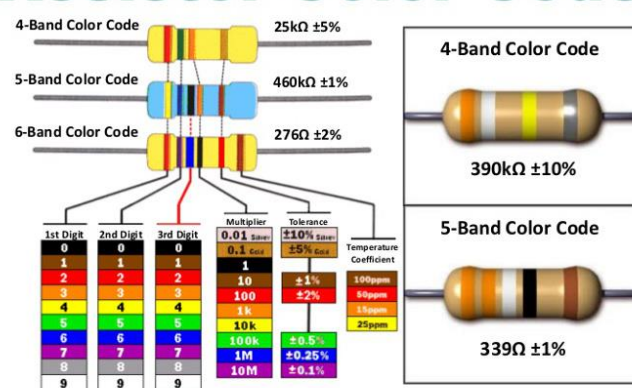
5 Watt Resistor

1.1.ការជ្រើសរើសវេស៊ីស្តង់

របៀបទី១

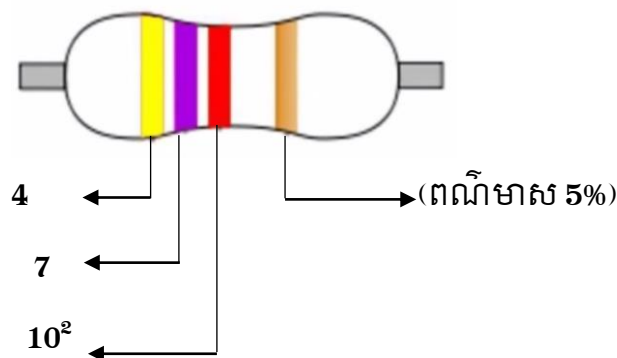
ភាពងាយស្រួលនិងឆាប់រហ័សក្នុងការជ្រើសរើសវេស៊ីស្តង់យកមកប្រើគឺតាមរយៈការមើលពណ៌របស់វា ។

Resistor Color Code



● ពណ៌ខ្មៅ = 0	● ពណ៌បៃតង = 5
● ពណ៌ត្នោត = 1	● ពណ៌ខៀវ = 6
● ពណ៌ក្រហម = 2	● ពណ៌ស្វាយ = 7
● ពណ៌ទឹកក្រូច = 3	● ពណ៌ប្រផេះ = 8
● ពណ៌លឿង = 4	○ ពណ៌ស = 9

ឧទាហរណ៍ គេចង់ដឹងពី Power Dissipation របស់វេស៊ីស្តង់មួយដូចខាងក្រោម

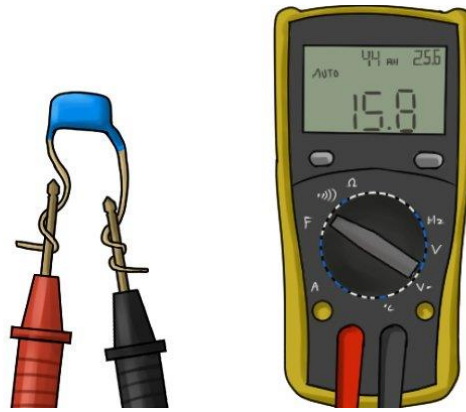


ចំពោះពណ៌មាននៅខាងស្តាំគេបង្អស់មានន័យថាចរន្តអាចរត់ឆ្លងកាត់ 5% មានកម្រិតល្បឿន ។

ចំណែកពណ៌បីផ្សេងទៀតជាពណ៌ដែលបញ្ជាក់តម្លៃរបស់វេស៊ីស្តង់ ។ យើងសង្កេតឃើញថាពណ៌ដំបូងពីខាងស្តាំជាពណ៌លឿងមានតំណាងឱ្យលេខ 4 ហើយពណ៌បន្ទាប់គឺពណ៌ស្វាយស្មើនឹង 7 នោះវាមានន័យថា 47 ។ ចំណែកពណ៌នៅកណ្តាលចុងក្រោយតំណាងឱ្យខ្ទង់ដែលត្រូវគុណនឹង 10^2 ស្វ័យគុណ ។ ឥឡូវ វេស៊ីស្តង់នេះមានពណ៌ពណ៌ក្រហមត្រូវនឹង 2 នោះត្រូវនឹង 10^2 ។ ដូចនេះវេស៊ីស្តង់មាន Power Dissipation គឺ 4700Ω ។

របៀបទី២

យើងក៏អាចប្រើវិធីផ្សេងទៀតដើម្បីពិនិត្យតម្លៃ វេស៊ីស្តង់ផងដែរ គឺតាមរយៈការប្រើ អូមម៉ែត្រ



នៅពេលដែលអ្នកប្រើអ្នមម៉ែត្រគេត្រូវ កាច់ទៅកាន់បង់ អ្នម ការវាស់បែបនេះវានឹងមានតម្លៃ ខុសខ្លះទៅនឹងការមើលក្នុងពណ៌ ។

2. តង់ស្យុង

$$U = R \times I$$

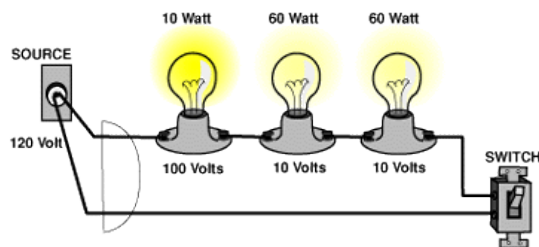
- U = តង់ស្យុង (V)
- I = ចរន្ត (A)
- R = រេស៊ីស្តង់ (Ω)

3. អានុភាពអគ្គិសនី

- $P = U \times I$ P = អានុភាព (W)

4. សៀគ្វីអគ្គិសនី

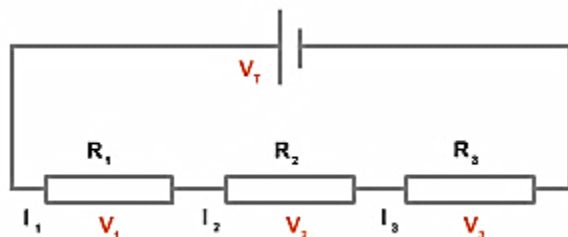
បង្កើតសៀគ្វីស៊េរី



រូបមន្តសមមូល៖

- $R = R_1 + R_2 + R_3$
- $U = U_1 + U_2 + U_3$
- $I = I_1 = I_2 = I_3$

បង្កើតខ្លួន រេស៊ីស្តង់ចុងទាំងសងខាង ត្រូវគេភ្ជាប់ចូលគ្នាតែមួយដូចខាងរូប



រូបមន្ត

- $\frac{1}{R} = \frac{1}{R_1} + \frac{1}{R_2} + \frac{1}{R_3}$
- $U = U_1 = U_2 = U_3$
- $I = I_1 + I_2 + I_3$

តុល្យភាពនិងអនុតុល្យភាពនៃអំពិល

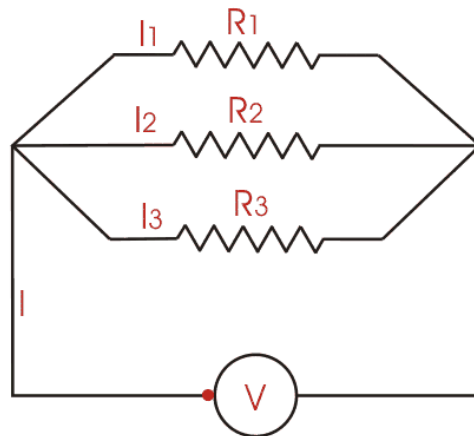
kA	A	mA	μA
1	000	000	000
	1	000	000
		1	000

តុល្យភាពនិងអនុតុល្យភាពនៃវ៉ុល

kV	V	mV
1	000	000
	1	000

5. កុងដង់សាទ័រ (capacitor)

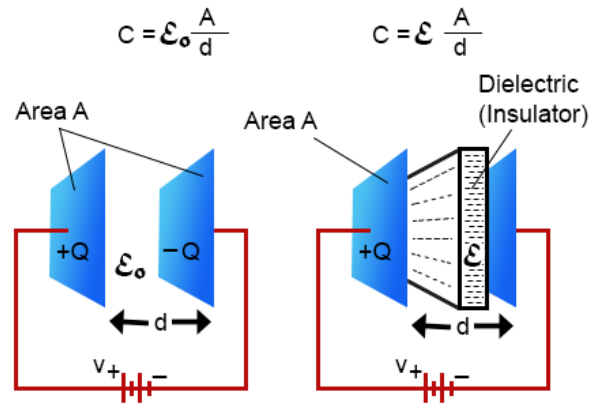
កុងដង់សាទ័រជាបន្ទុះលោហៈពីរដាក់ស្របគ្នា និងមានសូលុយស្យុងឌីអេឡិចត្រិចនៅចន្លោះបន្ទុះទាំងពីរ។ វាត្រូវបានគេប្រើដើម្បីផ្ទុក ឬផ្ទេរបន្ទុកអគ្គិសនីក្នុងរយៈពេលវែងអាស្រ័យទៅលើកាប៉ាស៊ីតេរបស់វា។ មុខងាររបស់វាស្រដៀងទៅនឹងអាគុយ ឬថ្មពិលដែរ ដោយខុសតែត្រង់វាឆាប់អស់ថ្មជាងថ្មពិលតែប៉ុណ្ណោះ ។

**ថាមពលអគ្គិសនី**

$$W = QV$$

- $W(J)$ ជាថាមពលអគ្គិសនី
- $Q(c)$ ជាបរិមាណបន្ទុកអគ្គិសនី
- $V(V)$ ជាងតង់ស្យុងឬបរិមាណអគ្គិសនី។





-ឌីអេឡិចត្រិចអាចត្រូវបានបង្កើតឡើងពីសាធាតុស្តើងមួយចំនួនដូចជា៖ក្រដាស សំឡីកែវ ជ័រកៅស៊ូ សេរ៉ាមិច ឬ វត្ថុទាំងឡាយណាដែលអាចឱ្យអ៊ីយ៉ុងឆ្លងកាត់បាន។

-បន្ទះលោហៈត្រូវបានបង្កើតចេញពី៖បន្ទះអាលុយមីញ៉ូម ទីតាញ៉ូម ប្រាក់ ឬលោហៈធាតុផ្សេងៗទៀត ។

-តម្លៃ capacity មានខ្នាតគិតជា ហ្វារ៉ាត (Farad) មានរូបមន្ត

$$C = \epsilon \frac{A}{d}$$

- C : ជាកាប៉ាស៊ីតេ គិតជា (F)
- A : ផ្ទៃនៃបន្ទះលោហៈ គិតជា(m²)
- D : ប្រវែងគំលាតពីបន្ទះលោហៈមួយទៅមួយទៀត គិតជា (m)
- ϵ : dielectric's relative permittivity

ដំណើរការដែលកុងដង់សាទ័រធ្វើការផ្ទុក និងផ្ទេរចរន្តអគ្គិសនីនៅកម្រិត 60% មានរូបមន្ត

$$\tau = R.C$$

- τ : រយៈពេលផ្ទុក ឬផ្ទេររបស់កុងដង់សាទ័រនៅ 60%
- R : ផ្ទៃនៃបន្ទះលោហៈ គិតជា(Ω)

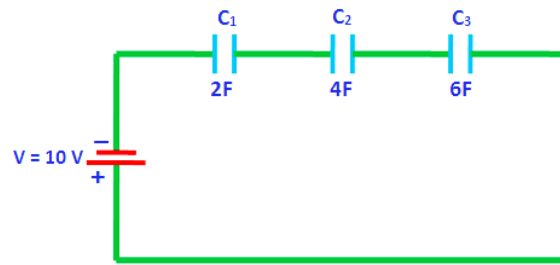
បម្រែបម្រួលចរន្តតាមខណៈ

$$i = C \frac{dv}{dt}$$

- i : អាំងតង់ស៊ីតេតាមខណៈ គិតជា (A)
- $\frac{dv}{dt}$: តង់ស្យុងបម្រែបម្រួលតាមខណៈ (V)

កុងដង់សាទ័រត្រូវបានគេផ្គុំជាស៊េរី ឬ ខ្លែងដូចអេស៊ីស្តង់ដែរ ៖

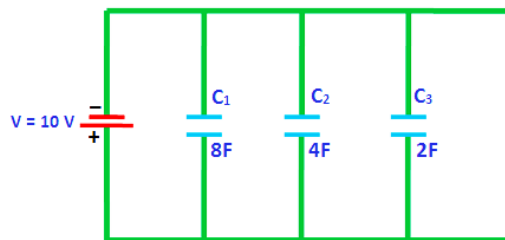
បង្គុំជាស៊េរី



Series capacitor circuit

$$\frac{1}{C} = \frac{1}{C_1} + \frac{1}{C_2} + \frac{1}{C_3}$$

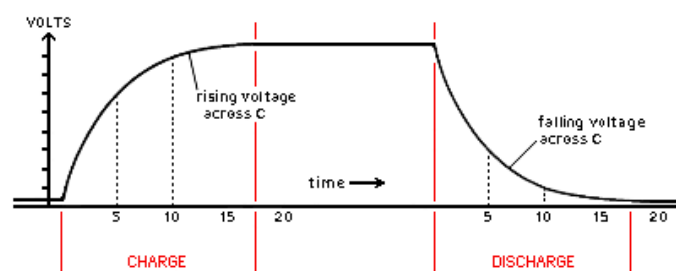
បង្គុំជាខ្លែង



Parallel capacitor circuit

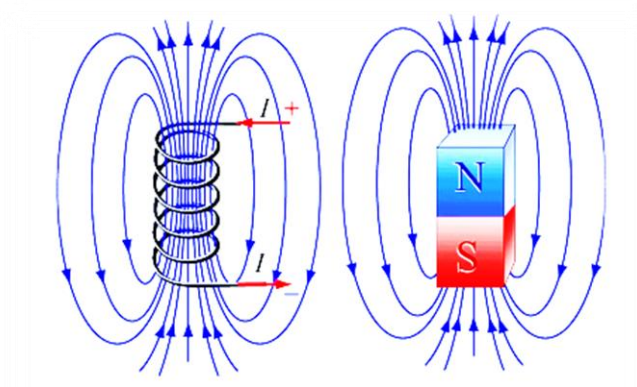
$$C = C_1 + C_2 + C_3$$

ក្រាបខាងក្រោមបង្ហាញពីទម្រង់ ផ្ទុកនិងផ្ទេររបស់កុងដង់សាទ័រ

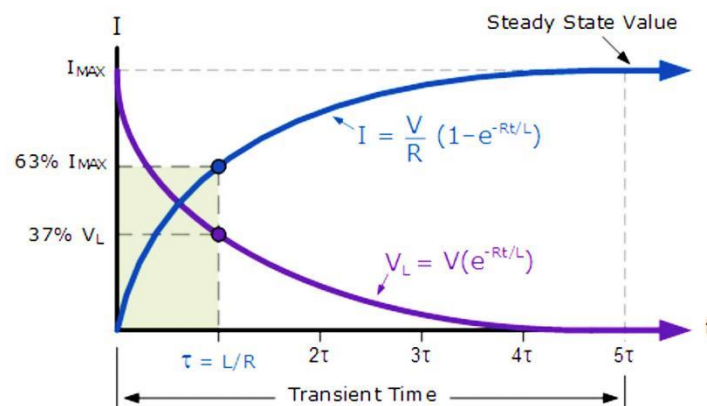


៦. អាំងឌុចតង់

វាជារប្រភេទនៃខ្សែចម្លងមានអ៊ីស៊ីឡង់ស្តើង រីឯស្មៅលើអង្គធាតុដែលអាចឱ្យមានជម្រៀបម៉ាញ៉េទិចបានដូចជាលោហៈដែក Ferrite...។ល។ នៅពេលដែលមានលំហូរចរន្តឆ្លងកាត់រប័រអាំងឌុចតង់បង្កើតដែនម៉ាញ៉េទិច ហើយដែនម៉ាញ៉េទិចនេះមិនងាយនឹងផ្លាស់ប្តូរលក្ខណៈរបស់វាទេ ។ ជាលទ្ធផលនៃលក្ខណៈពិសេសនេះអាំងឌុចតង់ផ្អាកថាមពលក្នុងរប័របស់វា ។ អាំងឌុចតង់ត្រូវបានគេប្រើសម្រាប់ការការពារចរន្ត(អាំងតង់ស៊ីតេ) ពីការធ្វើបម្រែបម្រួលរបស់វា ។ ឧទាហរណ៍នៅពេលដែលចរន្តឆ្លងកាត់ អាំងឌុចតង់ប្រែប្រួលឬក៏ធ្លាក់ចុះ វានឹងខំប្រឹងបង្កើតកម្លាំងមួយរក្សាចរន្តឱ្យនៅថេរ ។

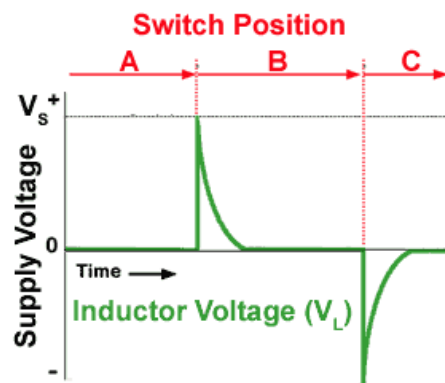


ទោះបីជាវាប្រឆាំងនឹងបម្រែបម្រួលចរន្តក៏ពិតមែន តែវានៅតែផ្តល់ភាពងាយស្រួលដល់ចរន្តជាប់ឆ្លងកាត់វានៅ Steady State ។ រូបខាងក្រោមបង្ហាញពី Steady State របស់អាំងឌុចតង់



ចរន្ត និងតង់ស្យុងតាមខណៈនៅ Transient State របស់អាំងឌុចតង់មានរូបមន្ត

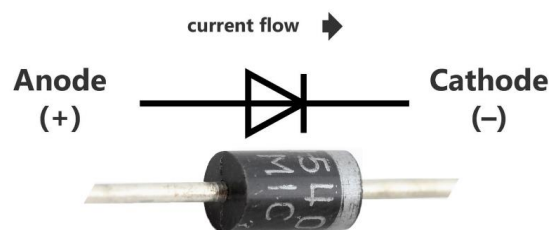
$$V(t) = -L \frac{di}{dt}$$



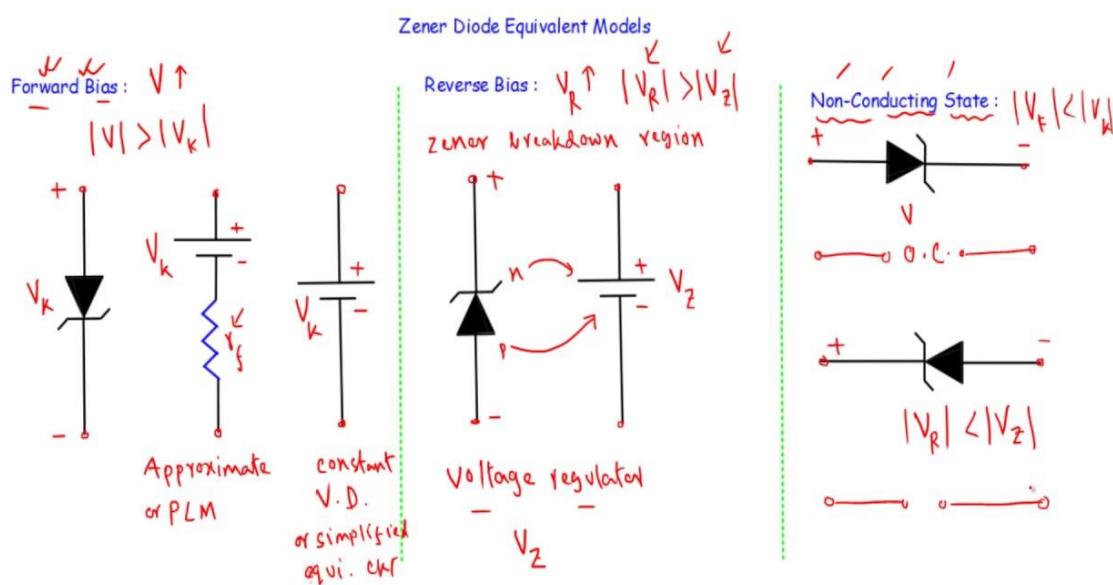
អាំងឌុចតង់អាចត្រូវបានគេផ្ដើជាស៊េរី និងខ្លែងដូចស៊ីស្តង់ ហើយមានរូបមន្តដូចគ្នាផងដែរ ។

7. ឌីយ៉ូត និង LED

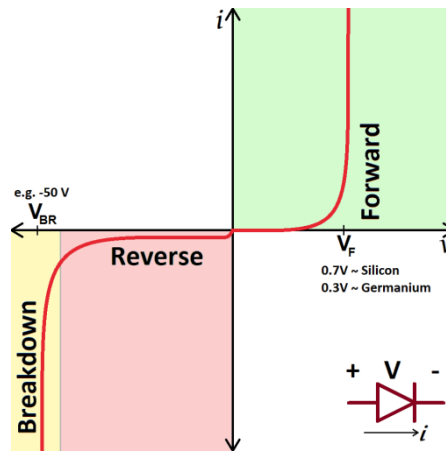
ឌីយ៉ូតជា Semi-Conductor ដែលមានមុខងារចម្លងចរន្តអគ្គិសនីក្នុងទិសដៅមួយជាក់លាក់ និងអាចនិយាយបានថាមានប៉ូលកម្ម ។



ខាងក្រោមនេះជាសៀគ្វីសមមូលនៃប៉ូលកម្មរបស់ឌីយ៉ូត ៖



ការសិក្សាអំពីឌីយ៉ូតត្រូវបានគេសិក្សាផ្ដោតជាសំខាន់លើទំនាក់ទំនងរវាងចរន្ត និងតង់ស្យុង ។ ខាងក្រោមនេះបង្ហាញពីក្រាបនៃទំនាក់ទំនង ៖



យោងទៅតាមតង់ស្យុងនៃប្រភពឌីយ៉ូតនឹងប្រតិបត្តិការចែកជាបីតំបន់ ៖

- **Forward bias** : នៅពេលដែលតង់ស្យុងឆ្លងកាត់ឌីយ៉ូតវិជ្ជមាននោះឌីយ៉ូតបើក ហើយចរន្តអាចឆ្លងកាត់បាន ។ ដើម្បីឱ្យចរន្តឆ្លងកាត់បានអ្នកត្រូវផ្គត់ផ្គង់តង់ស្យុងឱ្យធំជាង V_F ជាធម្មតា $V_F=0.5V$ ហើយយើងអាចកំណត់វាបានតាមរយៈការប្រើប្រាស់ឧបករណ៍វាស់ ឬមើលលើសៀវភៅរបៀបប្រើប្រាស់វាដែលរកបានតាមរយៈប្រព័ន្ធ Internet ។



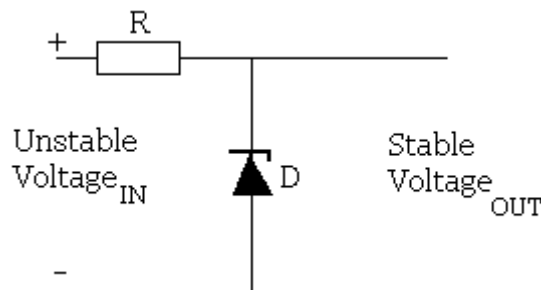
- **Reverse bias** : វាបិទនៅពេលដែលតង់ស្យុងទាបជាង V_F ហើយធំជាង V_{BR} ។ ចរន្តត្រូវបានរាំងស្ទះខណៈដែលឌីយ៉ូតស្ថិតក្នុងប៉ូលកម្មប្រាស ។ បើពិនិត្យមើលរូប នៅ 6.អាំងឌុចតង់ ចំណុច Steady State យើងឃើញថាចរន្តក្នុងតំបន់នេះពុំស្មើសូន្យនោះទេវាមានតម្លៃមួយគិតជា nA ដែលចរន្តនេះត្រូវបាន គេហៅថាចរន្តផ្ដុំត (Saturation) ។
- **Breakdown** : វា កើតឡើងក្នុងពេលដែលតង់ស្យុងខ្ពស់ជាង $50V$ ទោះស្ថិតក្នុងប៉ូលកម្មប្រាសក៏ដោយ ក៏ចរន្តមួយចំនួននៅតែអាចឆ្លងកាត់ឌីយ៉ូតបានដដែល ។

- ✓ Schottky Diode ជាប្រភេទឌីយ៉ូតដែលធ្វើឱ្យតង់ស្យុង V_F ធ្លាក់ចុះតិច ។ វាមានតម្លៃចន្លោះពី 0.15V ទៅ 0.45V ។ វាត្រូវបានគេប្រើក្នុងសៀគ្វីទាំងឡាយណាដែលតម្រូវឱ្យបាត់បង់នូវតង់ស្យុងទាបលើឌីយ៉ូតពេលវាដំណើរការ ។

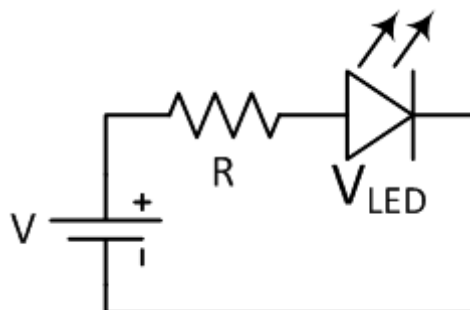


Schottky diode symbol

- ✓ Zener Diode វាក៏ដូចជាឌីយ៉ូតធម្មតាដែរ ប៉ុន្តែវាត្រូវបានគេបង្កើតសម្រាប់ប្រើប្រាស់ក្នុងការចម្លងនៅប៉ូលកម្មប្រាស ។ វាត្រូវបានគេផលិតឡើងដោយធ្វើឱ្យតង់ស្យុង Breakdown របស់វាមានភាពលំអៀងមហាសែនតិចបំផុត ។ តង់ស្យុងនោះត្រូវបានគេហៅថា Breakdown Voltage ឬ Zener Voltage ។ នៅពេលដែលចរន្តគ្រាប់គ្រាន់បានឆ្លងកាត់វាពេលនោះតង់ស្យុងនៅចន្លោះ Zener ត្រូវបានស្ថិតក្នុងសភាពថេរ ឬតង់ស្យុងស្មើតង់ស្យុង Zener ។ ទាញចេញពីគុណសម្បត្តិនៃ Breakdown Voltage នេះវាត្រូវបានគេប្រើក្នុងសៀគ្វីអេឡិចត្រូនិចជាតង់ស្យុងធៀប ហើយក៏ត្រូវបានគេប្រើជាប្រភពផ្គត់ផ្គង់តង់ស្យុងថេរមួយសម្រាប់ប្រព័ន្ធសៀគ្វីតូចៗផងដែរ (Regulator Voltage) ។ ប៉ុន្តែវាមិនអាចប្រើជាប្រភពនៃសៀគ្វីមួយដែលមានតម្រូវការចរន្តឆ្លងកាត់ធំនោះទេ ។



អំពូល LED ក៏ជាឌីយ៉ូតដែរ វាអាចបញ្ចេញពន្លឺបាន ។ ជាធម្មតា LED មាន V_F ពី 0.6V-1.7V និងចរន្តអតិបរមាប្រហែល 20mA ។



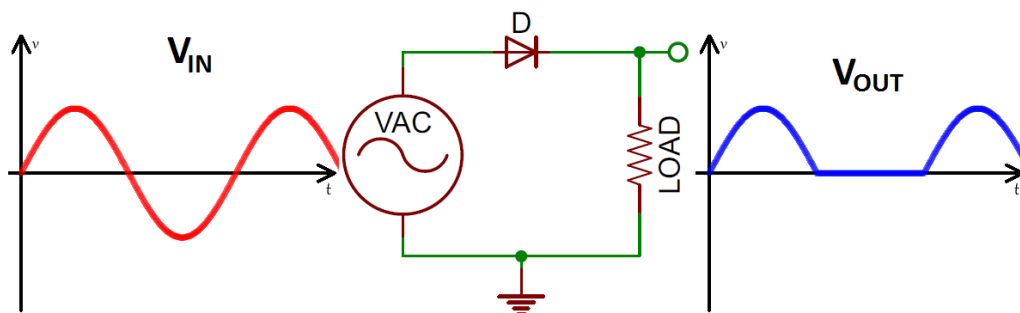
រេស៊ីស្តង់ R ត្រូវបានគេប្រើសម្រាប់កម្រិតតម្លៃអាំងតង់ស៊ីតេចរន្តដែលហូរឆ្លងកាត់ LED គឺធ្វើយ៉ាងណាមិនឱ្យលើសពី 20mA ។



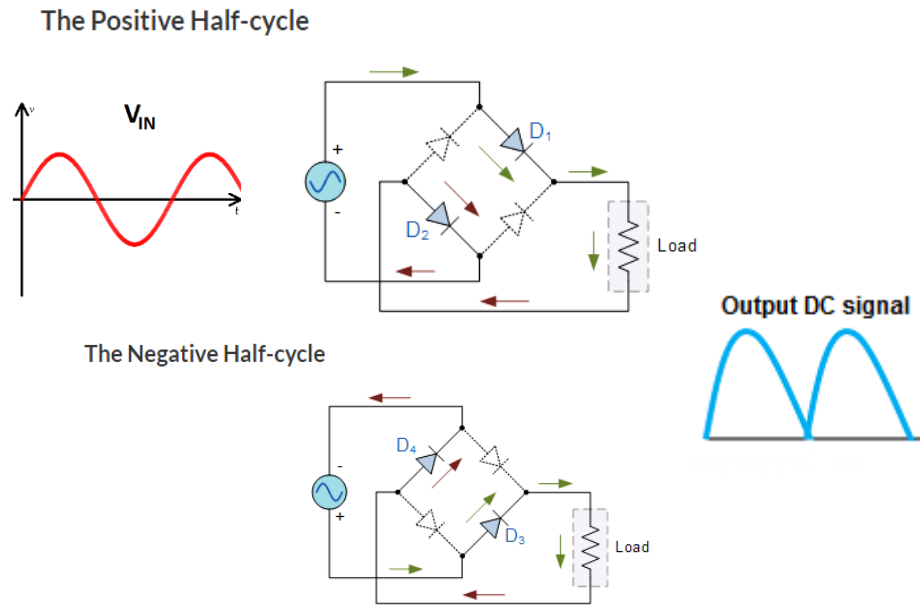
✓ Rectifiers Diode

ប្រសិនបើអ្នកមានឧបករណ៍អេឡិចត្រូនិចណាមួយខូចដូចជា ឆ្នាំងសាកទូរសព្ទជាដើម ហើយអ្នកគាត់វាមើល នោះអ្នកនឹងឃើញឌីយ៉ូតនៅខាងក្នុង ។ Rectifiers ជាសៀគ្វីដែល បំប្លែងពីចរន្តឆ្លាស់ឱ្យទៅជាចរន្តជាប់ ។ ចរន្តឆ្លាស់ត្រូវបានផ្គត់ផ្គង់ពីរោងចក្រអគ្គិសនីចូល មកក្នុងផ្ទះរបស់យើង ប៉ុន្តែឧបករណ៍ប្រើប្រាស់ក្នុងផ្ទះរបស់យើងភាគច្រើនប្រើប្រាស់ចរន្ត ជាប់ ពិសេសគឺបរិក្ខារអេឡិចត្រូនិច ដូចជាកុំព្យូទ័រ ទូរសព្ទ .. ។ល។ ចរន្តឆ្លាស់ប្តូរទិសដៅ ចរន្តរហូតខុសពីចរន្តជាប់ ដែលមានទិសដៅចរន្តតែមួយ ការប្រើប្រាស់ឌីយ៉ូតគឺអាចកំណត់ ទិសដៅឱ្យចរន្តរត់ក្នុងទិសតែមួយ ដោយសារតែវាមិនចម្លងក្នុងប៉ូលកម្មប្រាស ។ សៀគ្វី ខាងក្រោមជារបៀបដែលឌីយ៉ូត ត្រូវបានគេប្រើសម្រាប់កំណត់ទិសនៃចរន្តដែលវាចែក ចេញជាពីរបៀបខុសគ្នាគឺ៖

- Half-wave Rectifiers : ជារបៀបដែលឌីយ៉ូតធ្វើឱ្យ ទម្រង់រលកតង់ស្យុងនៃចរន្តឆ្លាស់កើត បានកន្លះខ្ទប់ ។ រូបខាងក្រោមជាដំណើរការដែលឌីយ៉ូតធ្វើឱ្យខ្ទប់នៃចរន្តឆ្លាស់នៅសល់តែ ពាក់កណ្តាល ។



- Full-wave bridge Rectifiers : ជារបៀបដែលឌីយ៉ូតធ្វើឱ្យ ទម្រង់រលកតង់ស្យុងនៃចរន្តឆ្លាស់កើតបាន មួយខ្ទប់ពេញក្នុងទិសដៅតែមួយ ។

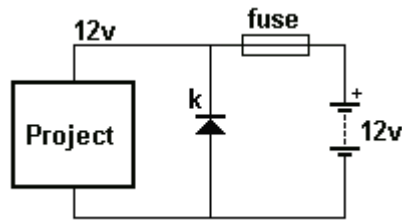


7.1.សុវត្ថិភាពភ្លើងបញ្ចាស

ការប្រើប្រាស់ឌីយ៉ូតតែមួយមិនមែនជាជម្រើសល្អបំផុតនោះទេ ឧទាហរណ៍ប្រសិនបើអ្នកតសៀវត្រីអគ្គិសនី ដោយច្រឡំប៉ូដូចជាអាកុយ ឬថ្មពិល នោះវានឹងធ្វើឱ្យឧបករណ៍ខូចមិនខាន ។ ដើម្បីដោះស្រាយនិងបញ្ហានេះអ្នកគួរប្រើហ្វុយស៊ីបក្នុងសៀគ្វីរបស់អ្នក ។



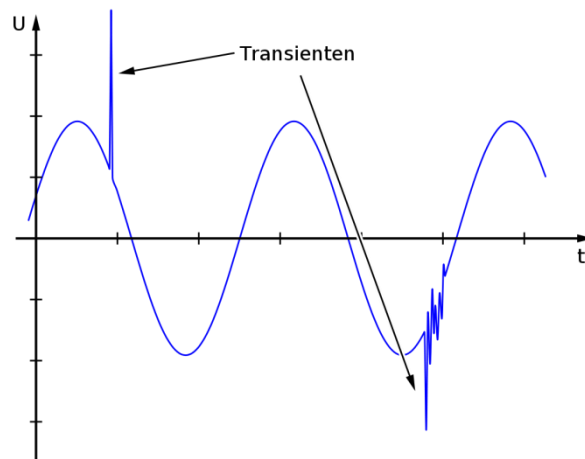
វាមានសារៈប្រយោជន៍ណាស់នៅពេលដែលអ្នកដោតភ្លើងដោយពុំមានកំណត់ចំណាំប៉ូលកម្មច្បាស់លាស់ដែលវាងាយនឹងធ្វើឱ្យអ្នកប្រើប្រាស់ភ័ន្តច្រឡំ ហើយអាចធ្វើឱ្យអេឡិចត្រូនិចរបស់អ្នកខូច ។ រូបខាងក្រោមនេះជាសៀគ្វីតាងដោយប៉ូលកម្មត្រូវបានដាក់ឱ្យបញ្ចាស ឌីយ៉ូតបានរាំងស្ទះចរន្តមិនឱ្យរត់កាត់សៀគ្វីទេ ធ្វើដូចនេះមានន័យថាសៀគ្វីរបស់អ្នកមានសុវត្ថិភាពហើយ ។



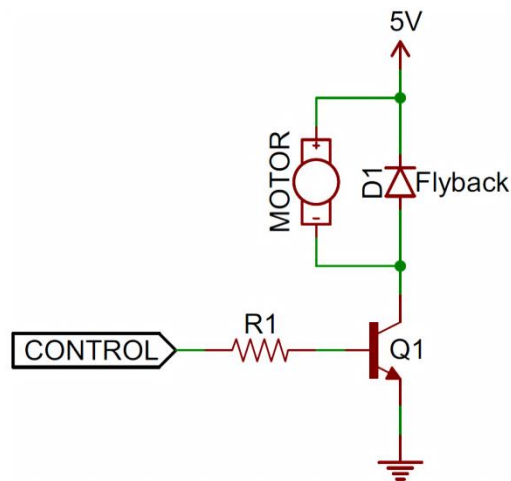
ការដាក់ឌីយ៉ូតសម្រាប់ការការពារក្នុងសៀគ្វីធ្វើឱ្យមានការបាត់បង់នូវតង់ស្យុងមួយចំនួន ដោយសារ V_F របស់ ឌីយ៉ូត ។ ដូចនេះ ការប្រើប្រាស់ Schottky Diode គឺល្អសម្រាប់សៀគ្វីបែបនេះ ព្រោះវាមាន V_F ធ្លាក់ចុះទាប ។

7.2. Flyback Diode ការការពារនឹង Voltage Spike

ឌីយ៉ូតក៏ត្រូវបានគេប្រើផងដែរសម្រាប់កំណត់កម្រិត ដែលអាចខូចខាតដោយប៉ូតង់ស្យែល ណាមួយ ពីការឡើងតង់ស្យុងដោយពុំបានព្រាងទុក ។ ការឡើងតង់ស្យុងនេះត្រូវបានគេហៅថា Voltage Spike ។



ដោយមូលហេតុនេះហើយទើបឌីយ៉ូតមួយប្រភេទត្រូវបានបង្កើតឡើង វាមានលក្ខណៈស្រដៀង គ្នានឹង Zener Diode ដែរ វាមានតង់ស្យុង V_{BR} ទាប(20V) ប៉ុន្តែមានកម្រិតអនុភាពខ្ពស់ ដែលពេល ខ្លះអាចមានដល់ទៅរាប់គីឡូវ៉ាត់ ។ វាត្រូវបានបង្កើតឡើង ដើម្បីដើរតួជា Shunt Current និង ស្រូបយកថាមពលនៅពេលដែល Voltage Spike កើតមានឡើង។ ឌីយ៉ូតដែលមានមុខងារដូច រៀបរាប់នេះមានឈ្មោះថា Flyback Diode



ជំពូកទី២

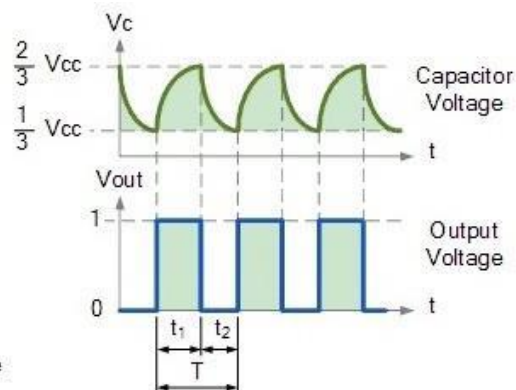
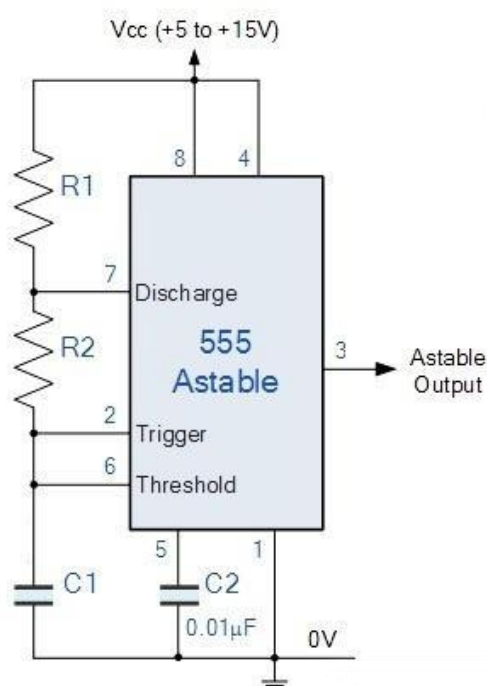
ការអនុវត្តអេឡិចត្រូនិច និងកម្មវិធី Arduino

នៅក្នុងជំពូកនេះអ្នកនឹងសិក្សាពី hard ware និង soft ware ជាពិសេសអ្នកនឹងរៀនពីការចាប់ផ្តើមសរសេរកូដងាយៗជាច្រើនតាមរយៈ Arduino Board ។

I.Digital Output

1.Blinking តាមរយៈ ICsNE555

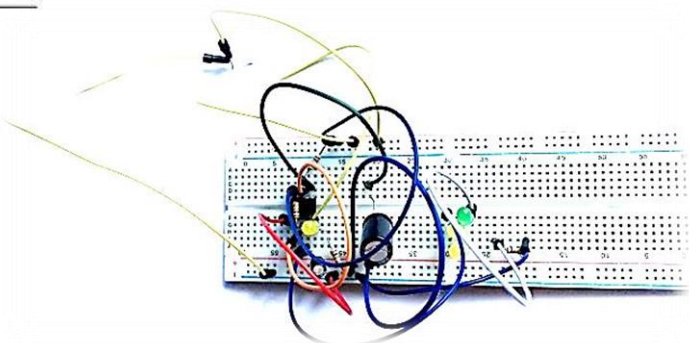
Blinking ឬក៏ភ្លើងលោតផ្លិចផ្តាសៀគ្វីងាយមួយដែលអាចធ្វើបានទម្រង់ផ្សេងៗ ។ វាអាចត្រូវបានគេបង្កើតឡើងតាមរយៈសៀគ្វីអេឡិចត្រូនិចដែលមានស្រាប់ ឬក៏ការសរសេរកូដ ។ សៀគ្វីងាយមួយដែលគេភាគច្រើនស្គាល់នោះគឺ ICsNE555 គេប្រើវាសម្រាប់ការបង្កើតសញ្ញាលំនេះ ។ ចូរអ្នកតសៀគ្វីដូចគំនូសតាងខាងក្រោម



$$t_1 = 0.693(R_1 + R_2) \cdot C$$

$$t_2 = 0.693 \times R_2 \times C$$

$$f = \frac{1}{t_1 + t_2} \quad D.C. = \frac{t_1}{t_2}$$



ក្រោយពេលដែលតសៀត្យនេះរួចគេអាចយក ខ្សែបញ្ចូលថ្មទូរស័ព្ទ(adaptor)ដែលមានចរន្ត 5v ដើម្បីសាកល្បងថាអំពូលភ្លើងដែរឬទេ ហើយរយៈប៉ុន្មានដែលវាភ្លើងនិងរលត់ ដោយធ្វើកាតកណា នា ។

នៅក្នុងសៀត្យ Oscillator ខាងលើ រយៈពេល t_1 , t_2 ជារយៈពេលផ្ទុក និងផ្ទេររឿងគ្នារបស់ កុងដង់សាទ័រ C_1 តាមរយៈរេស៊ីស្តង់ R_1, R_2 ។ កុងដង់ផ្ទុកតាមរយៈរេស៊ីស្តង់ R_1+R_2 រហូតដល់ $2/3V_{cc}$ ហើយផ្ទេរវិញតាមរយៈរេស៊ីស្តង់ R_2 ។ រយៈពេលនេះបង្កើតបានជារូបមន្តទូទៅមួយគឺ៖

$$\begin{aligned} t_1 &= 0.693 \times (R_1 + R_2) \times C_1 \\ t_2 &= 0.693 \times R_2 \times C_1 \end{aligned}$$

ហ្វេកង់នៃរយៈពេលនេះមានរូបមន្ត
$$f = \frac{1.44}{(R_1 + R_2) \times C_1}$$

Duty Cycle% របស់វាគឺ
$$\frac{R_1 + R_2}{R_1 + 2R_2}$$

តាមរយៈរូបមន្តខាងលើអ្នកអាចធ្វើបម្រែបម្រួលរយៈពេល ON និង OFF បានដោយធ្វើបម្រែបម្រួលតម្លៃណាមួយនៃរេស៊ីស្តង់ទាំងពីរ ឬតម្លៃរបស់កុងដង់សាទ័រ ។

II. Arduino

Arduino គឺជាប្រព័ន្ធអេកូផ្នែក hardware និង Software ដែលជាកម្មវិធីបើកទូលាយឈាន មុខគេនៅក្នុងពិភពលោក ។ ក្រុមហ៊ុនរបស់ Arduino ផ្តល់ជូននូវកម្មវិធី និងឯកសារជាច្រើន ដែល អាចឱ្យនរណាម្នាក់ថ្លៃប្រឌិតជាមួយនិងបច្ចេកវិទ្យា ។

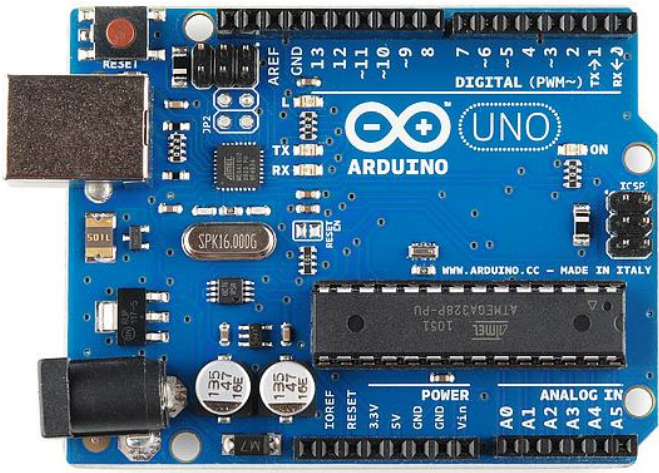
Arduino គឺជាឧបករណ៍ដ៏ពេញនិយមមួយសម្រាប់ការអភិវឌ្ឍផលិតផលច្រើន ព្រមទាំងឧបករណ៍ មួយដែលទទួលបានជោគជ័យបំផុតសម្រាប់ការអប់រំ STEM / STEAM ។ អ្នករចនា វិស្វករ សិស្ស និស្សិត អ្នកបង្កើតនិងអ្នកផលិត រាប់ពាន់នាក់នៅជុំវិញពិភពលោកកំពុងប្រើប្រាស់ Arduino ដើម្បី ថ្លៃប្រឌិត តន្ត្រី ហ្គេម ឧបករណ៍ផ្ទះឆ្លាតវៃ កសិដ្ឋាន រថយន្តស្វ័យគំនិត និងអ្វីៗជាច្រើនទៀត។ ដើម ឡើយវាគ្រាន់ជាគម្រោងស្រាវជ្រាវដោយ Massimo Banzi, David Cuartielles, Tom Igoe, Gianluca Martino និង David Mellis នៅវិទ្យាស្ថាន Interaction Design Ivrea នៅដើមទសវត្សរ៍ឆ្នាំ 2000 តែ ប៉ុណ្ណោះ ។ វាត្រូវបានបង្កើតឡើងនៅលើគម្រោងកែច្នៃដែលជាភាសាមួយសម្រាប់រៀនពីរបៀប ក្នុងការដាក់កូដនៅក្នុងបរិបទនៃសិល្បៈទស្សន៍វិស័យដែលបានអភិវឌ្ឍដោយ Casey Reas និង

Ben Fry ក៏ដូចជាគម្រោងនិក្ខេបបទដោយ Hernando Barragan ។ ស្ថាបនិកនៃ Arduino គឺលោក Massimo Banzi និង លោក David Cuartielles បានបង្កើតឡើងក្នុងគោលបំណងធ្វើឱ្យមានតម្លៃថោក និង ងាយស្រួលសម្រាប់មនុស្សទូទៅ ។ តើ Arduino ជាអ្វី ?

Arduino ជាបង្កើតសៀគ្វីរបស់ Microconller Atmel ដែលវាផ្តល់ឱ្យអ្នកប្រើប្រាស់ទូទៅមានភាពងាយស្រួល ក្នុងការបន្តសៀគ្វីដោយពុំចាំបាច់មានគ្រឹះផ្នែកអេឡិចត្រូនិចពីមុនមក ។ ក្នុងគេហទំព័រ Arduino ផ្តល់នូវ គំរូកូដ សៀគ្វីស្រាប់ និង Library សម្រាប់ប្រើជាមួយឧបករណ៍អេឡិចត្រូនិចជាច្រើន ដែលគេបង្កើតឡើងសម្រាប់ តម្រូវការភ្ជាប់ជាមួយ Arduino ស្រាប់ដោយពុំស្មុគស្មាញ ។

ក្រុមប្រឹក្សាភិបាល Arduino ដំបូងត្រូវបានបង្កើតឡើងក្នុងឆ្នាំ 2005 ដើម្បីជួយកែលម្អនិស្សិតដែលមិនមានបទពិសោធពីមុនក្នុងការផលិតអេឡិចត្រូនិច ឬកម្មវិធីត្រួតពិនិត្យ (microcontroller) ដើម្បីបង្កើតគំរូការងារដែលតភ្ជាប់ពិពិភពលោកទៅនឹងពិភពឌីជីថល ។ ចាប់តាំងពីពេលនោះមកវាបានក្លាយជាឧបករណ៍គំរូអេឡិចត្រូនិចដ៏ពេញនិយមបំផុតដែលត្រូវបានប្រើដោយវិស្វករនិងសាជីវកម្មធំៗ ។

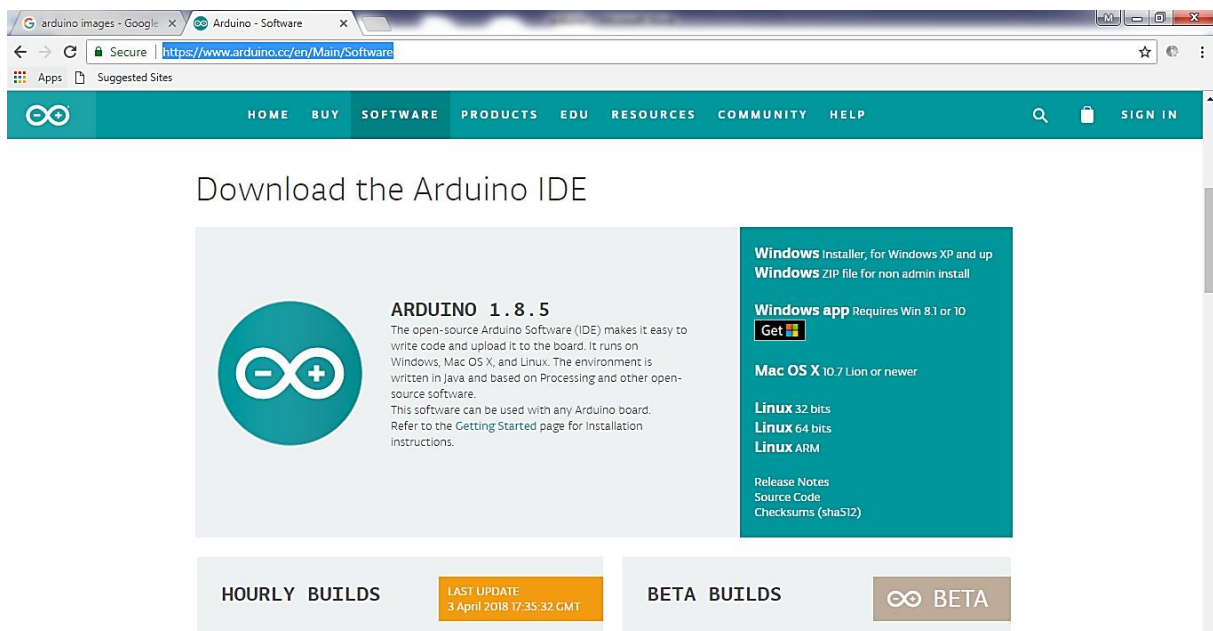
ចាប់តាំងពីគ្រឹះរបស់គម្រោង Arduino ក្រុមប្រឹក្សាអភិវឌ្ឍន៍ថ្មីនិងបណ្តាលយផ្នែកទន់ត្រូវបានណែនាំឱ្យពង្រីកលទ្ធភាពដែលអាចរកបានសម្រាប់សហគមន៍ ។ សព្វថ្ងៃនេះជាងមួយទសវត្សរ៍មកហើយ Arduino នៅតែបន្តផ្តល់នូវប្រភពកូដចំហនិងសូហ្វវែរដើម្បីនាំយកគំនិតថ្មីៗទៅនឹងជីវិតរស់នៅប្រចាំថ្ងៃ ។ ការបើកចំហនិងភាពងាយស្រួលក្នុងការប្រើប្រាស់គម្រោងបាននាំឱ្យមានការអនុម័តយ៉ាងធំធេងនូវគម្រោងអេឡិចត្រូនិចដែលមានមូលដ្ឋានលើឧបករណ៍បញ្ចេញខ្នាតតូចនិងជាកត្តាជំរុញដល់ការបង្កើតចលនារបស់អ្នកបង្កើត ។ Arduino បានក្លាយជាជម្រើសទី 1 សម្រាប់ក្រុមហ៊ុនផលិតអេឡិចត្រូនិចជាពិសេសសម្រាប់ការអភិវឌ្ឍដំណោះស្រាយសម្រាប់ទីផ្សារ IoT ដែលត្រូវបានគេព្យាករណ៍ថានឹងក្លាយជាទីផ្សារ 6 ពាន់ពាន់លាននៅឆ្នាំ 2021 ។



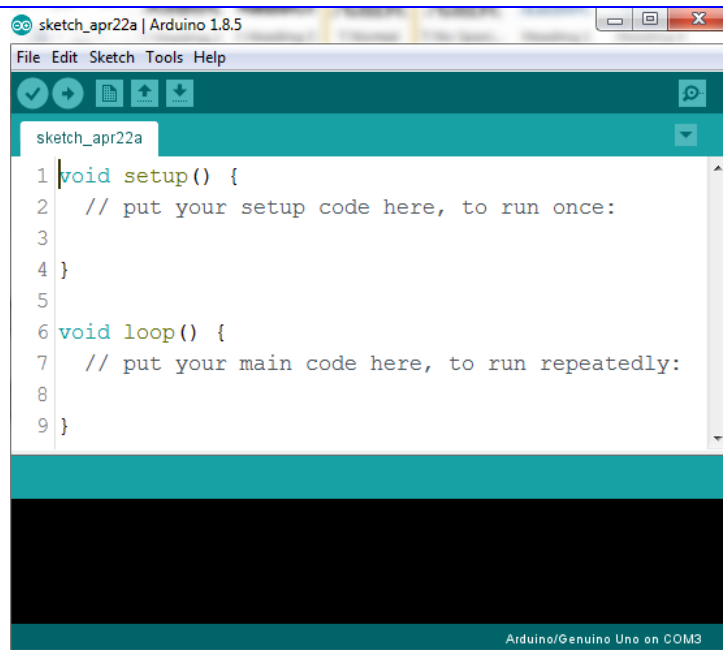
ថ្វីបើ Arduino ជាប្រភេទ Board ដែលមានសៀគ្វីស្រាប់ៗមែនតែវាក៏មិនអាចធ្វើការដោយខ្លួនឯងបានដែរព្រោះវាគ្រាន់តែជាខ្នាតរក្សាលអេឡិចត្រូនិចមួយតែប៉ុណ្ណោះ ម៉្លេះហើយវាត្រូវធ្វើការជាមួយសៀគ្វីអេឡិចត្រូនិចដទៃទៀត ដែលប្រៀបដូចជាដៃនិងជើងរបស់មនុស្សយើងដែរ ដើម្បីឱ្យប្រតិបត្តិការនៃដំណើរការរបស់វាបានពេញលេញ ។ ជាមួយគ្នានោះដែរ ឥទ្ធិពលនៃសញ្ញាលំខានពីក្រៅ (Nois) ក្រោមរូបភាពណាមួយក៏អាចបង្អាក់ដំណើរការត្រឹមត្រូវរបស់ Arduino បានដែរ ។ នៅក្នុងមេរៀននេះអ្នកនឹងទទួលបានការសិក្សាមូលដ្ឋានគ្រឹះនៃការសរសេរកូដ ។

1.ការបង្ហាញអំពី IDE Arduino

ដើម្បីទាញយកកម្មវិធី Arduino មកប្រើប្រាស់បានក្នុងកុំព្យូទ័ររបស់អ្នកបាន អ្នកត្រូវចូលទៅកាន់គេហទំព័រ Arduino ដែលមានឈ្មោះថា <https://www.arduino.cc/en/Main/Software> (ទាញយកកម្មវិធីដោយឥតគិតថ្លៃ)



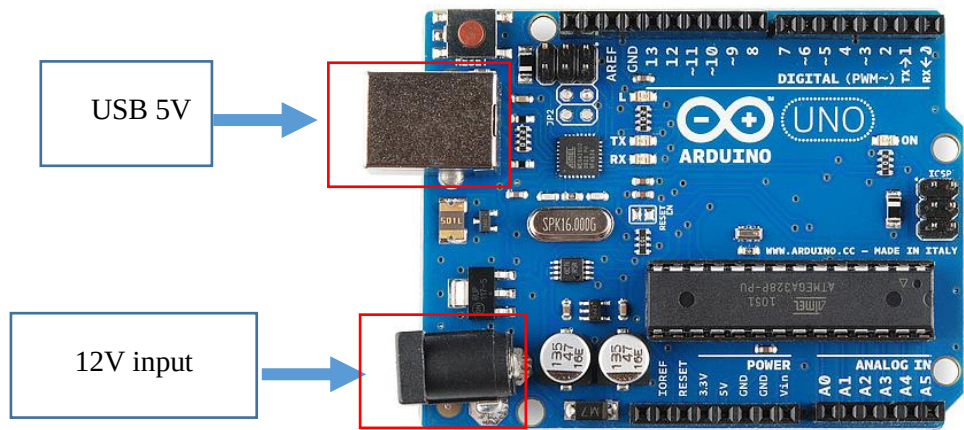
ក្រោយពេលអ្នកដំឡើងកម្មវិធី Arduino រួច អ្នកនឹងអាចប្រើវាបាន ដោយគ្រាន់តែបើកវាដែលមាននៅលើ desktop បន្ទាប់មកវានឹងបង្ហាញទម្រង់កម្មវិធីដូចខាងក្រោម



- Verify: ប្រើសម្រាប់ផ្ទៀងផ្ទាត់កូដដែលសរសេរខុសឬអត់
- Upload: ប្រើសម្រាប់បញ្ជូនកូដទៅក្នុង Arduino Board
- New Tab: ប្រើសម្រាប់បង្កើតផ្ទាំងកូដថ្មី
- Open: ប្រើសម្រាប់ទាញយកកូដសរសេររួចហើយ ដែលបានរក្សាទុកមកនៅត្រង់ណាមួយ
- Save: ប្រើសម្រាប់រក្សាទុកកូដនៅត្រង់កន្លែងណាមួយនៃកុំព្យូទ័រ
- Serial Monitor: ប្រើសម្រាប់បញ្ជាឬទទួលទិន្នន័យរវាង Arduino និង កុំព្យូទ័រ
- Code goes here: គ្រប់កូដទាំងអស់
- Console: ស្ថានភាព ឬ ប្រតិបត្តិការទាំងឡាយ រួមទាំង Error ត្រូវបានបង្ហាញ
- void setup(){} : កូដទាំងឡាយណាដែលស្ថិតក្នុង void setup() គឺធ្វើការតែម្តងគត់នៅពេលដែលយើងភ្ជាប់ភ្លើងឱ្យវា ។
- void loop(){} : កូដរបស់យើងធ្វើការដដែលៗនៅក្នុងនេះជាមួយគ្នា

2.លក្ខណៈរបស់បន្ទះ Arduino Uno

នៅក្នុងលើ Arduino Uno មាន input ចំនួនពីរគឺ USB 5V និង 12V input



គួរបញ្ជាក់ថាថាមពលអគ្គិសនីដែលយកទៅចិញ្ចឹម Arduino Uno មិនអាចមានតង់ស្យុងលើពី 20V បានឡើយ ។

USB Port ត្រូវមានចរន្តអតិបរមា 500mA បើលើសពី Fuse របស់វានឹងដាច់ ហើយ USB port ក៏ត្រូវបានប្រើសម្រាប់ការបញ្ចូលកូដទៅកាន់ Arduino ផងដែរ ។



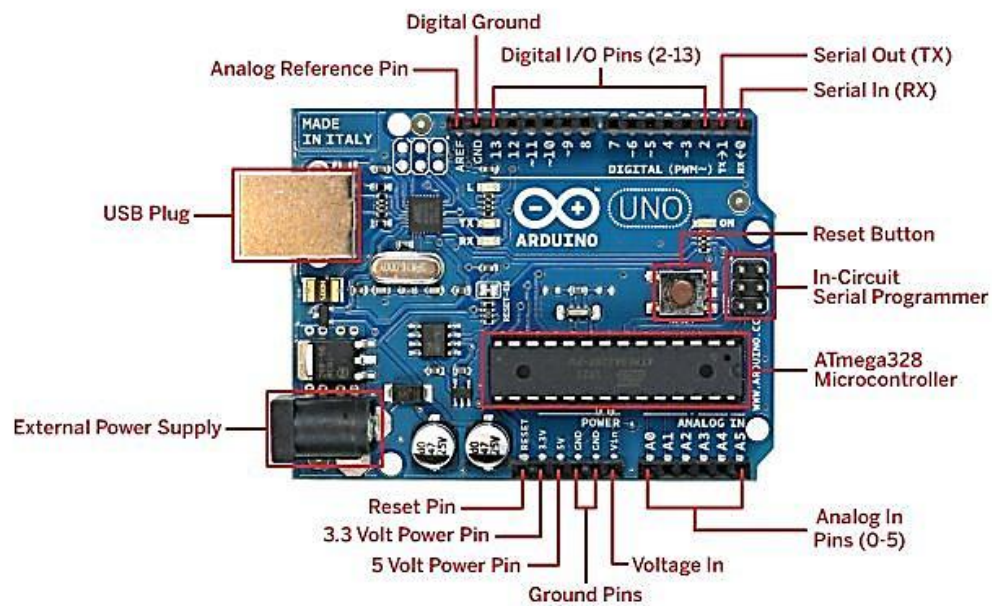
(រូប ក)

ក្នុងរូប (ក) បង្ហាញពីជើង Power Supply(I/O) របស់វា និងជើង Analog Input ដែលយើងនឹងបង្ហាញមុខងាររបស់វានៅក្នុងមេរៀនក្រោយ(មេរៀនទី៧) ។ ជើង Analog Input នីមួយៗមិនអាចមានតង់ស្យុងចូលលើពី 5V ឡើយ ។



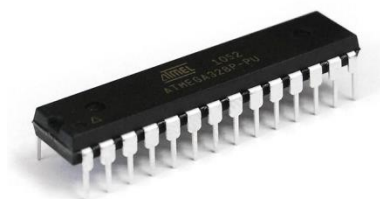
(រូប ខ)

ក្នុងរូប (ខ)បង្ហាញពីជើង Digital I/O របស់វាដែលមាន ១៣ ជើង ។ ជើងខ្លះក្នុងចំណោមជើងទាំងនេះមាន មុខងារពីរគឺជើងដែលមានសញ្ញាទិករកល(~) វាអាចត្រូវបានគេប្រើសម្រាប់បង្កើតសញ្ញាល់ PWM ដែលនឹងមាននៅមេរៀនក្រោយ ។ ក្នុងករណីដែលជើង Digital I/O របស់វាត្រូវបានប្រកាសជាជើង Output ចរន្តអតិបរមាដែលអាចផ្គត់ផ្គង់ឱ្យគ្រឿងទទួលបានគឺត្រឹម 20mA ប៉ុណ្ណោះ បើសិនជាលើសនឹងធ្វើឱ្យ Arduino Pin ឬ Board របស់អ្នកខូចពុំខាន ។



Technical specs

Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage (recommended)	7-12V
Input Voltage (limit)	6-20V
Digital I/O Pins	14 (of which 6 provide PWM output)
PWM Digital I/O Pins	6
Analog Input Pins	6
DC Current per I/O Pin	20 mA
DC Current for 3.3V Pin	50 mA
Flash Memory	32 KB (ATmega328P) of which 0.5 KB used by bootloader
SRAM	2 KB (ATmega328P)
EEPROM	1 KB (ATmega328P)
Clock Speed	16 MHz
Length	68.6 mm
Width	53.4 mm
Weight	25 g



3. Blinking ភ្លេងរំពងៈ Arduino

3.1.អញ្ញាត(Variable)

ប្រភេទ	ចំនួន Byte	តម្លៃ	ការប្រើប្រាស់
int	2	-32768 ទៅ 32768	ជាចំនួនលេខគត់ដែលមានតម្លៃដូចខាងឆ្វេង
unsigned	2	0 ទៅ 65535	ជាចំនួនលេខគត់ដែលមិនមានតម្លៃអវិជ្ជមាន
long	4	-2147483648 ទៅ 2147483647	ជាចំនួនលេខគត់ដែលមានតម្លៃធំជាង int
unsigned long	4	0 ទៅ 4294967295	ជាចំនួនលេខគត់ដែលមានតម្លៃធំជាង int និងគ្មានតម្លៃអវិជ្ជមាន
float	4	-3.4028235E+38 ទៅ 3.4028235E+38	សម្រាប់ប្រើតម្លៃទសភាគ
double	8	-3.4028235E+38 ទៅ 3.4028235E+38	សម្រាប់ប្រើតម្លៃទសភាគដែលមានល្បឿនលឿន
byte	1	0 ទៅ 255	លេខគត់តម្លៃតិចជាង 255
boolean	1	ពិត(1) និង មិនពិត(0)	តម្លៃតក្កវិទ្យា

3.2.ទិសដៅដើមរបស់ Arduino Uno

ជើង Digital របស់ Arduino មានទិសដៅពីរ គឺទិសដៅចេញនិងចូល ដូច្នេះ មុននឹងប្រើជើងនេះយើងត្រូវប្រកាសទិសដៅរបស់វាជាមុនសិន ។ Syntax របស់វាគឺ

Syntax

`pinMode(pin,mode);`

ដែល pin : លេខជើងឌីជីថល (unsigned int)

mode : កំណត់ជើងឌីជីថលដំណើរការជា OUTPUT, INPUT ឬជា INPUT_PULLUP

ឧទាហរណ៍ ៖ `pinMode(9,OUTPUT);`

Mode នីមួយៗគេប្រើដើម្បី៖

OUTPUT : ប្រកាសឱ្យជើងណាមួយជាជើងចេញសញ្ញាលំដាប់ឌីជីថល

INPUT : ប្រកាសឱ្យដឹងណាមួយជាដឹងចូលសញ្ញាល់ឌីជីថល

INPUT_PULLUP : ប្រកាសដឹងណាមួយជាដឹងចូលតែមានរេស៊ីស្តង់ pull up នៅពីខាងក្នុងហើយស្រេច

3.3.Delay Time

ការពន្យារពេលនៃព្រឹត្តិការណ៍មួយដែលកំពុងកើតឡើងនៅក្នុងកូដរបស់អ្នក ជួយអ្នកងាយស្រួលពិនិត្យមើលព្រឹត្តិការណ៍នោះ ។ នៅក្នុង Arduino IDE , Delay ចែកចេញជាពីរ Syntax ៖

Syntax

`delay(ms);`
`delayMicroseconds(us);`

ដែល ms : រយៈពេលគិតជាមីលីវិនាទី (unsigned int)

us : រយៈពេលគិតជាមីក្រូវិនាទី (unsigned int)

3.4. ការបញ្ចេញសញ្ញាឌីជីថល (Digital Write)

របៀបដែល Arduino បញ្ចេញសញ្ញាល់ទៅក្រៅដើម្បីបញ្ជូនទៅកាន់អ្វីមួយត្រូវបានហៅតាមរយៈ Syntax ៖

Syntax

`digitalWrite(pin,value);`

ដែល pin : ដឹងដែលត្រូវបញ្ចេញ (unsigned int)

value : តម្លៃជាឌីជីថលដែលត្រូវចេញ ពិត (1)មិនពិត (0) ឬ HIGH,LOW

កាលពីមេរៀនមុនអ្នកបានសិក្សាពីរបៀបនៃការធ្វើឱ្យ LED Blinking តាមរយៈការប្រើ IcsNE555 គឺជាវិធីបុរាណ ហើយវាពិបាកកំណត់ពីដំណើររបស់អំពូល ហេតុនេះហើយការទើបគេបង្កើតបន្ទុះ Arduino UNO ដើម្បីងាយស្រួលក្នុងការសរសេរកូដនិងមានជម្រើសច្រើនដើម្បីឱ្យមានអន្តរកម្មជាមួយឧបករណ៍ផ្សេងៗទៀត ។

3.5. LED Blink

របៀបធ្វើឱ្យអំពូល Blink តាមរយៈការសរសេរកូដ មានទម្រង់ច្រើនរបៀបអាស្រ័យទៅនឹងការចូលចិត្តក្នុងការសរសេរដោយគ្រាន់តែបំពេញលក្ខណៈរបស់ Syntax របស់វា ។

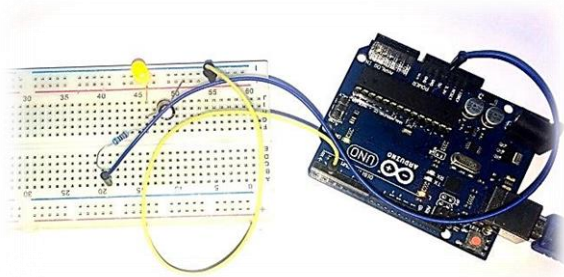
មេរៀនទី១

សម្រាប់អំពូលមួយ

`void setup(){`

```
pinMode(2,OUTPUT);}

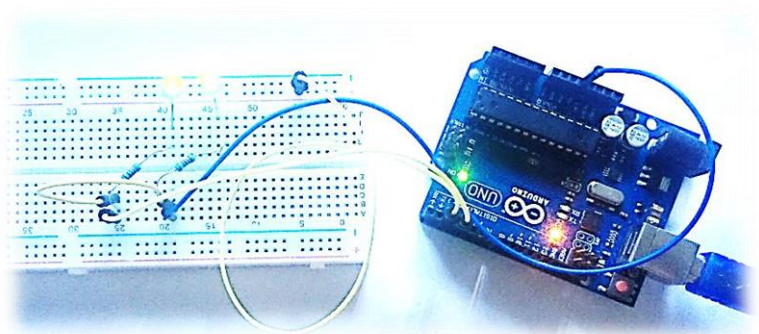
void loop(){
  digitalWrite(2,HIGH);//HIGH=1
  delay(1000);
  digitalWrite(2,0);
  delay(1000);}
```



សម្រាប់អំពូលពីរ

```
void setup(){
  pinMode(2,OUTPUT);
  pinMode(3,OUTPUT);
}

void loop(){
  digitalWrite(2,HIGH);//HIGH=1
  digitalWrite(3,HIGH);
  delay(1000);
  digitalWrite(2,0);
  digitalWrite(3,0);
  delay(1000);}
```



របៀបទី២

យើងអាច copy ហើយ past ដើម្បីបង្កើតអំពូល Blink ជាច្រើនផ្សេងទៀត តែបើសិនបើយើងចង់ប្រើអំពូលច្រើនគេអាចប្រើ for

3.6.For Loop statement

for loop ជាកន្សោមដែលមានមុខដែលមានមុខងារមួយដែលធ្វើឡើងដ៏ដែលៗក្នុងចំនួនកំណត់មួយ ដែលចំនួននោះអាចត្រូវបានគេរាប់ចុះ ឬរាប់ឡើង ។

Syntax for(initialization; condition; increment){//statement(s)}

រូបខាងក្រោមបង្ហាញអំពីដំណើរការរបស់ for loop ធ្វើការ

រង់ចាំ

ការប្រកាសអថេរ

initialize

test

កែតម្រូវ

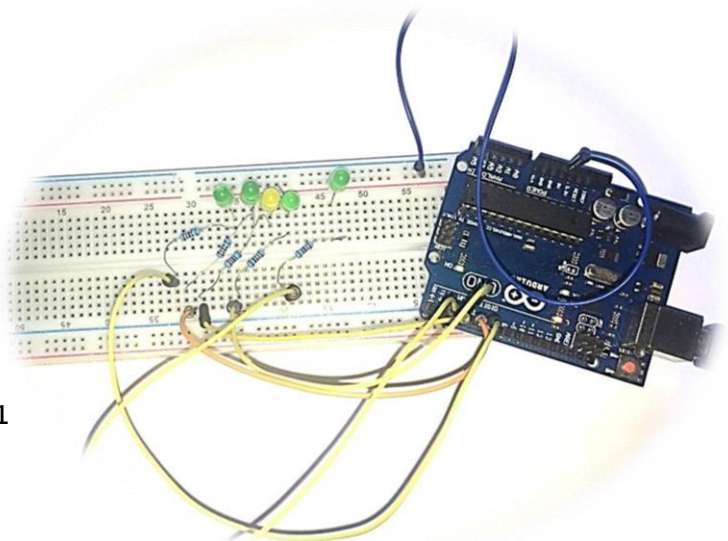
```
for(int x=0 ; x<100;x++){
    printin(x); //prints 0 to 99}
```

ឧទាហរណ៍

```
void setup(){
    pinMode(7,OUTPUT);
    pinMode(9,OUTPUT);
    for (int x=0;x<10;x++){
        digitalWrite(7,HIGH);
        delay(500);
        digitalWrite(7,LOW);
        delay(500);
    }
}
```

```
void loop(){
    digitalWrite(9,HIGH);
    delay(100);
    digitalWrite(9,LOW);
    delay(100);
}
```

```
int i;
int j;
void setup(){
    for (i=2;i<5;i++){//i=i+1
        pinMode(i,OUTPUT);}
}
void loop(){
    for(j=2;j<5;j++){
        digitalWrite(j,HIGH); //HIGH=1
        delay(100);
        digitalWrite(j,0);
        delay(100);}
```



}

ដើម្បីឱ្យវាវាត់ទៅមកគេអាចបន្ថែម

```
for(j=4;j>=2;j--){
  digitalWrite(j,HIGH);//HIGH=1
  delay(100);
  digitalWrite(j,0);
  delay(100);}
```

ដើម្បីឱ្យវាមានចលនាបន្ថែមគេអាចបន្ថែម for ទៀត

```
for(k=0;k<=9;k++){
  digitalWrite(2,HIGH);//HIGH=1
  digitalWrite(3,HIGH);
  digitalWrite(4,HIGH);
  delay(200);
  digitalWrite(2,LOW);
  digitalWrite(3,LOW);
  digitalWrite(4,LOW);
  delay(200);}
```

យើងបានកូដទាំងអស់

```
int i;
int j;
int k;
void setup(){
  for (i=2;i<=6;i++){//i=i+1
    pinMode(i,OUTPUT);}
}
void loop(){
  for(j=2;j<6;j++){
    digitalWrite(j,HIGH);//HIGH=1
    delay(1000);
    digitalWrite(j,0);
    delay(1000);}
  for(j=4;j>=2;j--){
    digitalWrite(j,HIGH);//HIGH=1
    delay(1000);
    digitalWrite(j,0);
    delay(1000);}
  for(k=0;k<=9;k++){
    digitalWrite(2,HIGH);//HIGH=1
    digitalWrite(3,HIGH);
```

```

    digitalWrite(4,HIGH);
    digitalWrite(5,HIGH);
    digitalWrite(6,HIGH);
    delay(200);
    digitalWrite(2,LOW);
    digitalWrite(3,LOW);
    digitalWrite(4,LOW);
    digitalWrite(5,LOW);
    digitalWrite(6,LOW);
    delay(200);}
}

```

គេអាចសរសេរកូដដែលមានដែលមានទម្រង់ផ្សេងៗដើម្បីឱ្យអំពូលភ្លឺជាចលនាផ្សេងៗដូចកូដខាងក្រោម

```

void setup() {
  for(int pin=2;pin<14;pin++){
    pinMode(pin,OUTPUT);
  }
}
void loop() {
  int t=10;
  //----->>>For this side
  for(int i=2; i<14; i++){
    digitalWrite(i,HIGH);
    delay(t);
    digitalWrite(i+1,HIGH);
    delay(t);
    digitalWrite(i+2,HIGH);
    delay(t);
    digitalWrite(i,LOW);
    delay(t);
    digitalWrite(i+1,LOW);
  } // <<<-----For this side
  for(int i=30; i>1; i--){
    digitalWrite(i,HIGH);
    delay(t);
    digitalWrite(i-1,HIGH);
    delay(t);
    digitalWrite(i-2,HIGH);
    delay(t);
    digitalWrite(i,LOW);
  }
}

```

```

delay(t);
digitalWrite(i-1,LOW);
    }
}

```

III.ប្រព័ន្ធគោលរោង

នៅក្នុងការសរសេរកូដ ប្រព័ន្ធគោលលេខចំនួនបី ត្រូវបានគេប្រើពេញនិយមជាងគេ នោះ ប្រព័ន្ធគោល 2 (Binary Number) ប្រព័ន្ធគោល 10 (Decimal Number) និង ប្រព័ន្ធគោល 16 (Hexadecimal Number) ។ នៅក្នុង Arduino លេខទាំងនោះត្រូវបានគេសរសេរក្នុងទម្រង់ខុសគ្នា។

ឧទាហរណ៍

byte X = B00001011; //binary number of X =11

byte X= 11; //decimal number of X=11

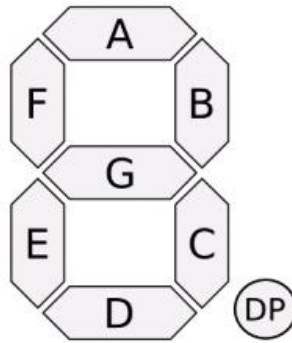
byte X=0x0b; //hexadecimal number of X=11

ឧទាហរណ៍ ប្តូរប្រព័ន្ធគោល 10 ទៅប្រព័ន្ធគោល 2 និងគោល 16

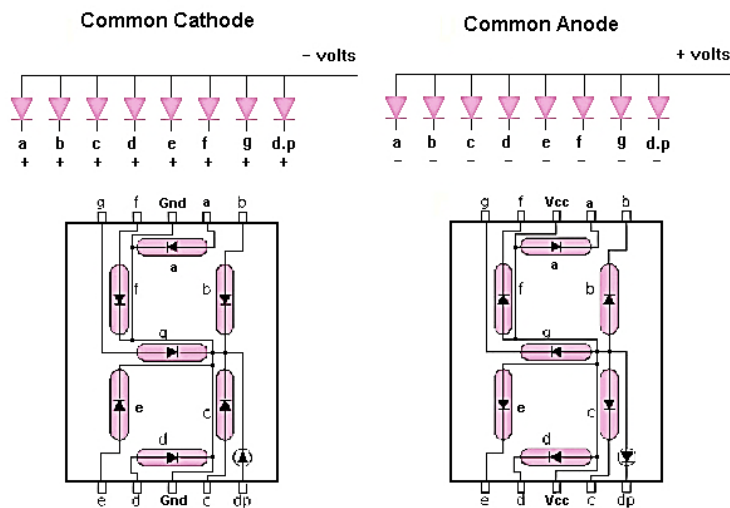
Decimal	Binary	Hexa
0		
1		
2		
3		
4		
5		
6		
7		
8		
9		

IV.អង្គភាព 7-segment

7-segment ជាតម្រៀប LED ចំនួន 7 LEDs បង្កើតបានជាទម្រង់នៃលេខឡាតាំ ពីលេខ 0 ដល់១ ដូចខាងក្រោម ៖



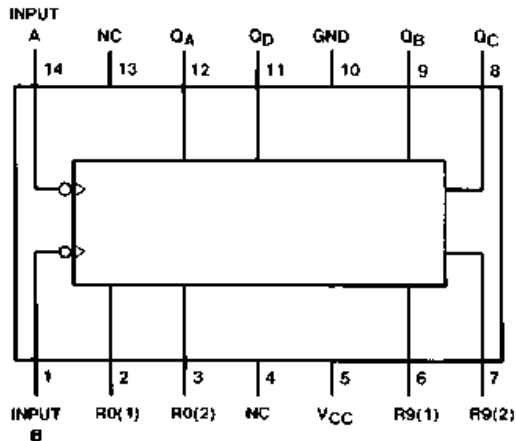
7 segment មានពីរប្រភេទគឺ **common-cathode** និង **common-anode**



Ics counter

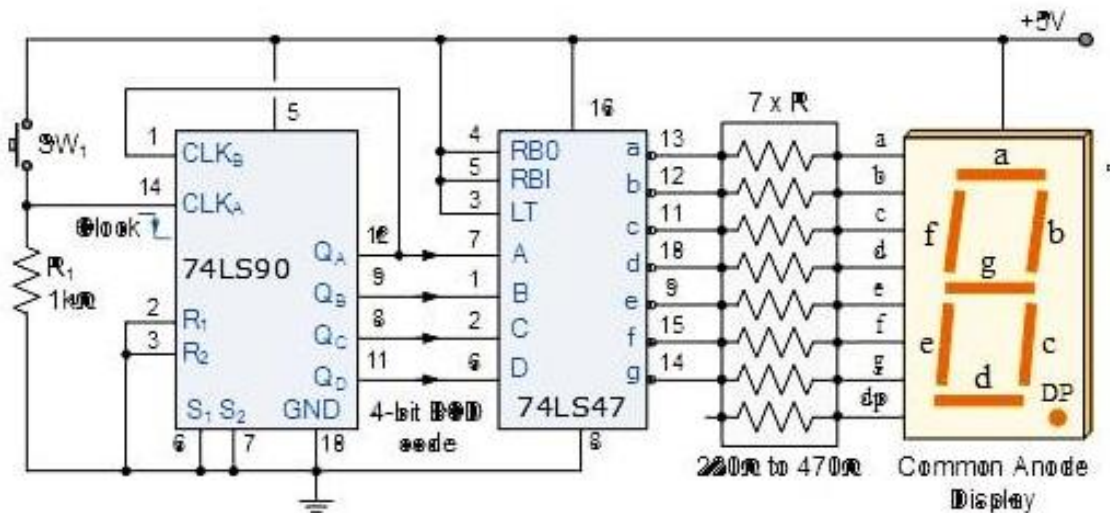
មាន ICs ពពួក TTL ជាច្រើនដែលអាចធ្វើជា counter បាន តែដើម្បីងាយស្រួលតសៀវភៅនេះ គេគួតែប្រើស៊េរីស្តង់ដារ ។

Connection Diagram



Reset/Count Truth Table

Reset Inputs				Output			
R0(1)	R0(2)	R9(1)	R9(2)	Q _D	Q _C	Q _B	Q _A
H	H	L	X	L	L	L	L
H	H	X	L	L	L	L	L
X	X	H	H	H	L	L	H
X	L	X	L	COUNT			
L	X	L	X	COUNT			
L	X	X	L	COUNT			
X	L	L	X	COUNT			



ក្នុងករណីនេះយើងនឹងយក common-anode មកធ្វើការសាកល្បងហើយភ្ជាប់ជើង Arduino ទៅនឹងជើងរបស់ 7-segment មកសិក្សាជាមុនសិន ។

ដើម្បីបានលេខឡាតាំងគេត្រូវពិចារណាថា ត្រូវឱ្យអំពូលណាខ្លះភ្លឺ

ជើងទី១ ទៅនឹងជើង a

ជើងទី៥ ទៅនឹងជើង e

ជើងទី២ ទៅនឹងជើង b

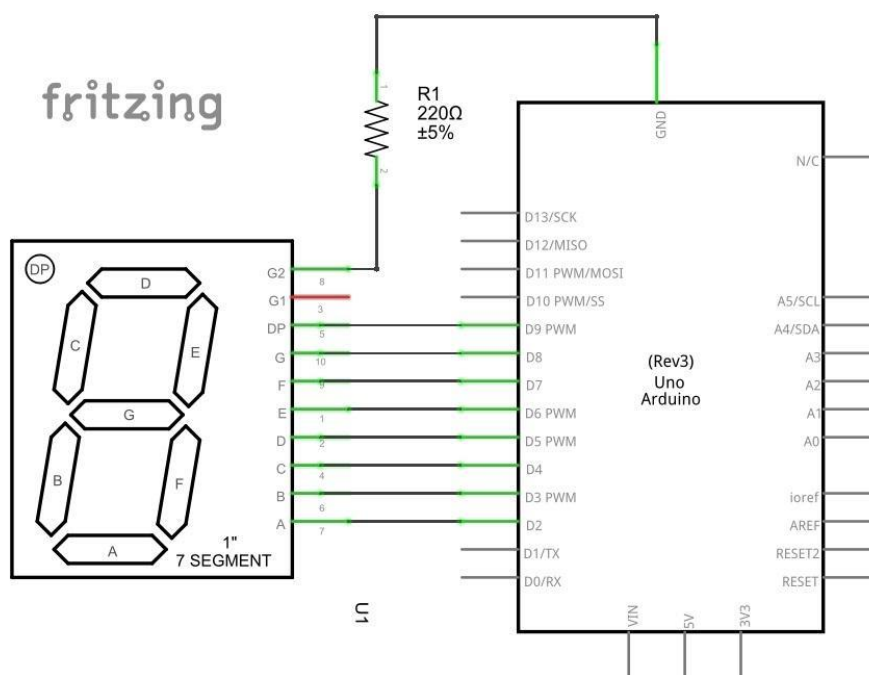
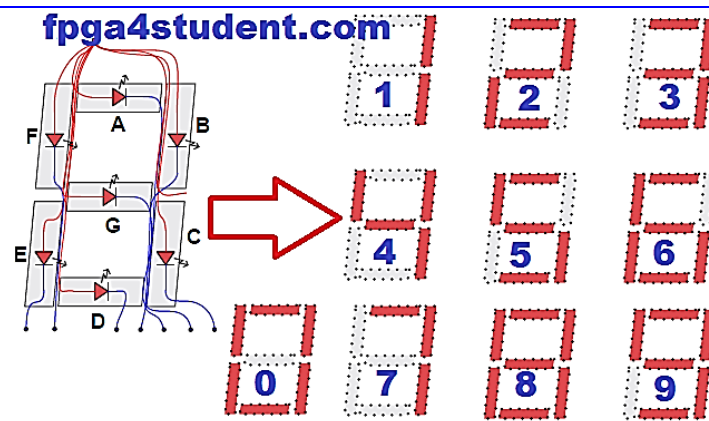
ជើងទី៦ ទៅនឹងជើង f

ជើងទី៣ ទៅនឹងជើង c

ជើងទី៧ ទៅនឹងជើង g

ជើងទី៤ ទៅនឹងជើង d

ជើង 5V ទៅនឹង Vcc ដោយភ្ជាប់រេស៊ីស្តង់



ជាដំបូងយើងត្រូវចេះតម្លៃត្រីនេះជាមុនសិន ហើយតាមរយៈការបង្ហាញជំហានខាងលើ យើងបានដូចរូបខាងក្រោម ។ ទន្ទឹមគ្នានេះដែរ ដើម្បីបានលេខ ០ ទៅ ១ គេត្រូវចេះសរសេរកូដឱ្យបានលេខ០ ជាមុនសិនបន្ទាប់មកទើបឱ្យវា countពី០ ទៅ ១ ។ ខាងក្រោមនេះជាកូដដើម្បីបង្ហាញលេខ

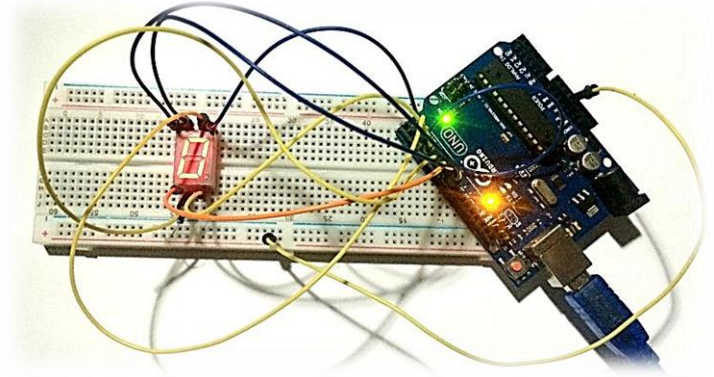
```
void setup() {
  for(int i=2;i<9;i++){
    pinMode(i,OUTPUT);}
}

void loop() { // To show number 0
  digitalWrite(2,0); //Led A Light
  digitalWrite(3,0); //Led B Light
  digitalWrite(4,0); //Led C Light
  digitalWrite(5,0); //Led D Light
```

```

digitalWrite(6,0);//Led E Light
digitalWrite(7,0);//Led F Light
digitalWrite(8,1);//Led G Light
}

```



បន្ទាប់មកយើងនឹងដាក់ឱ្យ 7-segment រត់ពីលេខ ០ ទៅលេខ ១

```

void setup() {
  for(int i=2;i<9;i++){
    pinMode(i,OUTPUT);}
}
void loop() {
  // To show number 0
  digitalWrite(2,0);//Led A Light
  digitalWrite(3,0);//Led B Light
  digitalWrite(4,0);//Led C Light
  digitalWrite(5,0);//Led D Light
  digitalWrite(6,0);//Led E Light
  digitalWrite(7,0);//Led F Light
  digitalWrite(8,1);//Led G Light
  delay(5000); // TO show number 1
  digitalWrite(2,1);//Led A Light
  digitalWrite(3,0);//Led B Light
  digitalWrite(4,0);//Led C Light
  digitalWrite(5,1);//Led D Light
  digitalWrite(6,1);//Led E Light
  digitalWrite(7,1);//Led F Light
  digitalWrite(8,1);//Led G Light
  delay(1000);//To Show number 2
  digitalWrite(2,0);//Led A Light
  digitalWrite(3,0);//Led B Light
  digitalWrite(4,1);//Led C Light
  digitalWrite(5,0);//Led D Light
  digitalWrite(6,0);//Led E Light
  digitalWrite(7,1);//Led F Light
  digitalWrite(8,0);//Led G Light
  delay(1000); //To show number 3
}

```

```

digitalWrite(2,0);//Led A Light
digitalWrite(3,0);//Led B Light
digitalWrite(4,0);//Led C Light
digitalWrite(5,0);//Led D Light
digitalWrite(6,1);//Led E Light
digitalWrite(7,1);//Led F Light
digitalWrite(8,0);//Led G Light
delay(1000);// To Show number 4
  digitalWrite(2,1);//Led A Light
  digitalWrite(3,0);//Led B Light
  digitalWrite(4,0);//Led C Light
  digitalWrite(5,1);//Led D Light
  digitalWrite(6,1);//Led E Light
  digitalWrite(7,0);//Led F Light
  digitalWrite(8,0);//Led G Light
delay(1000);//To show number 5
  digitalWrite(2,0);//Led A Light
  digitalWrite(3,1);//Led B Light
  digitalWrite(4,0);//Led C Light
  digitalWrite(5,0);//Led D Light
  digitalWrite(6,1);//Led E Light
  digitalWrite(7,0);//Led F Light
  digitalWrite(8,0);//Led G Light
delay(1000);//To show number 6
  digitalWrite(2,0);//Led A Light
  digitalWrite(3,1);//Led B Light
  digitalWrite(4,0);//Led C Light
  digitalWrite(5,0);//Led D Light
  digitalWrite(6,0);//Led E Light
  digitalWrite(7,0);//Led F Light
  digitalWrite(8,0);//Led G Light
delay(1000);//To show number 7
  digitalWrite(2,0);//Led A Light
  digitalWrite(3,0);//Led B Light
  digitalWrite(4,0);//Led C Light
  digitalWrite(5,1);//Led D Light
  digitalWrite(6,1);//Led E Light
  digitalWrite(7,1);//Led F Light
  digitalWrite(8,1);//Led G Light
delay(1000);// To show number8
  digitalWrite(2,0);//Led A Light

```

```

digitalWrite(3,0);//Led B Light
digitalWrite(4,0);//Led C Light
digitalWrite(5,0);//Led D Light
digitalWrite(6,0);//Led E Light
digitalWrite(7,0);//Led F Light
digitalWrite(8,0);//Led G Light
delay(1000);// To Show number 9
digitalWrite(2,0);//Led A Light
digitalWrite(3,0);//Led B Light
digitalWrite(4,0);//Led C Light
digitalWrite(5,0);//Led D Light
digitalWrite(6,1);//Led E Light
digitalWrite(7,0);//Led F Light
digitalWrite(8,0);//Led G Light
delay(1000);
}

```

****** ក្នុងករណីដែលយើងប្រើប្រាស់ 7-segment ជា common anode នោះលេខ 1 បិទអំពូល ហើយលេខ 0 បើកអំពូល ។

ការប្រើ Syntax ខាងលើនេះផងនិងពិបាកក្នុងការប្រើទៅលើ 7 segments ពីរខ្ទង់ ហេតុដូចនេះហើយគេត្រូវជំនួស Syntax ថ្មីមួយទៀតគឺ bitRead(x,n) ដែល x :ជាលេខដែលត្រូវយកមក អានតាម bit ហើយ n : ជាលំដាប់ bit ដែលត្រូវអាន ។ យើងត្រូវដឹងថាជើង digital ទី 0 និងទី 1 មានលក្ខណៈពិសេស ដែលពិបាក upload ក្នុងចូល ហេតុដូចនេះជាធម្មតាយើងតែងតាំង i ពីជើងទី 2 ទៅ

ការប្រើ 7-segment ខ្ទង់រាមិនមែនជាបញ្ហាទេ ប៉ុន្តែវាជាការលំបាកដែលធ្វើឱ្យ 7-segment មានលេខពីរខ្ទង់ខុសគ្នា ហើយរត់ពី 00 ទៅ 99 ។ ក្នុងករណីនេះ យើងនឹងប្រើការតាំង អាវេ ហើយឈប់ប្រើជើង 5V ដោយជំនួសជើង digital ពីរដើម្បីឱ្យវាដើរតួនាទីជាអ្នកផ្តល់ភ្លើងវិញម្តង ។

7.អង្គរ (Array)

អាវេជាការប្រមូលផ្តុំនៃអថេរប្រើក្នុងកន្សោមតែមួយ ហើយត្រូវបានគេហៅមកប្រើវិញតាមរយៈការហៅថាធាតុ ឬសន្ទទស របស់វា ។ ខាងក្រោមជាឧទាហរណ៍ខ្លះៗរបស់វា ៖

```

int myInt[6];
int myPins[]={2,4,8,3,6};
int mySensVals[6]={2,4,-8,3,2};

```

ឧទាហរណ៍

```
int mySensVals[6]={2,4,-8,3,2};
int x=mySensVals[2];// The value of x=-8
```

1.ការបង្ហាញ 7 segment តាម bitRead

វិធីសាស្ត្រក្នុងការសរសេរកូដខាងលើហាក់ប្រើប្រាស់ភាពសាមញ្ញ តែត្រូវប្រើប្រាស់ memory របស់ Arduino ច្រើន ។ យើងអាចសម្រួលការសរសេររបស់យើងឱ្យបានខ្លីជាងមុនតាមរយៈការប្រើប្រាស់ Syntax ថ្មីបន្ថែមទៀត ។ ដំបូងអ្នកត្រូវបំបែកលេខពីប្រព័ន្ធរបាស់គោល10ទៅគោល16 ជាមុនសិន ។

Number	g	f	e	d	c	b	a	Hex
0	1	0	0	0	0	0	0	0x40
1								
2								
3								
4								
5								
6								
7								
8								
9								

Syntax ថ្មីដែលនឹងយកមកប្រើប្រាស់នេះគឺ

Syntax

bitRead(x,n)

ដែល x : លេខដែលត្រូវអានយក bit

n : លំដាប់ bit ដែលត្រូវអាន

ឧទាហរណ៍

```
X = 0x70;//B00000111
```

```
Y=bitRead(x,5);//y=0
```

ជាដំបូងយើងនឹងបង្ហាញលេខ 0 តាម bitRead

```
void setup() {
  for (int i=2;i<=8;i++){
    pinMode(i,OUTPUT);
  }
}
void loop() { // To show number 0
  int x = B1000000;//x=0x40
  for (int j=2; j<=8;j++){
    digitalWrite(j,bitRead(x,j-2));
  }
}
```

គេអាចដាក់delay ខាងក្រោម digitalWrite(j,bitRead(x,j-2)); ដើម្បីបញ្ចេញលេខ០ ទៅ ១ ដោយប្រើអារេ

```
int x[]={0x40,0x79,0x24,0x30,0x19,0x12,0x78,0x02,0x00,0x10};
void setup() {
  for (int i=2;i<=8;i++){
    pinMode(i,OUTPUT);
  }
}
void loop(){
  for (int k=0;k<=9;k++){
    for (int j=2; j<=8;j++){
      digitalWrite(j,bitRead(x[k],j-2));
    }
    delay(1000);
  }
}
```

2.ការបង្ហាញ 7 segment លេខពីរ ខ្ទង់ ខុសគ្នាតាម bitRead

ដើម្បីឱ្យបញ្ចេញលេខពីរខ្ទង់ខុសគ្នាគេត្រូវ

```
int x[]={0x40,0x79,0x24,0x30,0x19,0x12,0x78,0x02,0x00,0x10};
void setup() {
    for (int i=2;i<=10;i++){
        pinMode(i,OUTPUT);
    }
}
void loop(){
    int x=0x24;
    digitalWrite(9,HIGH);
    digitalWrite(10,LOW);
    for (int j=2; j<=8;j++){
        digitalWrite(j,bitRead(x,j-2));
    }
    delay(10);
    int y=0x12;
    digitalWrite(9,LOW);
    digitalWrite(10,HIGH);
    for (int j=2; j<=8;j++){
        digitalWrite(j,bitRead(y,j-2));
    }
    delay(10);
}
```

ការសរសេរ 7 segment ដែលមានពីរខ្ទង់ហើយមានលេខខុសលេខ១ និង លេខ 2

```
int x[]={0x40,0x79,0x24,0x30,0x19,0x12,0x78,0x02,0x00,0x10};
void setup() {
    for (int i=0;i<=10;i++){
        pinMode(i,OUTPUT);
    }
}
void loop(){
    digitalWrite(9 ,HIGH);
    digitalWrite(10,LOW);
```



```

for (int j=2; j<=8;j++){
  digitalWrite(j,bitRead(x[1],j-2));
}
delay(1000);
digitalWrite(9,LOW);
digitalWrite(10,HIGH);
for (int j=2; j<=8;j++){
  digitalWrite(j,bitRead(x[2],j-2));
}
delay(1000);
}

```

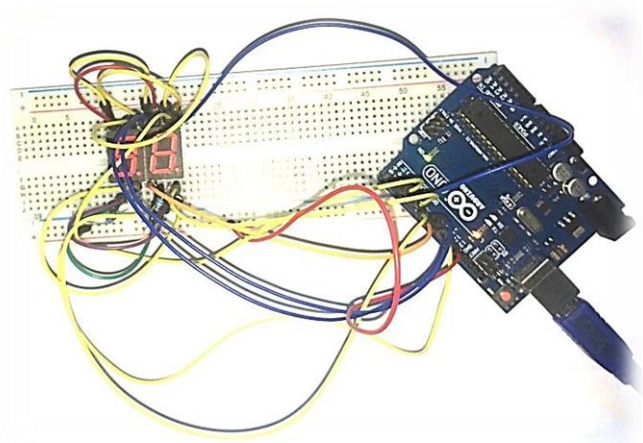
ក្នុងករណីនេះគេយើងប្រើជើងទី ៩ និង ទី១០ ដើម្បីឱ្យវាជាអ្នកផ្តល់ភ្លើង គេមិនប្រើជើង5v ទៀតទេ ដោយយកជើងទី10ភ្ជាប់ទៅនឹង7-segment ទី1 ហើយ ជើងទី៩ ទៅ 7-segmentទី2

ដើម្បីឱ្យ វា count ពី០០ដល់ ៩៩ គេប្រើ syntax ចំពោះ សម្រាប់រាប់ 50 ដង for (int m=0 ; m<=49;m++){ ។ ខាងក្រោមនេះជាកូដដែលរាប់ពី០០ ទៅ៩៩

```

int x[]={0x40,0x79,0x24,0x30,0x19,0x12,0x02,0x58,0x00,0x10};
void setup() {
  for (int i=2;i<=10;i++){
    pinMode(i,OUTPUT); }
}
void loop(){
  for (int k=0;k<=99;k++){
    int d1=k/10;
    int d2=k-(d1*10);
    for (int m=0 ; m<=49;m++)
    {
      digitalWrite(9 ,HIGH);
      digitalWrite(10,LOW);
      for (int j=2; j<=8;j++){
        digitalWrite(j,bitRead(x[d1],j-2));
      }
      delay(10);
      digitalWrite(9,LOW);
      digitalWrite(10,HIGH);
      for (int j=2; j<=8;j++){
        digitalWrite(j,bitRead(x[d2],j-2));
      }
    }
  }
}

```



```

    delay(10);
  }
}

```

ខាងក្រោមនេះជាកូដ 7 segments រាប់ពី 00 ទៅ 99 ។ កូដនេះយើងគ្រាន់តែដូរ for (int i=2;i<=10;i++) ហើយ for (int k=99;k>=0;k--) ។ ក្នុងករណីដែលមានលេខពីរខ្ទង់ យើងចែកនឹង 10 គឺ int d1=k/10; បើ បីខ្ទង់ចែកនឹង 100 គឺ int d1=k/100;

```

int x[]={0x40,0x79,0x24,0x30,0x19,0x12,0x02,0x58,0x00,0x10};
void setup() {
  for (int i=2;i<=10;i++){
    pinMode(i,OUTPUT);
  }
}
void loop(){
  for (int k=99;k>=0;k--){
    int d1=k/10;
    int d2=k-(d1*10);
    for (int m=0 ; m<=49;m++){
      digitalWrite(9 ,HIGH);
      digitalWrite(10,LOW);
      for (int j=2; j<=8;j++){
        digitalWrite(j,bitRead(x[d1],j-2));
      }
      delay(10);
      digitalWrite(9,LOW);
      digitalWrite(10,HIGH);
      for (int j=2; j<=8;j++){
        digitalWrite(j,bitRead(x[d2],j-2));
      }
      delay(10);
    }
  }
}

```

បើអ្នកចង់បន្ថែមខ្ទង់ ទៀតគេត្រូវថែម digitalWrite ហើយបន្ថែម port របស់ខ្សែគេអាចប្រើវិធី modulo

3.ការបង្កើត function

```
void setup() {
  for (int i=2;i<=4;i++){
    pinMode(i,OUTPUT);
  }
}
void loop(){
  test(1,0,1);
  test(1,0,1);
  test(1,0,1);
}
void test(int x,int y, int z ){
  digitalWrite(2,x);
  digitalWrite(3,y);
  digitalWrite(4,z);
  delay(1000);
}
```

ដើម្បីឱ្យ 7-segment 3 ដើរគេត្រូវបង្កើត void

```
int x[]={0x40,0x79,0x24,0x30,0x19,0x12,0x02,0x78,0x00,0x10};
void setup() {
  for (int i=2;i<=10;i++){
    pinMode(i,OUTPUT);
  }
}
void loop(){
  for (int k=0;k<=99;k++){
    int d1=k/10;
    int d2=k%10;
    for (int m=0 ; m<=49;m++){
      segment(1,0,d1);
      segment(0,1,d2);
    }
  }
}
void segment(int display1, int display2, int digit){
  digitalWrite(9,display1);
  digitalWrite(10,display2);
}
```

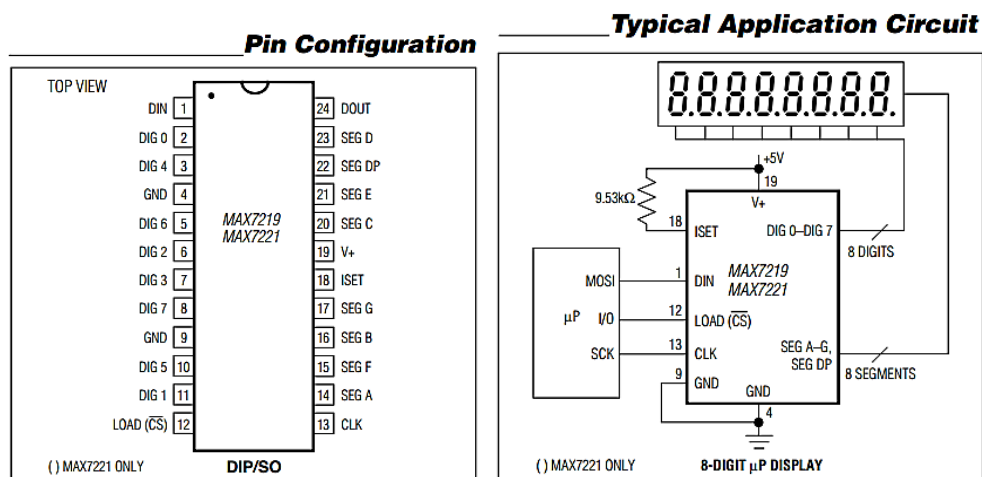
```

for(int j=2;j<=8;j++){
  digitalWrite(j,bitRead(x[digit],j-2));
}
delay(10);
}

```

VI.Driver for 7 segments more than 2 digits (Max7219)

ចំពោះវិធីសាស្ត្រខាងលើក្នុងការបង្ហាញលេខពីរខ្ទង់វាមានភាពងាយស្រួលតែក្នុងករណីដែលកូដតិច និងមិនមានកិច្ចការច្រើនសម្រាប់ឱ្យ Arduino ធ្វើការ ។ ប៉ុន្តែបើសិនជាមានកូដច្រើនយើងមិនអាចប្រើវាបានទេ ពីព្រោះប្រសិនបើមាន delay នៅក្នុងកូដនោះ shift 7-segment របស់យើងនឹងយឺត ហើយយើងច្បាស់ជាអាចមើលវាមិនទាន់ ។ ដូច្នេះការប្រើប្រាស់ IC driver សម្រាប់ករណីនេះងាយស្រួលជាង ហើយចំណេញទាំងពេល និងទាំង Hardware ។ តែយ៉ាងណាក៏វាមានការលំបាកដែលសម្រាប់អ្នកដែលទើបចាប់ផ្តើមប្រើដំបូង និងមាន protocol ស្មុគស្មាញបន្តិច ។ ខាងក្រោមនេះជាសៀគ្វីសម្រាប់ត



MAX7219/MAX7221 ជាប្រភេទ compact ដែលមានច្រកចេញច្រកចូលជា common-cathode ដែលបង្ហាញជា microprocessors(μPs) ទៅនឹង 7-segment ជា LED ចំនួនឱ្យបង្ហាញលេខ 8 ខ្ទង់ ប្រភេទ MAX7219/MAX7221 គេប្រើភ្លើង 3V វាមានតួនាទី បួនយ៉ាងគឺ

-Bar-Graph Displays

-Panel Meters

-Industrial Controllers

-LED Matrix Displays

ចំពោះដំណទៅកាន់ datasheet របស់វាគឺ

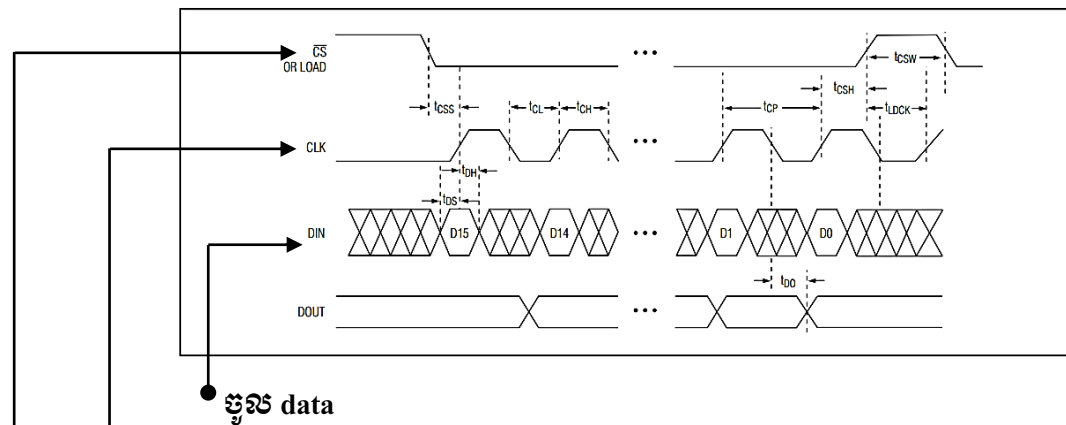
<https://datasheets.maximintegrated.com/en/ds/MAX7219-MAX7221.pdf>

ខាងក្រោមនេះជាតារាងបង្ហាញថា Max7219 ធ្វើការទទួលទិន្នន័យម្តងចំនួន 16 bits ដែលពី bit ទី 15 ដល់ទី 12 មិនគិត តែ bit ទី 11-8 ជា address សំដៅលើទីតាំងរបស់ខ្ទង់ណាមួយដែលត្រូវបង្ហាញលេខ និង ពី 7-0 ជាលេខដែលត្រូវបង្ហាញនៅ address នោះ ។

Table 1. Serial-Data Format (16 Bits)

D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0
X	X	X	X	ADDRESS				DATA							
								MSB							LSB

ចំពោះការបញ្ជូនទិន្នន័យនៃ bit នីមួយៗគឺត្រូវគោលតារាង Time Diagram ខាងក្រោម bits ទី 15 ត្រូវបញ្ជូនទៅមុនគេ ហើយ bits បន្ទាប់ៗទៀតត្រូវបញ្ជូនរៀងរាល់ clk Rising edge និង load ស្ទើរមួយម្តងពេលគ្រប់ 16 bits ។



ផ្ទេរ data

ទាញ data

ដំណើរការនៅពេលដែល clk បិទបើក

ដើម្បីឱ្យយើងប្រើប្រាស់នូវ Max7219 យើងត្រូវស្គាល់ពីការ register នៅ address មួយចំនួនជាមុនសិន ។ ខាងក្រោមនេះជា address to register

Table 2. Register Address Map

REGISTER	ADDRESS					HEX CODE
	D15-D12	D11	D10	D9	D8	
No-Op	X	0	0	0	0	0xX0
Digit 0	X	0	0	0	1	0xX1
Digit 1	X	0	0	1	0	0xX2
Digit 2	X	0	0	1	1	0xX3
Digit 3	X	0	1	0	0	0xX4
Digit 4	X	0	1	0	1	0xX5
Digit 5	X	0	1	1	0	0xX6
Digit 6	X	0	1	1	1	0xX7
Digit 7	X	1	0	0	0	0xX8
Decode Mode	X	1	0	0	1	0xX9
Intensity	X	1	0	1	0	0xXA
Scan Limit	X	1	0	1	1	0xXB
Shutdown	X	1	1	0	0	0xXC
Display Test	X	1	1	1	1	0xXF

- Address ពី 0-7 : យើងសង្កេតឃើញថា ជួរឈរតារាង HEX CODE មាន 0xX0, 0xX1,0xX2,...,0xXF ដែល X(អ៊ីចធំ) មានន័យថា don't care វាអាចមានតម្លៃ 0 ឬ 1 ។
- Shutdown Mode : មានតួនាទី បិទឬបើកឱ្យដំណើរការ ឧទាហរណ៍ ក្នុងproject មួយដែលយើងចង់ឱ្យផ្នែកណាមួយមិនដំណើរការរហូតឬមួយរយៈ ហើយ controller អាចប្រើប្រាស់ទៅលើdisplayដូចជា sensor ជាដើមដើរដូចធម្មតា ។ ក្នុង Shutdown Mode លេខ 1 ដំណើរការ ហើយលេខ០ បិទដំណើរការ

0xX0 បិទដំណើរការ Ex : 0xa0

0xX1 បើកដំណើរការ Ex : 0xa1

— **Shutdown Register Format (Address (Hex) = 0xXC)**

MODE	ADDRESS CODE (HEX)	REGISTER DATA							
		D7	D6	D5	D4	D3	D2	D1	D0
Shutdown Mode	0xXC	X	X	X	X	X	X	X	0
Normal Operation	0xXC	X	X	X	X	X	X	X	1

- Decode Mode :** សម្រាប់កំណត់ឱ្យ Max7219 បំប្លែងលេខ (decode binary number) ដែលយើងបញ្ចូលទៅឱ្យទៅជា លេខសម្រាប់ 7-segment ឬអត់ ។ ប្រសិនបើអ្នកចង់បើក digit ណាខ្លះ ឧទាហរណ៍ យើងមាន 8 digits យើងចង់ប្រើតែ4digits ទេគឺពី 0ទៅ 3 គេត្រូវធ្វើ Decode-Mode ជាមុនសិន ។

***នៅពេលដែលយើងបង្កើត LED ម៉ាទ្រីស ឬ តួបមិនចាំបាច់ Decode-Mode ឡើយ ដោយសារយើងមិនចង់បានលេខ 0,1នៅលើ7-segmentទេ យើងគ្រាន់តែចង់បានLEDគួរភ្លឺនៅកន្លែងណាមួយប៉ុណ្ណោះ

Figure 2-10 Decode-Mode Register Examples (Address (Hex) = 0xX9)

DECODE MODE	REGISTER DATA								HEX CODE
	D7	D6	D5	D4	D3	D2	D1	D0	
No decode for digits 7-0	0	0	0	0	0	0	0	0	0x00
Code B decode for digit 0 No decode for digits 7-1	0	0	0	0	0	0	0	1	0x01
Code B decode for digits 3-0 No decode for digits 7-4	0	0	0	0	1	1	1	1	0x0F
Code B decode for digits 7-0	1	1	1	1	1	1	1	1	0xFF

វាមានភាពស្រដៀងគ្នាទៅនឹង IC74ls47 ដែលយើងធ្លាប់រៀនពីមុនដែលគ្រាន់តែថាវាប្រើសម្រាប់ 7-segment CC ។ ខាងក្រោមនេះជាតារាងលេខរបស់វាដែលបង្ហាញលើអង្កត់លេខ ។

Code B Font

7-SEGMENT CHARACTER	REGISTER DATA						ON SEGMENTS = 1							
	D7*	D6-D4	D3	D2	D1	D0	DP*	A	B	C	D	E	F	G
0		X	0	0	0	0		1	1	1	1	1	1	0
1		X	0	0	0	1		0	1	1	0	0	0	0
2		X	0	0	1	0		1	1	0	1	1	0	1
3		X	0	0	1	1		1	1	1	1	0	0	1
4		X	0	1	0	0		0	1	1	0	0	1	1
5		X	0	1	0	1		1	0	1	1	0	1	1
6		X	0	1	1	0		1	0	1	1	1	1	1
7		X	0	1	1	1		1	1	1	0	0	0	0
8		X	1	0	0	0		1	1	1	1	1	1	1
9		X	1	0	0	1		1	1	1	1	0	1	1
—		X	1	0	1	0		0	0	0	0	0	0	1
E		X	1	0	1	1		1	0	0	1	1	1	1
H		X	1	1	0	0		0	1	1	0	1	1	1
L		X	1	1	0	1		0	0	0	1	1	1	0
P		X	1	1	1	0		1	1	0	0	1	1	1
blank		X	1	1	1	1		0	0	0	0	0	0	0

*The decimal point is set by bit D7 = 1

ការប្រើប្រាស់ Max7219 មានភាពងាយស្រួលនិងចាប់រហ័ស ដោយសារវាមានបានជាបំប្លែងក្នុងពី decimal ទៅ binary ប៉ុន្តែអ្នកត្រូវដឹងថា វាប្រើបានតែcommon cathod ទេ ។ យើងមិនចាំបាច់បង្កើត អាវេដេមីបំប្លែងក្នុងតទៅរហូតដល់ចប់នោះទេ ។

- **Scan Limit(0xB):** សម្រាប់កំណត់ចំនួនខ្ទង់ដែលត្រូវបង្ហាញតាមចិត្តចង់របស់យើង៖

Scan-Limit Register Format (Address (Hex) = 0xB)

SCAN LIMIT	REGISTER DATA								HEX CODE
	D7	D6	D5	D4	D3	D2	D1	D0	
Display digit 0 only*	X	X	X	X	X	0	0	0	0x0
Display digits 0 & 1*	X	X	X	X	X	0	0	1	0x1
Display digits 0 1 2*	X	X	X	X	X	0	1	0	0x2
Display digits 0 1 2 3	X	X	X	X	X	0	1	1	0x3
Display digits 0 1 2 3 4	X	X	X	X	X	1	0	0	0x4
Display digits 0 1 2 3 4 5	X	X	X	X	X	1	0	1	0x5
Display digits 0 1 2 3 4 5 6	X	X	X	X	X	1	1	0	0x6
Display digits 0 1 2 3 4 5 6 7	X	X	X	X	X	1	1	1	0x7

*See Scan-Limit Register section for application.

- **Intensity(0xA):** សម្រាប់កំណត់កម្រិតពន្លឺរបស់ 7-segment តាមរយៈ Programmable ដែលមានន័យថាយើងអាចធ្វើឱ្យពន្លឺខ្លាំងឬខ្សោយដោយសារតែក្នុង Max7219 មានរេស៊ីស្តង់ស្រាប់ យើងដំណើរការតាមរយៈការ register ទៅកាន់ address នេះនិង data ណាមួយដែលយើងត្រូវការ។

Intensity Register Format (Address (Hex) = 0xA)

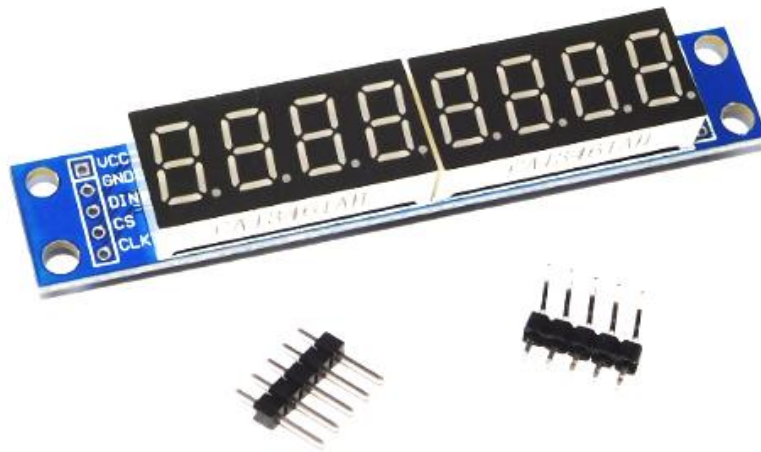
DUTY CYCLE		D7	D6	D5	D4	D3	D2	D1	D0	HEX CODE	ខ្សោយ ខ្លាំង
MAX7219	MAX7221										
1/32 (min on)	1/16 (min on)	X	X	X	X	0	0	0	0	0x0	
3/32	2/16	X	X	X	X	0	0	0	1	0x1	
5/32	3/16	X	X	X	X	0	0	1	0	0x2	
7/32	4/16	X	X	X	X	0	0	1	1	0x3	
9/32	5/16	X	X	X	X	0	1	0	0	0x4	
11/32	6/16	X	X	X	X	0	1	0	1	0x5	
13/32	7/16	X	X	X	X	0	1	1	0	0x6	
15/32	8/16	X	X	X	X	0	1	1	1	0x7	
17/32	9/16	X	X	X	X	1	0	0	0	0x8	
19/32	10/16	X	X	X	X	1	0	0	1	0x9	
21/32	11/16	X	X	X	X	1	0	1	0	0xA	
23/32	12/16	X	X	X	X	1	0	1	1	0xB	
25/32	13/16	X	X	X	X	1	1	0	0	0xC	
27/32	14/16	X	X	X	X	1	1	0	1	0xD	
29/32	15/16	X	X	X	X	1	1	1	0	0xE	
31/32	15/16 (max on)	X	X	X	X	1	1	1	1	0xF	

- **Display Test (0xF)**
សម្រាប់ប្រើធ្វើតេស្តថាមានអង្កត់ LED ណាមួយមិនភ្លឺ ។ ប៉ុន្តែគួរចងចាំថានៅពេលដែលតេស្តរួចហើយត្រូវបិទ register នេះវិញដើម្បីត្រលប់មកភាពដើមវិញ

**Display-Test Register Format
(Address (Hex) = 0xF)**

MODE	REGISTER DATA							
	D7	D6	D5	D4	D3	D2	D1	D0
Normal Operation	X	X	X	X	X	X	X	0
Display Test Mode	X	X	X	X	X	X	X	1

Note: The MAX7219/MAX7221 remain in display-test mode (all LEDs on) until the display-test register is reconfigured for normal operation.



7 segment one digit

```
const int din = 9;
const int load = 10;
const int clk = 11;
void setup() {
  pinMode(din,OUTPUT);
  pinMode(load,OUTPUT);
  pinMode(clk,OUTPUT);
  maxim(0x0c,0x01); //Operation(shutdown/Normal)
  maxim(0x09,0xff); //Decode Mode
  maxim(0x0a,0x07); //Intensity Mode
  maxim(0x0b,0x07); //Scan limit
}
void loop() {
  maxim(1,1);
  maxim(2,9);
}
void maxim(int address, int data){
  digitalWrite(load,LOW);
  for (int i=7;i>=0;i--){
    digitalWrite(din,bitRead(address,i));
    digitalWrite(clk,LOW);
    digitalWrite(clk,HIGH);
  }
  for (int i=7;i>=0;i--){
    digitalWrite(din,bitRead(data,i));
    digitalWrite(clk,LOW);
```

```

    digitalWrite(clk,HIGH);
  }
  digitalWrite(load,HIGH);
}

```

ដើម្បី count ពី 00-99

```

const int din = 9;
const int load = 10;
const int clk = 11;
void setup() {
  pinMode(din,OUTPUT);
  pinMode(load,OUTPUT);
  pinMode(clk,OUTPUT);

  maxim(0x0c,0x01); //Operation(shutdown/Normal)
  maxim(0x09,0xff); //Decode Mode
  maxim(0x0a,0x07); //Intensity Mode
  maxim(0x0b,0x07); //Scan limit
}
void loop() {
  for (int j=0;j<=99;j++){
    int d2=j/10;
    int d1=j%10;
    maxim(1,d1);
    maxim(2,d2);
    delay(1000);
  }
}
void maxim(int address, int data){
  digitalWrite(load,LOW);
  for (int i=7;i>=0;i--){
    digitalWrite(din,bitRead(address,i));
    digitalWrite(clk,LOW);
    digitalWrite(clk,HIGH);
  }
  for (int i=7;i>=0;i--){
    digitalWrite(din,bitRead(data,i));
    digitalWrite(clk,LOW);
    digitalWrite(clk,HIGH);
  }
}

```

```

    }
    digitalWrite(load,HIGH);
}

```

count ពី 000-999

```

const int din = 9;
const int load = 10;
const int clk = 11;
void setup() {
    pinMode(din,OUTPUT);
    pinMode(load,OUTPUT);
    pinMode(clk,OUTPUT);
    maxim(0x0c,0x01); //Operation(shutdown/Normal)
    maxim(0x09,0xff); //Decode Mode
    maxim(0x0a,0x07); //Intensity Mode
    maxim(0x0b,0x07); //Scan limit
}

void loop() {
    for (int j=0;j<=999;j++){
        int d3=j/100;
        int d2=j/10%10;
        int d1=j%10;
        maxim(1,d1);
        maxim(2,d2);
        maxim(3,d3);
        delay(10);
    }
}

void maxim(int address, int data){
    digitalWrite(load,LOW);
    for (int i=7;i>=0;i--){
        digitalWrite(din,bitRead(address,i));
        digitalWrite(clk,LOW);
        digitalWrite(clk,HIGH);
    }
    for (int i=7;i>=0;i--){
        digitalWrite(din,bitRead(data,i));
        digitalWrite(clk,LOW);
        digitalWrite(clk,HIGH);
    }
    digitalWrite(load,HIGH);
}

```

}

ដោយគ្រាន់តែបន្ថែម កូដនៅខាងក្រោម void loop យើងអាចបង្កើតលេខ៤ខ្ទង់ដោយរត់ពី០-៩៩៩៩
តាមរូបមន្តខាងក្រោម

```
void loop() {
  for (int j=0;j<=9999;j++){
    int d4=j/1000;
    int d3=j/100%10;
    int d2=j%100/10;
    int d1=j%10;
    maxim(1,d1);
    maxim(2,d2);
    maxim(3,d3);
    maxim(4,d4);
    delay(100);
  }
}
```

ដោយគ្រាន់តែបន្ថែម កូដនៅខាងក្រោម void loop យើងអាចបង្កើតលេខ៤ខ្ទង់ដោយរត់ពី០-៩៩៩៩៩៩
តាមរូបមន្តខាងក្រោម

```
for (int j=0;j<=99999;j++){
  int d5=j/10000;
  int d4=j/1000%10;
  int d3=j%1000/10;
  int d2=j%100;
  int d1=j%10;
  maxim(1,d1);
  maxim(2,d2);
  maxim(3,d3);
  maxim(4,d4);
  maxim(5,d5);
  delay(5);
}
}
```

នៅពេលដែលកូដដើម្បីសរសេរ 7-segment មានលេខច្រើនជា ៥ខ្ទង់យើងត្រូវប្តូរ អថេរ int ទៅ
long វិញ

```
void loop() {
  for (long j=0;j<=999999;j++){
    long d6=j/100000;
```

```

    long d5=j/10000%10;
    long d4=j%1000/10;
    long d3=j%100/10;
    long d2=j%10/10;
    long d1=j%10;
    maxim(1,d1);
    maxim(2,d2);
    maxim(3,d3);
    maxim(4,d4);
    maxim(5,d5);
    maxim(6,d6);
    delay(1);
  }
}

```

8.ការបង្កើតក្លែងសញ្ញាចរាចរណ៍

```

void setup() {
  for(int m=2;m<=7;m++){
    pinMode(m,OUTPUT);
  }
}

void traffic(boolean R1,boolean Y1, boolean G1,boolean R2, boolean Y2,
boolean G2){
  digitalWrite(2,R1);
  digitalWrite(3,Y1);
  digitalWrite(4,G1);
  digitalWrite(5,R2);
  digitalWrite(6,Y2);
  digitalWrite(7,G2);
}

void loop() {
  traffic(1,0,0,0,0,1);
  delay(2000);
  traffic(1,0,0,0,1,0);
  delay(500);
  traffic(0,0,1,1,0,0);
  delay(2000);
  traffic(0,1,0,1,0,0);
  delay(500);
}

```

បើយើងចង់ចេញលេខ គេប្រើ for

traffic with number:

```
const int din=9;
const int load=10;
const int clk=11;
void setup() {
    for(int m=2;m<=7;m++){
        pinMode(m,OUTPUT);
    }
    maxim(0x0c,0x01); //operation mode
    maxim(0x09,0x0f); //decode mode
    maxim(0x0a,0x07); //intensity
    maxim(0x0b,0x03); //scan limit
}
void traffic(boolean R1,boolean Y1, boolean G1,boolean R2, boolean Y2,
boolean G2){
    digitalWrite(2,R1);
    digitalWrite(3,Y1);
    digitalWrite(4,G1);
    digitalWrite(5,R2);
    digitalWrite(6,Y2);
    digitalWrite(7,G2);
}
void maxim(int address, int data){
    digitalWrite(load,0);
    for (int i=7;i>=0;i--){
        digitalWrite(din,bitRead(address,i));
        digitalWrite(clk,0);
        digitalWrite(clk,1);
    }
    for (int i=7;i>=0;i--){
        digitalWrite(din,bitRead(data,i));
        digitalWrite(clk,0);
        digitalWrite(clk,1);
    }
    digitalWrite(load,1);
}
void loop() {
    int road1=25;
    int road2=20;
    for (int p = 1; p<=20;p++){
        maxim(1,road1%10);
```

```

    maxim(2,road1/10);
    maxim(3,road2%10);
    maxim(4,road2/10);
    traffic(1,0,0,0,0,1);
    delay(1000);
    road1--;
    road2--;
}
for (int p = 1; p<=5;p++){
    traffic(1,0,0,0,1,0);
    delay(1000);
}
for (int p = 1; p<=20;p++){
    traffic(0,0,1,1,0,0);
    delay(1000);
}
for (int p = 1; p<=5;p++){
    traffic(0,1,0,1,0,0);
    delay(1000);
}
}

```

Traffic light full ភ្លើងបរាបរណ៍

```

const int din=9;
const int load=10;
const int clk=11;
void setup() {
    for(int m=2;m<=7;m++){
        pinMode(m,OUTPUT);
    }
    pinMode(din,OUTPUT);
    pinMode(load,OUTPUT);
    pinMode(clk,OUTPUT);

    maxim(0x0c,0x01); //operation mode
    maxim(0x09,0x0f); //decode mode
    maxim(0x0a,0x07); //intensity
    maxim(0x0b,0x03); //scan limit
}

void traffic(boolean R1,boolean Y1, boolean G1,boolean R2, boolean Y2,
boolean G2){

```

```

digitalWrite(2,R1);
digitalWrite(3,Y1);
digitalWrite(4,G1);
digitalWrite(5,R2);
digitalWrite(6,Y2);
digitalWrite(7,G2);
}
void maxim(int address, int data){
  digitalWrite(load,0);
  for (int i=7;i>=0;i--){
    digitalWrite(din,bitRead(address,i));
    digitalWrite(clk,0);
    digitalWrite(clk,1);
  }
  for (int i=7;i>=0;i--){
    digitalWrite(din,bitRead(data,i));
    digitalWrite(clk,0);
    digitalWrite(clk,1);
  }
  digitalWrite(load,1);
}
void loop() {
  int road1=25;
  int road2=20;
  for (int p = 1; p<=20;p++){
    maxim(1,road1%10);
    maxim(2,road1/10);
    maxim(3,road2%10);
    maxim(4,road2/10);
    traffic(1,0,0,0,0,1);
    delay(1000);
    road1--;
    road2--;
  }
  road1=5;
  road2=5;
  for (int p = 1; p<=5;p++){
    maxim(1,road1%10);
    maxim(2,road1/10);
    maxim(3,road2%10);
    maxim(4,road2/10);
  }
}

```



```

    traffic(1,0,0,0,1,0);
    delay(1000);
    road1--;
    road2--;
}
road1=20;
road2=25;
for (int p = 1; p<=20;p++){
    maxim(1,road1%10);
    maxim(2,road1/10);
    maxim(3,road2%10);
    maxim(4,road2/10);
    traffic(0,0,1,1,0,0);
    delay(1000);
    road1--;
    road2--;
}
road1=5;
road2=5;
for (int p = 1; p<=5;p++){
    maxim(1,road1%10);
    maxim(2,road1/10);
    maxim(3,road2%10);
    maxim(4,road2/10);
    traffic(0,1,0,1,0,0);
    delay(1000);
    road1--;
    road2--;
}
}

```

កូដកាត់សម្រាប់ traffic light counter (7 segment and LED)

```

const int din = 8;
const int load = 9;
const int clk = 10;
void setup() {
    for (int i=2;i<=10;i++){
        pinMode(i,OUTPUT);
    }
    Dara(0x0c,0x01); //operation mode

```

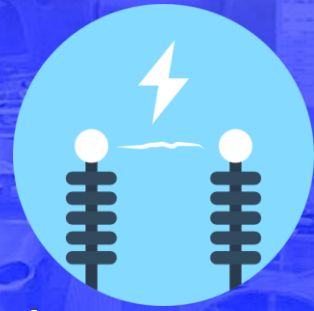
```

Dara(0x09,0xff); //Decode mode
Dara(0x0a,0x05); //intensity mode
Dara(0x0b,0x03); //scan limit for 4 digits
}
void Dara(int address,int data){
    digitalWrite(load,LOW);
    shiftOut(din,clk,MSBFIRST,address);
    shiftOut(din,clk,MSBFIRST,data);
    digitalWrite(load,HIGH);
}
void light(int R1,int Y1,int G1,int R2,int Y2,int G2){
    digitalWrite(2,R1); //Red1
    digitalWrite(3,Y1); //Yellow1
    digitalWrite(4,G1); //Green1
    digitalWrite(5,R2); //Red2
    digitalWrite(6,Y2); //Yellow2
    digitalWrite(7,G2); //Green2
}
void calculate(int num1,int num2){
    Dara(1,num1%10);
    Dara(2,num1/10);
    Dara(3,num2%10);
    Dara(4,num2/10);
    delay(1000);
}
void loop() {
    int road1 = 25;
    int road2 =20;
    for (int m=1;m<=20;m++){
        light(1,0,0,0,0,1);
        calculate(road1,road2);
        road1--; //road1=road1-1
        road2--; //road2=road2-1
    }
    road1 = 5;
    road2 = 5;
    for (int m=1;m<=5;m++){
        light(1,0,0,0,1,0);
        calculate(road1,road2);
        road1--;
        road2--;
    }
}

```

```
}  
road1 = 20;  
road2 = 25;  
for (int m=1;m<=20;m++){  
    light(0,0,1,1,0,0);  
    calculate(road1,road2);  
    road1--;  
    road2--;  
}  
road1 = 5;  
road2 = 5;  
for (int m=1;m<=5;m++){  
    light(0,1,0,1,0,0);  
    calculate(road1,road2);  
    road1--;  
    road2--;  
}  
}
```

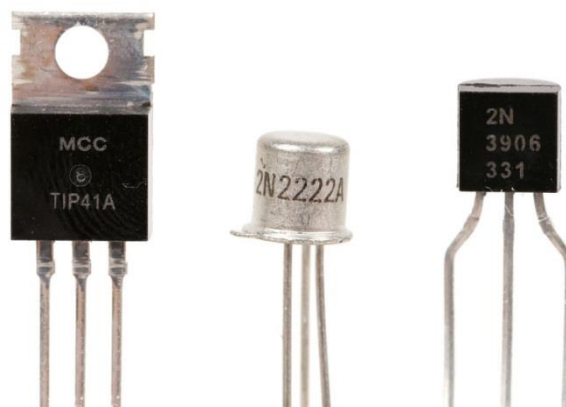
ជំពូកទី៣ ការប្រើប្រាស់ថាមពល



ខ្ពស់ជាងថាមពលInput របស់ Arduino

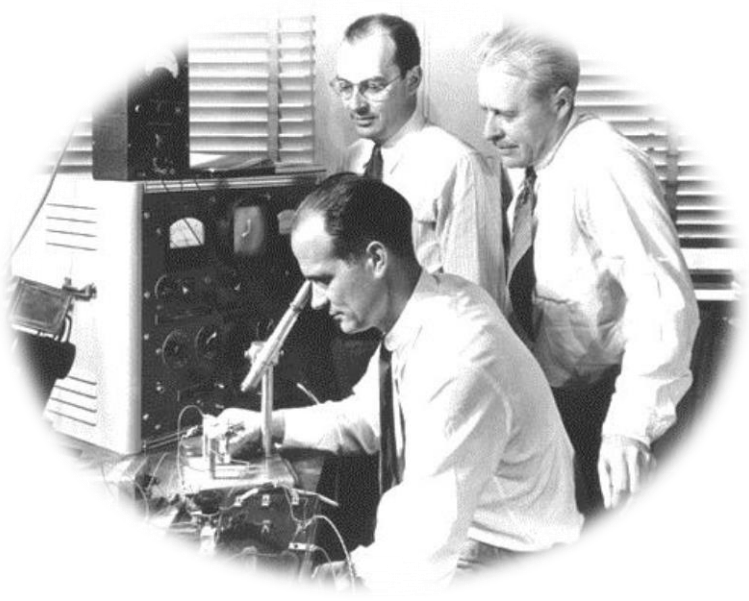
ទោះបីយ៉ាងណាក៏ដោយយើងមិនអាចប្រើប្រាស់ Arduino ដោយផ្ទាល់ជាមួយឧបករណ៍ទាំងឡាយណាដែលទាមទារអនុភាពធំ ឬត្រង់ស្បូងខុសគ្នាពីតង់ស្យុង Arduino នោះទេ ។ វាមានវិធីសាស្ត្រជាច្រើនអាចធ្វើបានហើយ ក៏ខុសគ្នាៗដែរអាស្រ័យលើឧបករណ៍ដែល Arduino បញ្ជូនទៅ ។ ក្នុងមេរៀននេះយើងនឹងប្រើប្រាស់វាជាមួយត្រង់ស៊ីស្ទ័រ និង រឺឡេ (relay) ។ ដោយសារតែArduino មិនអាចបញ្ជូនដោយផ្ទាល់ទៅកាន់រឺឡេ ហេតុដូច្នេះហើយបានជាយើងត្រូវប្រើប្រាស់ត្រង់ស៊ីស្ទ័រដែលមានមុខងារជាកុងតាក់អេឡិចត្រូនិចសម្រាប់ភ្ជាប់ឱ្យ រឺឡេដំណើរការ ។ ត្រង់ស៊ីស្ទ័រពង្រីកសញ្ញាលំតង់ស្យុងខ្សោយពី Arduino ឱ្យទៅជាតង់ស្យុងខ្ពស់របស់រឺឡេ ហើយកុងតាក់របស់រឺឡេ អាចភ្ជាប់ឱ្យឧបករណ៍ផ្សេងទៀតដំណើរការដូចជា អំពូលភ្លើង 220v ម៉ូទ័រ ។ ល ។

1.ត្រង់ស៊ីស្ទ័រ (Transistor)

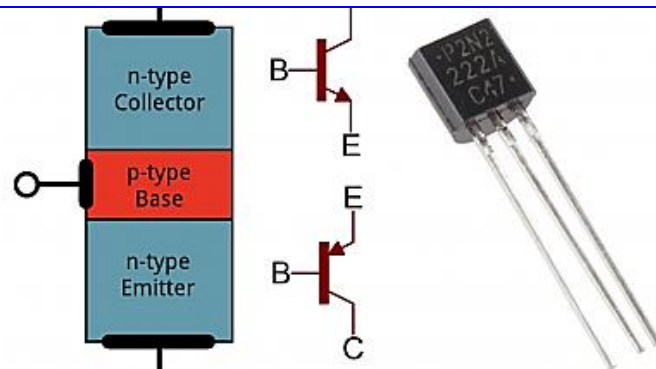


ភាគច្រើននៃបរិក្ខារអគ្គិសនីក្នុងផ្ទះ ហាង ប្រព័ន្ធផ្សព្វផ្សាយ ... ។ ល ។ ផ្ទុំឡើងពីបណ្តុំនៃ Silicon Chip ភ្ជាប់នឹងសៀគ្វីអេឡិចត្រូនិច ។ Silicon Chip ទាំងនោះជាបណ្តុំនៃ Transistor រាប់ពាន់ ឬក៏រាប់លាន ។

ប្រវត្តិការបង្កើត Transistor បានចាប់ផ្តើមបន្ទាប់ពីរឿងរ៉ាវវែងឆ្ងាយនៃការស្រាវជ្រាវវិទ្យាសាស្ត្រក្នុងទសវត្សរ៍ ១៨០០ របស់អ្នកវិទ្យាសាស្ត្រដូចជា Maxwell ,Hertz , Faraday , and Edison ដែលបានរកឃើញអគ្គិសនីសម្រាប់មនុស្សប្រើប្រាស់ ។ ហើយអ្នកបង្កើតថ្មីផ្សេងទៀតដូចជា Braun , Marconi ,Fleming , and DeForest បានយកទ្រឹស្តីខាងលើមកអភិវឌ្ឍន៍បង្កើតជាបរិក្ខារអគ្គិសនីមានប្រយោជន៍ផ្សេងទៀតដូចជា វិទ្យុ ។ ការងាររបស់ពួកគេនេះទៀតសោដបានបង្កើតឱ្យមានដំណាក់កាលថ្មីមួយទៀតសម្រាប់អ្នកវិទ្យាសាស្ត្រនៅមន្ទីរពិសោធន៍ Bell ក្នុងការប្រកួតប្រជែងប្រើប្រាស់ចំនេះដឹងរបស់គេ លើការបង្កើតបរិក្ខារអគ្គិសនីមានប្រយោជន៍សម្រាប់ប្រព័ន្ធទំនាក់ទំនងដូចគ្នាដែរ ។ ក្រុមអ្នកវិទ្យាសាស្ត្រនៃមន្ទីរពិសោធន៍ Bell រួមមាន Shockley ,Brattain ,Bardeen និងអ្នកដទៃទៀតបានត្រួតស្ទាត់បង្កើតយុគសម័យព័ត៌មានវិទ្យា តាមរយៈការបង្កើតនូវបរិក្ខារអេឡិចត្រូនិចដ៏អស្ចារ្យបំផុតមួយក្នុងសតវត្សរ៍ ២០ នោះគឺ Transistor នៅឆ្នាំ ១៩៤៧ ។

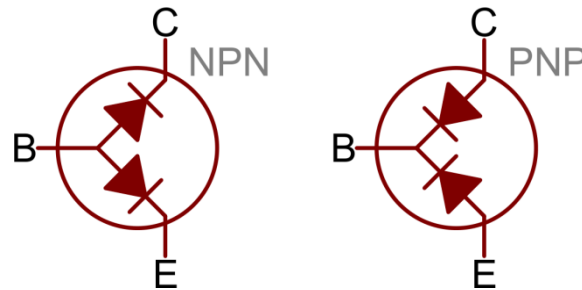


ក្នុងមេរៀននេះយើងនឹងសិក្សាអំពី Transistor Bi-Polar Junction(BJT)



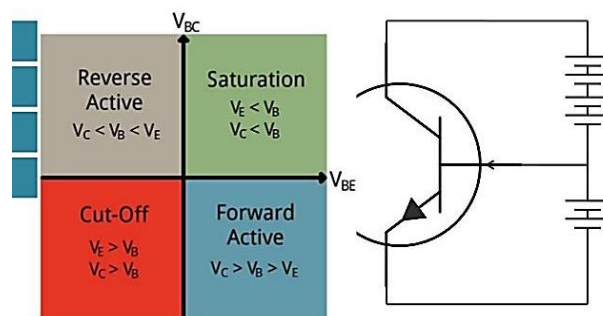
Transistor bi-polar មានជើង ៣ សំខាន់នោះគឺជើង Collector(C) , Emitter(E) និង Base(B) ។ ហើយវាត្រូវបានបែងចែកជាពីរប្រភេទនោះគឺ NPN និង PNP មាននិមិត្តសញ្ញា ។

ដើម្បីចង់ដឹងប្រភេទត្រង់ស៊ីស្តែមគេត្រូវ កាត់ h_{FE} ហើយបើគ្មាន PNP NPN គ្មានទីតាំងគេត្រូវកាត់មកយ៉ូត វាស់ដោយចាក់នៅលើជើងរបស់វា បើវាមានតម្លៃហូរចូល នោះវាជាអាចសម្គាល់

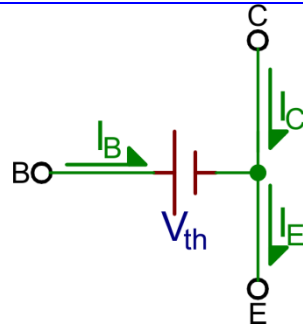


មិនដូចជា Resistor ទេដែលទំនាក់ទំនងរវាងស្បូង និងចរន្ត មានលក្ខណៈជា linear ផ្ទុយទៅវិញ Transistor គឺឧបករណ៍ Non-linear ។ វាបែងចែកជាបួនម៉ូតដែលពិពណ៌នាអំពីលំហូរចរន្តឆ្លងកាត់វា ។ នៅពេលគេនិយាយពីចរន្តហូរគេនិយាយពីចរន្តរត់ពី Collector ទៅ Emitter ។ ម៉ូតទាំងនោះមានដូចរូបខាងក្រោម ៖

MODE



- **Saturation** ជាតំបន់ដែល Transistor ដំណើរការហាក់ដូចជា ក្នុងសៀគ្វីដែលធ្វើឱ្យចរន្តហូរដោយសេរីពី Collector ទៅ Emitter ។

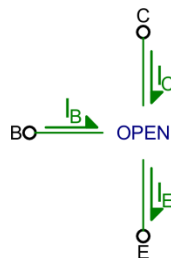


នៅក្នុង Saturation Mode ឌីយ៉ូតទាំងពីរនៅក្នុង Transistor ស្ថិតក្នុងស្ថានភាព Forward Biased ដែលមានន័យថា V_{BE} ត្រូវតែធំជាង 0 ហើយធំជាង V_{BC}

$$\begin{matrix} V_B > V_C \\ V_B > V_E \end{matrix}$$

ដោយសារតែ Junction ពីជើង Base ទៅ Emitter ហាក់បីដូចជាឌីយ៉ូតមួយ នៅក្នុងការអនុវត្តពិត V_{BE} ត្រូវតែធំជាង Threshold voltage (V_{th}) ដើម្បីបានក្លាយជា Saturation តម្លៃនៃ V_{th} នេះ ប្រែប្រួលពីប្រភេទ Transistor មួយទៅមួយនិងសីតុណ្ហភាព ។ ភាគច្រើននៃតម្លៃនេះធ្លាក់ចុះប្រហែល 0.6v ។ ក្នុងការអនុវត្តពិតមួយទៀតដែរ ការចំលងរបស់ Transistor ឬ អាចថាចំលងពីរ Collector ទៅ Emitter ពុំចំលងឥតខ្ចោះនោះទេ ។ វានៅតែមានតង់ស្យុង ធ្លាក់ចុះតូចមួយរវាងជើងទាំងពីរ ។ តង់ស្យុងតូចនោះអ្នកអាចរកឃើញបានតាមរយៈសៀវភៅរបៀបប្រើប្រាស់របស់វា Datasheet ត្រង់ចំណុច CE Saturation voltage $V_{EC(sat)}$ ដែលមានតម្លៃប្រហែលចន្លោះពី 0.05v-0.2v ។ តម្លៃនេះមានន័យថា V_C ត្រូវតែធំជាង V_E តែនៅតូចជាង V_B ដើម្បីបានដល់ Saturation mode ។

Cut-off : តំបន់ដែល Transistor ដើរតួជាសៀគ្វីចំហ ដែលមានន័យពុំមានចរន្តអាចឆ្លងកាត់បាន ។



ដើម្បីឈានដល់ Cut off mode តង់ស្យុង Base ត្រូវតែតូចជាងតង់ស្យុង Emitter និង Collector ។ មានន័យថា V_{BC} និង V_{BE} ត្រូវតែអវិជ្ជមាន ។

$$\begin{matrix} V_C > V_B \\ V_E > V_B \end{matrix}$$

នៅក្នុងការអនុវត្តជាក់ស្តែង V_{BE} អាចជាតម្លៃចន្លោះ 0V ទៅ V_{th} (0.6V) ។

Forward-Active : ចរន្តរត់ពី Collector ទៅ Emitter សមាមាត្រទៅនឹងចរន្តនៅជើង Base ។ ដើម្បីអាចដំណើរការក្នុង Forward-Active mode តង់ស្យុង V_{BE} របស់ Transistor ត្រូវតែរត់ធំជាងសូន្យ ហើយ V_{BC} ត្រូវតែអវិជ្ជមាន ។ ជាការពិតណាស់ តង់ស្យុង Base ត្រូវតែតូចជាងតង់ស្យុង Collector ប៉ុន្តែធំជាងតង់ស្យុង Emitter ។ នោះមានន័យថា Collector គឺធំជាង Emitter ឆ្ងាយ ។

$$V_C > V_B > V_E$$

ក្នុងការអនុវត្តពិត យើងត្រូវការតង់ស្យុងមិនមែនសូន្យ Forward Voltage Drop (V_{th}) ពីជើង Base ទៅ Emitter (V_{BE}) ដើម្បីបើក Transistor ឬ ហៅថាឱ្យវាចំលងចរន្ត ។ ជាធម្មតាតង់ស្យុងនោះគឺប្រហែល $V_{BE} = 0.6V$ ។

Revers-Active : ដូចជា Forward-active ដែរ តែចរន្តនៅជើង Base មានទិសប្រាស ។

- ចរន្ត Collector (I_C)

Active mode របស់ transistor ជាផ្នែកដ៏សំខាន់របស់វាដោយសារវាឆ្លើយឱ្យឆ្ពោះទៅរកការសិក្សាអំពី Amplifier ឬការពង្រីកសញ្ញា ។ ការពង្រីកនេះមានរូបមន្ត :

$$I_C = \beta I_B \text{ ដែល}$$

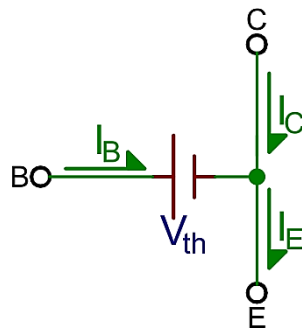
β ជា Gain របស់ Transistor ដែលប្រែប្រួលតាមប្រភេទនៃ Transistor នីមួយៗហើយអាចរកបានតាមរយៈការវាស់ដោយឧបករណ៍ ឬ Datasheet ។ នៅក្នុងសៀវភៅខ្លះតាងដោយ h_{FE} ។ តម្លៃរបស់វាជាធម្មតាប្រហែល 100 ឬ អាចចន្លោះពី 50 ទៅ 200 ហើយក៏អាចដល់ 2000 ដែរ ។

I_B ជាតម្លៃចរន្តតាមជើង Base របស់ Transistor

I_C ជាតម្លៃចរន្តឆ្លងកាត់ជើង Collector របស់ Transistor

ឧទាហរណ៍ថា មានចរន្តឆ្លងកាត់ Base = 1mA ដូច្នេះវាបង្កើតឱ្យមានចរន្តឆ្លងកាត់

Collector = 100mA ។ បើសិនជា Gain = 100 ។



- ចរន្ត Emitter (I_E)

$$I_E = I_C + I_B \quad \text{ឬ} \quad I_C = \alpha I_E \quad \text{ដែល} \quad \alpha = \frac{\beta}{\beta + 1}$$

ទំនាក់ទំនងរវាងត្រង់ស៊ីស្ទ័រ NPN and PNP

Voltage relation	NPN Mode	PNP Mode
$V_E < V_B < V_C$	Active	Reverse
$V_E < V_B > V_C$	Saturation	Cutoff
$V_E > V_B < V_C$	Cutoff	Saturation
$V_E > V_B > V_C$	Reverse	Active

• ការជ្រើសរើសត្រង់ស៊ីស្ទ័រ

មុនដំបូងគេធ្វើដូចម្តេចដើម្បីអាចជ្រើសរើស Transistor ណាមួយយកមកប្រើបានបើសិនជាវាមានច្រើនប្រភេទដូចនេះ ។ ខាងក្រោមជាចំណែកនៃការដកស្រង់ចេញពី Datasheet មួយរបស់ Transistor C1815 ហើយថាតើអ្នកត្រូវសិក្សាពីអ្វីខ្លះមុនគេ ៖

FEATURES

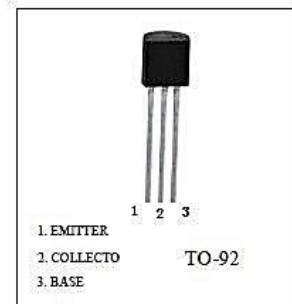
Power dissipation

MARKING:C1815

MAXIMUM RATINGS (TA=25 °C unless otherwise noted)

Parameter	Symbol	Value	Unit
Collector-Base Voltage	V_{CBO}	60	V
Collector-Emitter Voltage	V_{CEO}	50	V
Emitter-Base Voltage	V_{EBO}	5	V
Collector Current -Continuous	I_C	150	mA
Collector Power Dissipation	P_C	400	mW
Junction Temperature	T_J	150	°C
Storage Temperature	T_{stg}	-55-150	°C

C1815 (NPN)



របៀបសំគាល់ PNP

2N2222 BC547

1.E 1.C

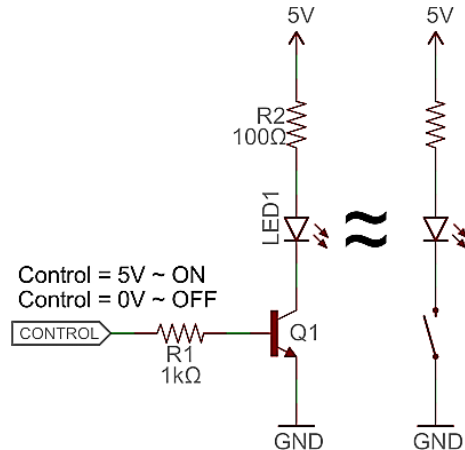
2.B 2.B

3.C 3.E

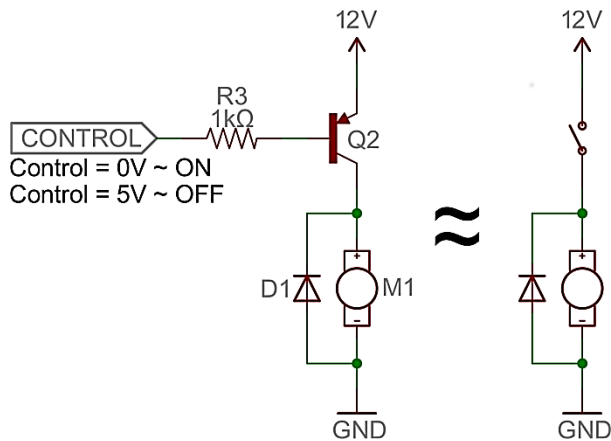
• ការប្រើប្រាស់ត្រង់ស៊ីស្ទ័រ

- កុងតាក់

មួយក្នុងចំណោមការងារជាច្រើនរបស់ត្រង់ស៊ីស្ទ័រនោះគឺវាដើរតួរជាកុងតាក់អេឡិចត្រូនិចសម្រាប់បិទបើកលំហូរចរន្តរបស់គ្រឿងទទួល ។ ដំណើរការធ្វើឡើងលើ Cut-off និង Saturation mode របស់ត្រង់ស៊ីស្ទ័រ ។ ជាមួយគ្នានេះដែរ វាក៏អាចបង្កើតបានជាកុងតាក់ Binary on/off ផងដែរ ។ ចូរក្រលេកមើលរូបខាងក្រោម៖



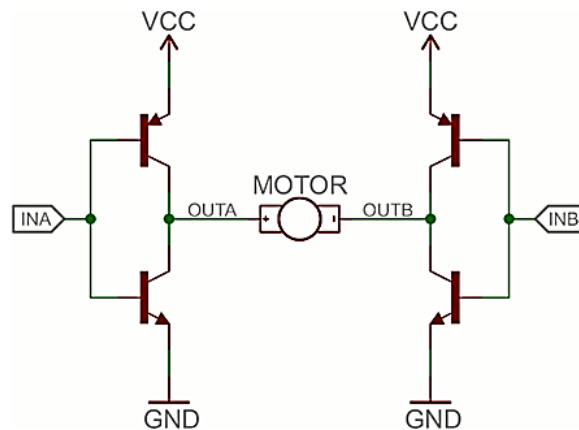
Controller របស់យើងបញ្ជូនស៊ីញ៉ាល់ចូលតាមជើង Base ដោយអនុវត្តតាមរូបមន្តនិងវិធានខាងលើ ត្រង់ស៊ីស្ទ័របានក្លាយជាកុងតាក់នៃអំពូល LED ។ ខណៈដែលកុងតាក់ធម្មតាជាកុងតាក់មេកានិច ដែលតម្រូវឱ្យអ្នកប្រើប្រាស់ត្រូវចុចដើម្បីភ្ជាប់ភ្លើង ឬ បិទភ្លើង ត្រង់ស៊ីស្ទ័រគឺប្រើតង់ស្យុងដើម្បីបិទបើក ដូច្នេះហើយវាមានលក្ខណៈប្រសើរក្នុងការប្រើប្រាស់ជាមួយពួក Microcontroller ដូចជា Arduino , PLC ,... ។ ល ។ ដែលសញ្ញាល់របស់វាគឺ Digital (0/1) ក្នុងការបិទឬ បើក ត្រង់ស៊ីស្ទ័រ ។



ដើម្បីប្រើម៉ូទ័រឱ្យវាវិលបានទៅមុខទៅក្នុងគេត្រូវប្រើម៉ូទ័រ 4 ក្នុងករណីនេះគេមិនអាចប្រើកុំព្យូទ័រដើម្បីធ្វើតេស្តបានទេព្រោះវាអាចធ្វើឱ្យនេះកុំព្យូទ័របើវាដោយផ្ទាល់ឬអំពែលឡើងតម្លៃធំ

-H-Bridge

វាជាបង្កើតត្រង់ស៊ីស្ទ័រដែលមានត្រង់ស៊ីស្ទ័របួស ប្រើសម្រាប់បញ្ជាទីរថាមពលខ្ពស់ជាង ក្នុងគោលបំណងប្តូរទិសម៉ូទ័ររិល ។



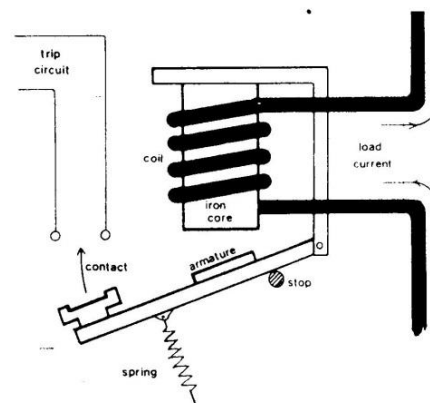
តារាងខាងក្រោមនេះបង្ហាញពីដំណើរការរបស់វាអាស្រ័យលើ INA និង INB

InputA	InputB	OutputA	OutputB	Motor Direction
0	0	1	1	Stopped (Braking)
0	1	1	0	Clockwise
1	0	0	1	Counter-clockwise
1	1	0	0	Stopped(Braking)

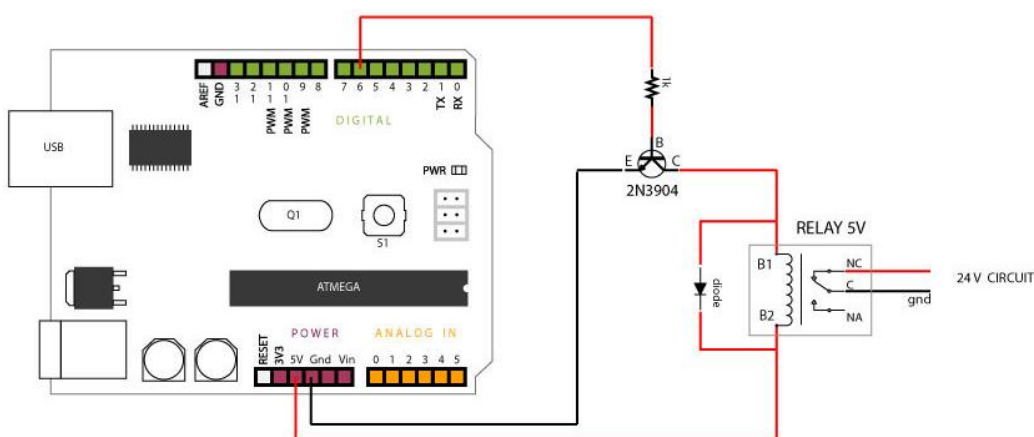
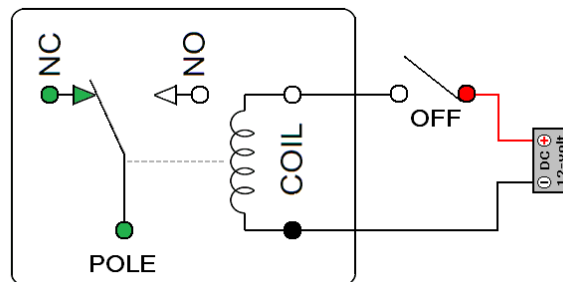
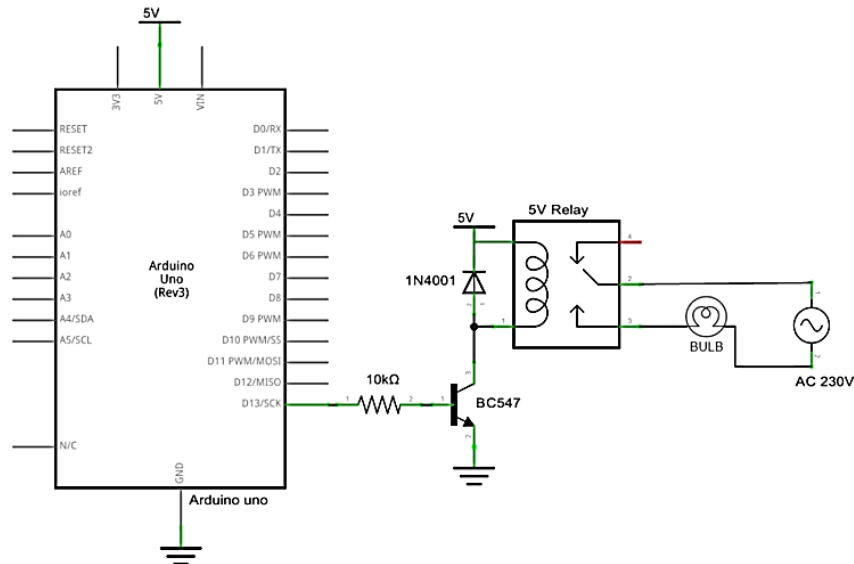
លំហាត់ ៖ ចូរសរសេរកូដដើម្បីឱ្យម៉ូទ័ររិល 10វិនាទីទៅឆ្វេង និង 10វិនាទីទៅស្តាំ ។

II. រ៉ឺឡេ (Relay)

Relay ត្រូវបានគេស្គាល់ជាយូរណាស់មកហើយ គេថាវាជាកុងតាក់អគ្គិសនីដែលមានរ៉ឺឡេ មួយ ហើយកុងតាក់របស់វាធ្វើការភ្ជាប់អគ្គិសនីនៅពេលមានចរន្តផ្គត់ផ្គង់ដល់រ៉ឺឡេរបស់វា ។
ខាងក្រោមនេះជាគំនូសបំព្រួញរបស់វា



ដោយសារតែវាជាកុងតាក់ដូច្នេះហើយ វាអាចភ្ជាប់ភ្លើងឱ្យឧបករណ៍ជាច្រើនដូចជា ម៉ូទ័រ អំពូល ម៉ាស៊ីន ... ។ ល។ ប៉ុន្តែ Arduino មិនអាចបញ្ជាដោយផ្ទាល់ទៅកាន់ឧបករណ៍ទាំងនេះបានទេ អញ្ចឹងហើយវាត្រូវបញ្ជាទៅកាន់ Relay ជាមុនសិន ។ តែទោះជាយ៉ាងក្តី Relay ភាគច្រើនគឺមានអនុភាពធំ ហើយយើងត្រូវការបញ្ជាវាតាមរយៈត្រង់ស៊ីស្ទ័រវិញ ពីព្រោះត្រង់ស៊ីស្ទ័រអាចពង្រីកសញ្ញាលំដាប់ខ្លោងរបស់ Arduino បាន ។



III.Digital Input

1.Comparison Operation

ប្រភេទ Comparison Operation	របៀបប្រើ
==	គេប្រើសម្រាប់ប្រៀបធៀបតម្លៃលេខតើស្មើគ្នា បើសិនស្មើគ្នាវានឹងចូលលក្ខណមួយទៀត ។
!=	គេប្រើសម្រាប់ប្រៀបធៀបតម្លៃលេខតើផ្ទុយគ្នាបើសិនផ្ទុយគ្នាវានឹងចូលលក្ខណមួយទៀត ។
<	គេប្រើសម្រាប់ប្រៀបធៀបតម្លៃលេខមួយ តូចជា តម្លៃមួយទៀត បើសិនជាតូចជាងវានឹងចូលលក្ខណមួយទៀត ។
>	គេប្រើសម្រាប់ប្រៀបធៀបតម្លៃលេខមួយ ធំជាង តម្លៃមួយទៀត បើសិនជាធំជាងវានឹងចូលលក្ខណមួយទៀត ។
<=	គេប្រើសម្រាប់ប្រៀបធៀបតម្លៃលេខមួយ តូចជាងឬស្មើ តម្លៃមួយទៀត បើសិនជាតូចជាងឬស្មើវានឹងចូលលក្ខណមួយទៀត ។
>=	គេប្រើសម្រាប់ប្រៀបធៀបតម្លៃលេខមួយ ធំជាងឬស្មើ តម្លៃមួយទៀត បើសិនជាធំជាងឬស្មើវានឹងចូលលក្ខណមួយទៀត ។

2.Bitwise Operator

ជាអនុគមន៍តក្កវិទ្យាដែលរួមមាន ៖ AND , OR , NOT , XOR , SHIFT LEFT , SHIFT RIGHT , ។
ប្រតិបត្តិការទាំងនោះធ្វើឡើងតែនៅលើតម្លៃ bit ។ ប្រតិបត្តិការនេះត្រូវបានគេហៅថា Logic Gate ។ ខាងក្រោមជាតារាងតក្កៈ

$$A \text{ AND } B = A.B$$

A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

$$A \text{ OR } B = A+B$$

A	B	A+B
0	0	0
0	1	1

1	0	1
1	1	1

$$A \text{ XOR } B = A \oplus B$$

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

$$\text{NOT}(A) = \overline{A}$$

A	$\overline{A}$
0	1
1	0

Arduino Symbol	
AND	&
OR	!
NOT	~
XOR	^
Shift Left	>>
Shift Right	>>

3.Boolean Operation

ត្រូវបានគេប្រើសម្រាប់តេស្តមើលក្លែងក្លាយ ឬច្រើន ។ វាមានលក្ខណៈខុសគ្នាពី Bitwise Operation ព្រោះវាជាការប្រៀបធៀបលក្ខណមិនមែនជាការគណនាតួរិទ្ធនោះឡើយ ។

Arduino Symbol		មុខងារ
AND	&&	ប្រៀបធៀបលក្ខណក្លែងក្លាយ ឬច្រើន បើដូចគ្នាទាំងអស់គឺពិត ផ្ទុយទៅវិញបើមានតែមួយខុសគឺ មិនពិត ។
OR		ប្រៀបធៀបលក្ខណក្លែងក្លាយ ឬច្រើនបើមានមួយណាក្នុងចំណោមនោះពិត គឺ ពិត ផ្ទុយទៅវិញបើទាំងអស់ខុសគឺ មិនពិត

NOT	!	វាពិតនៅពេលតម្លៃលេខវាផ្ទុយពីតម្លៃបច្ចុប្បន្ន
-----	---	---

4. Compound Operator

ជាប្រមាណវិធីគណិតវិទ្យាដែលត្រូវបានគេបង្កើតដើម្បីសរសេរឱ្យបានលឿន ចំនាយពេលតិចឱ្យឆាប់ចំពោះប្រមាណវិធីបណ្តាក់គ្នាលើអថេរតែមួយ ។

++	x++ ;	បូក x ដោយតម្លៃមួយ តែលទ្ធផលទទួលបានគឺយកតម្លៃចាស់មុនបូក1
	++x;	បូក x ដោយតម្លៃមួយតែលទ្ធផលទទួលបានគឺយកតម្លៃថ្មីក្រោបូក1 ។
--	x- ;	ដក x ដោយតម្លៃមួយតែលទ្ធផលទទួលបានគឺយកតម្លៃចាស់មុនដក 1
	--x;	ដក x ដោយតម្លៃមួយតែលទ្ធផលទទួលបានគឺយកតម្លៃថ្មីក្រោដក1 ។
+=	x += y	x = x + y;
-=	x -= y	x = x - y;
*=	x *= y	x = x* y;
/=	x /= y	x = x/ y;
%=	x % = y	x = x% y;
&=	x & = y	x = x& y;
=	x = y	x = x y;

5.Control structure

if(conditional) and ==,!=,<,> (comparison operators)

if ត្រូវបានប្រើសម្រាប់ប្រតិបត្តិការប្រៀបធៀបព្រឹត្តិការណ៍ពីរ ឬច្រើនជាមួយគ្នា ។ បើ ព្រឹត្តិការណ៍ទាំងនោះពិតជា កើតមានមែនវានឹងឱ្យជាផលអ្វីដូចកូដដែលគេសរសេរចង់បាន ។

Syn

```
if (someVariable>50){//do something here}
```

Example:

Way1: `if(x>120)digitalWrite(LEDpin,HIGH);`

Way2: `if(x>120){digitalWrite(LEDpin,HIGH);}`

Way3: `if(x>120){
digitalWrite(LEDpin1,HIGH);
digitalWrite(LEDpin2,HIGH);
}//all are correct`

5.1. if/else

else ត្រូវបានប្រើជាមួយ if ដែលលទ្ធផលរបស់វាធ្វើការនៅពេលព្រឹត្តិការណ៍នៅក្នុង if មិនពិត ឬ មិនកើតឡើង នោះវានឹងលោតទៅធ្វើក្នុងកូដរបស់ else ។

**Syn
tax**

```
if( someVariable>50){// do action A  
}else{// do action B }
```

Example :

```
if(pinFiveInput<500){// do Thing A  
}else if(pinFiveInput>=1000){//do Thing B  
}else{//do Thing C  
}
```

5.2.switch/case Statement

ស្រដៀងគ្នានឹង if តែវាមានដំណើរការខុសគ្នាត្រង់វាយកត់តម្លៃនៃអថេរមួយទៅ ធៀបនឹងតម្លៃកំណត់ជាច្រើន ហើយប្រសិនបើតម្លៃដែលធៀបនោះត្រូវនឹងតម្លៃណាមួយ ក្នុងចំណោមតម្លៃជាច្រើននោះ នៅវាធ្វើប្រតិបត្តិការកូដត្រង់ចំណុចនោះតែម្តង ។ ជាមួយ គ្នានេះដែរ ពាក្យគន្លឹះ break ត្រូវបានប្រើប្រាស់ដើម្បីបញ្ចប់ប្រតិបត្តិការនៅរាល់ករណី នីមួយៗរបស់តម្លៃកំណត់នោះក្រោយពេលវាបានធ្វើការដោយតម្លៃរបស់វាធៀបនឹងតម្លៃ អថេរមួយរួមគឺត្រូវគ្នា ។

Syntax

```
switch(var){case lable://statements
            break;
            case lable://statements
            break;
            default://statements
            break; }
```

Parameter :

var: ជាអថេរដែលតម្លៃរបស់វាត្រូវបានធៀបនឹងតម្លៃករណីនីមួយៗ

lable: តម្លៃដែលត្រូវយកទៅធៀបនឹងអថេរខាងលើ

Example:

```
switch(var){
    case1://do something when var equal1 break;
    case2:// do something when var equal2 break;
    default://if nothing else matches, do the default
    //default is optional break;}
```

5.3.While loops

- While loops ធ្វើការដដែលៗរាល់ពេលដែល Expression របស់វាពិត ហើយផ្ទុយទៅវិញរាល់ដង ។

Syntax

```
While(expression){//statement(s);}
```

Parameter:

Expression : ប្រើexpression ពិតនោះ while នឹងធ្វើប្រតិបត្តិការវិលទៅវិលមក ។

Example :

```
var=0;
while(var<200){// do something repetitive 200 time
    var++;}
```

5.4.do-while

ធ្វើការស្រដៀងគ្នានឹង while loops ដែរ តែខុសត្រង់ធ្វើការសិនបានតេស្តមើលលក្ខណៈជាក្រោយ ។

Syntax

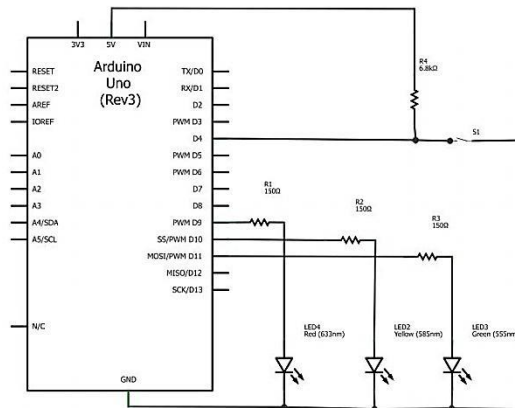
```
do{//statement block
}while(test condition);
```

Example:

```
do{delay(50); //wait fo sensors to stabilize
  x=readSensors(); //check the sensors
}while(x<100);
```

6.Digital Input arduino

ខាងក្រោមជាសៀគ្វីងាយមួយបង្ហាញពីការប្រើប្រាស់ input ចូលទៅកាន់ Arduino តាម Digital Pin របស់វា ៖



Example:

```
void setup(){
  pinMode(4,OUTPUT);
  pinMode(2,INPUT);
}
void loop(){
  if(digitalRead(2)==1){
    digitalWrite(4,HIGH);
  }
}
```

Syntax

`digitalRead(pin,number);`

IV.Filter

Filter ប្រើសម្រាប់ប្រែ៖ signal ។ Filter មានពីរប្រភេទ គឺ៖

Low pass Filter : ប្រើសម្រាប់ចាប់យកប្រេកង់ក្រោមជាងឬស្មើ

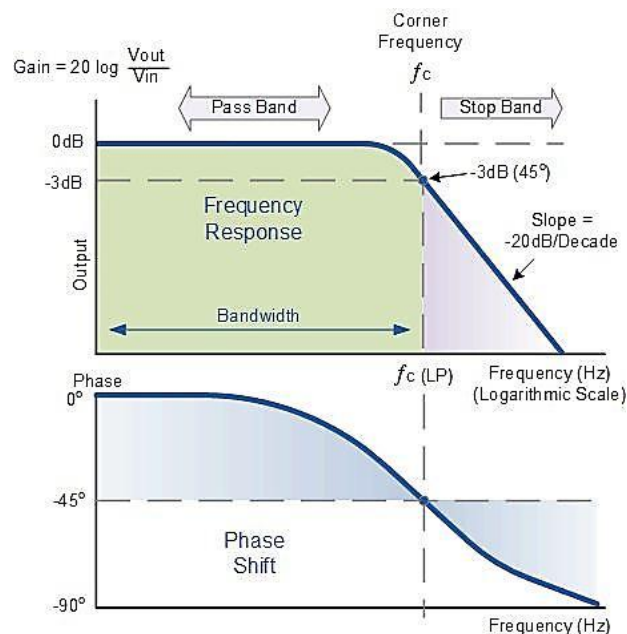
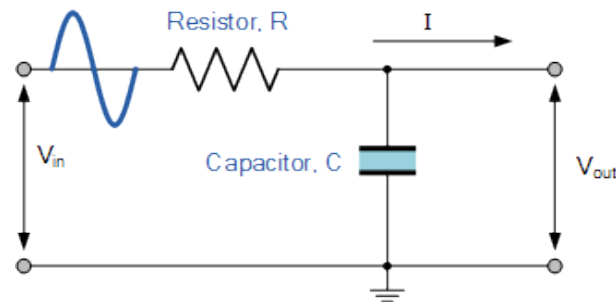
High pass Filter : សម្រាប់ចាប់យកប្រេកង់ដែលខ្ពស់ជាង

1.Hardware Filter

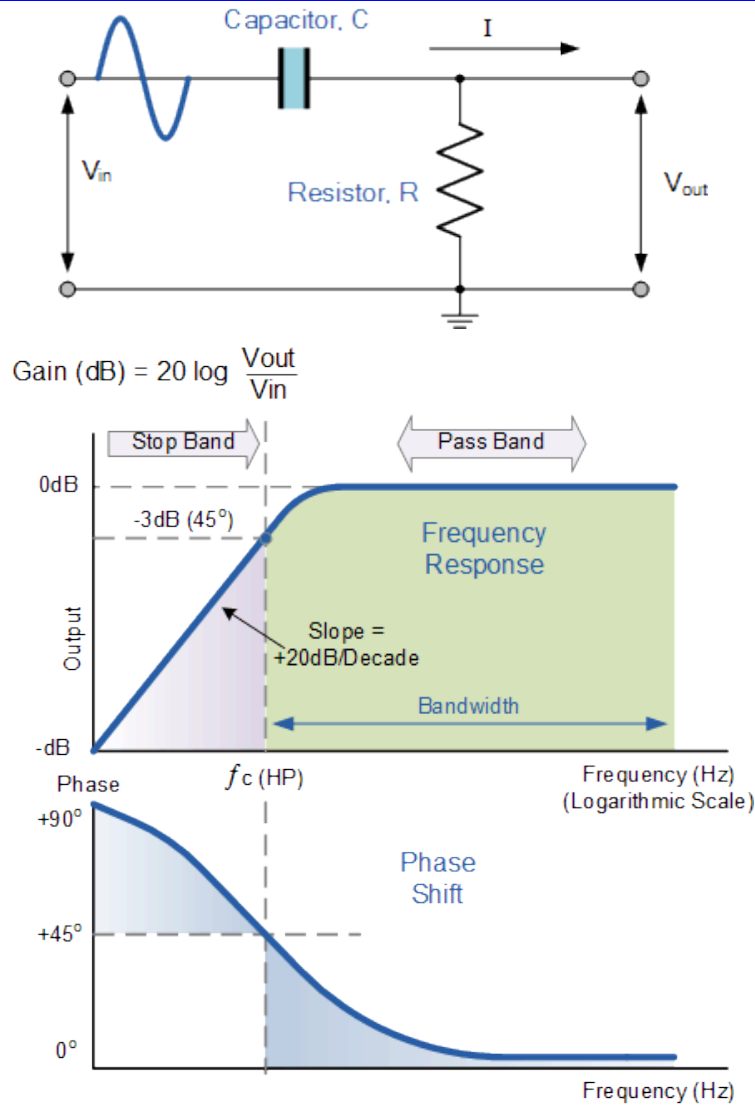
កាលពីការសិក្សាមុនអំពី counter យើងបានជួបប្រទះឃើញថាមាន Noise កើតឡើងនៅលើប៊ូតុងចុចរបស់សៀគ្វី ហេតុដូច្នេះហើយការសិក្សាពី Filter មានសារៈសំខាន់ណាស់ក្នុងការកាត់បន្ថយ Noise ទាំងនោះ ។ នៅក្នុងចំណុចនេះយើងនឹងសិក្សាបន្ថែមទៀតពី Filter ។ Filter ជាឧបករណ៍ម្យ៉ាងដែលមានតួនាទីក្នុងការច្រោះយកកម្រិតសញ្ញាលក់ក្នុងកម្រិតប្រេកង់ណាមួយ ។ គេចែក Filter ជាប្រភេទគឺ Low Pass Filter , High Pass Filter and Band Pass Filter, Band Stop Filter ។ ក្នុងករណីនេះ យើងនឹងសិក្សាតែលី Filter 1st Order តែប៉ុណ្ណោះ ។ រូបមន្ត Frequency cutoff របស់វាគឺ

$$f_c = \frac{1}{2\pi RC}$$

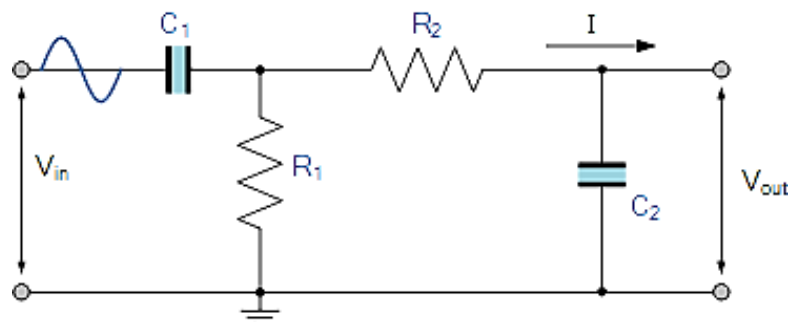
សៀគ្វី Low Pass Filter : ប្រេកង់ទាបជាង Freq Cutoff ឆ្លងកាត់បាន

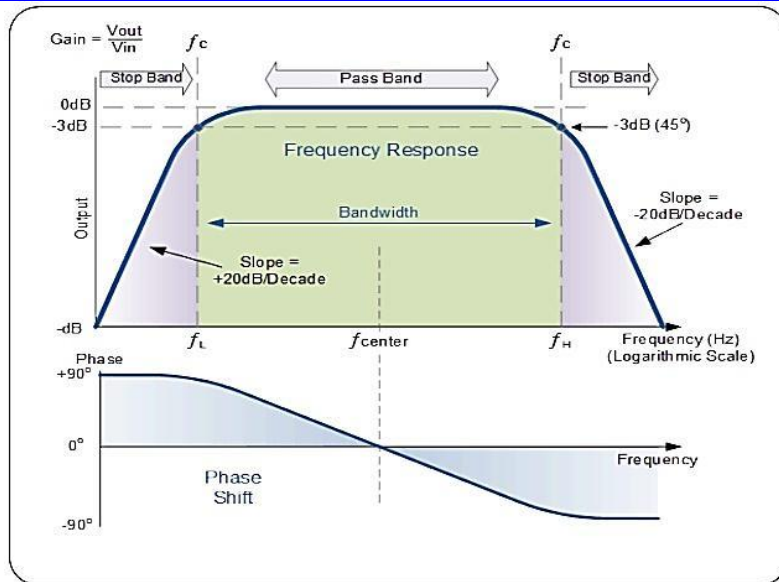


*សៀគ្វី High Pass Filter : ប្រេកង់ខ្ពស់ជាង Freq Cutoff អាចឆ្លងកាត់បាន



សៀគ្វី Band Pass Filter: ប្រភេទនេះ ប្រើប្រាស់ ២ ប្រភេទ ប្រេកង់ ទាំងពីរ ទើបអាចឆ្លងកាត់បាន ។ ប្រភេទនេះ ហៅថា Centre Frequency





$$f_r = \sqrt{f_L \cdot f_H}$$

2. Software Filter

ដោយសារប្រភេទថ្មីច្រើនប្រភេទយើងបង្កើតឱ្យមាន Debouncer ដូចនេះយើងអាចកាត់បន្ថយវាបានតាមរយៈសៀគ្វីខាងក្រៅ ឬការសរសេរកូដពីខាងក្នុង ។ ខាងក្រោមជាគំរូកូដ សម្រាប់កាត់ Debouncer ។

```
int lastButtonState=LOW;
unsigned long lastDebounceTime=0;
unsigned long debounceDelay=50;
void setup(){
    pinMode(2,OUTPUT);
}
Void loop(){
    int reading=digitalRead(2);
    if(reading!=lastButtonState){
        lastDebounceTime=millis();
    }
    if ((millis()-lastDebounceTime)>debounceDelay){//Write your code here;
        }
        lastButtonState=reading;
    }
}
```

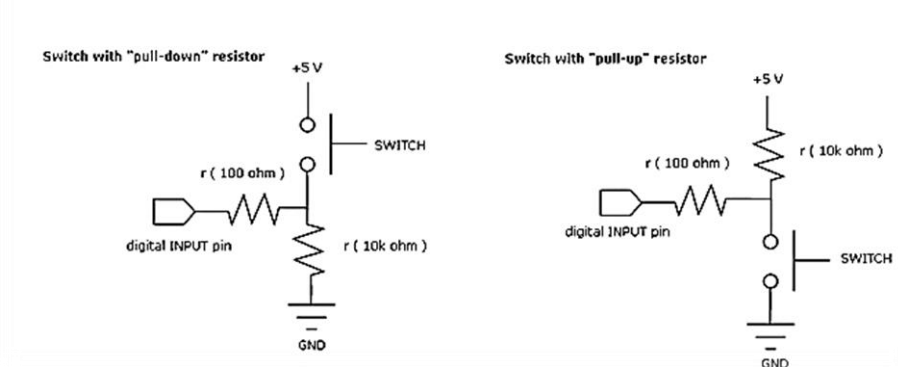
ក្នុងជីមួយទៀតត្រូវបានប្រទះឃើញ នោះគឺ millis() ត្រូវបានប្រើសម្រាប់រយៈពេលគិតជាមីលីវិនាទី ដែលចាប់រាប់តាំងពីពេល Arduino ត្រូវបានផ្គត់ផ្គង់ភ្លើងឱ្យរហូតដល់វាត្រូវបានគេផ្ដាច់ភ្លើងវិញ ។ គួរបញ្ជាក់ថាវាអាចរាប់បានរយៈពេល 50 ថ្ងៃទើប reset មក 00 វិញ ។

V. Push Button

អ្នកនឹងរៀនពីការតសៀវគ្គនិងសរសេរកូដដើម្បីប្រើប្រាស់ប៊ូតុង

1.ការប្រើប៊ូតុងចុចមួយ

ដំណើរការរបស់ Push Button



ក្នុងករណីនេះយើងត្រូវប្រើស៊ីស្តង់មួយដោយភ្ជាប់ពីជើង input ភ្ជាប់ទៅម៉ាស ព្រោះវាមាន noise នោះវាមានការរំខាននៃចរន្ត នោះការភ្ជាប់ស៊ីស្តង់ដើម្បីកាត់បន្ថយការរំខាននៅពេលចុច

```
void setup() {
  pinMode(2,INPUT);
  pinMode(3,OUTPUT);
}
void loop() {
  if (digitalRead(2)==1){
    digitalWrite(3,HIGH);
  }
}
```

លំហាត់

- 1) ចូរសរសេរកូដដោយប្រើប៊ូតុងចុចម្តងភ្លឺម្តងរលត់ ។
- 2) ចូរសរសេរកូដដោយប្រើប៊ូតុងចំនួនពីរសម្រាប់រាប់លេខចុះ និងឡើងដោយប្រើ Max7219

2.Two buttons LED on and off

```
void setup() {
```

```

pinMode(2,INPUT);
pinMode(3,OUTPUT);
pinMode(4,INPUT);
}
void loop() {
  if (digitalRead(2) == 1){
    digitalWrite(3,HIGH);
  }
  else if (digitalRead(4) == 1){
    digitalWrite(3,LOW);
  }
}

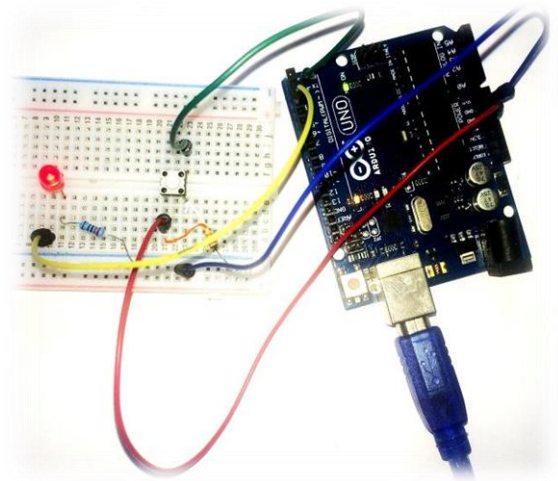
```

3. One button LED on and off:

```

int i = 1;
void setup() {
  pinMode(2,INPUT);
  pinMode(3,OUTPUT);
  // pinMode(4,INPUT);
}
void loop() {
  if (digitalRead(2) == 1){
    digitalWrite(3,i);
    i=!i;
    delay(200); //for debounce
  }
}

```



4. One digit count up 0-9

```

const int din = 5;
const int load = 6;
const int clk = 7;
int i = 0;
void setup() {
  pinMode(2,INPUT);
  pinMode(din,OUTPUT);
  pinMode(clk,OUTPUT);
  pinMode(load,OUTPUT);
  Piseth(0x0c,0x01);
  Piseth(0x09,0x0f);
  Piseth(0x0a,0x05);
}

```

```

    Piseth(0x0b,0x03);
}
void Piseth(int address,int data){
    digitalWrite(load,LOW);
    shiftOut(din,clk,MSBFIRST,address);
    shiftOut(din,clk,MSBFIRST,data);
    digitalWrite(load,HIGH);
}
void loop() {
    if (digitalRead(2) == 1){
        Piseth(1,i);
        i++;
        delay(200);
    }
}

```

5. ប្តូរកូដពីរ មួយ count 0-9 និង មួយទៀត 9-0 (មួយខ្ទង់)

```

const int din = 5;
const int load = 6;
const int clk = 7;
int i = 9;
int k=0;
void setup() {
    pinMode(2,INPUT);
    pinMode(3,INPUT);
    pinMode(din,OUTPUT);
    pinMode(clk,OUTPUT);
    pinMode(load,OUTPUT);
    Piseth(0x0c,0x01);
    Piseth(0x09,0x0f);
    Piseth(0x0a,0x05);
    Piseth(0x0b,0x03);
}
void Piseth(int address,int data){
    digitalWrite(load,LOW);
    shiftOut(din,clk,MSBFIRST,address);
    shiftOut(din,clk,MSBFIRST,data);
    digitalWrite(load,HIGH);
}
void loop() {
    if (digitalRead(2) == 1){
        Piseth(1,i);

```



```

    i--;
    delay(200);
}
if (digitalRead(3)==1){
    Piseth(1,k);
    k++;
    delay(200);
}
}

```

6. ប្តូរកូដមួយចុចពី 00-99

```

const int din = 5;
const int load = 6;
const int clk = 7;
int i = 0;
void setup() {
    pinMode(2,INPUT);
    pinMode(3,INPUT);
    pinMode(din,OUTPUT);
    pinMode(clk,OUTPUT);
    pinMode(load,OUTPUT);
    Piseth(0x0c,0x01);
    Piseth(0x09,0x0f);
    Piseth(0x0a,0x05);
    Piseth(0x0b,0x03);
}
void Piseth(int address,int data){
    digitalWrite(load,LOW);
    shiftOut(din,clk,MSBFIRST,address);
    shiftOut(din,clk,MSBFIRST,data);
    digitalWrite(load,HIGH);
}
void loop() {
    if (digitalRead(2) == 1){
        int d1=i%10;
        int d2=i/10;
        Piseth(1,d1);
        Piseth(2,d2);
        i++;
        delay(200);
    }
}

```

```

    }
}

```

7. ប្តូរពី មួយចុចពី 00-99 មួយទៀត 99-00

```

const int din = 5;
const int load = 6;
const int clk = 7;
int i = 0;
int k=99;
void setup() {
    pinMode(2,INPUT);
    pinMode(3,INPUT);
    pinMode(din,OUTPUT);
    pinMode(clk,OUTPUT);
    pinMode(load,OUTPUT);
    Piseth(0x0c,0x01);
    Piseth(0x09,0x0f);
    Piseth(0x0a,0x05);
    Piseth(0x0b,0x03);
}
void Piseth(int address,int data){
    digitalWrite(load,LOW);
    shiftOut(din,clk,MSBFIRST,address);
    shiftOut(din,clk,MSBFIRST,data);
    digitalWrite(load,HIGH);
}
void loop() {
    if (digitalRead(2) == 1){
        int d1=i%10;
        int d2=i/10;
        Piseth(1,d1);
        Piseth(2,d2);
        i++;
        delay(200);
    }
    if (digitalRead(3) == 1){
        int d1=k%10;
        int d2=k/10;
        Piseth(1,d1);
        Piseth(2,d2);
    }
}

```

```

    k--;
    delay(200);
  }
}

```

យើងអាចដាក់រូបមន្តអញ្ញាត i ដើម្បីងាយស្រួល

```

const int din = 5;
const int load = 6;
const int clk = 7;
int i = 0;
void setup() {
  pinMode(2,INPUT);
  pinMode(3,INPUT);
  pinMode(din,OUTPUT);
  pinMode(clk,OUTPUT);
  pinMode(load,OUTPUT);
  Piseth(0x0c,0x01);
  Piseth(0x09,0x0f);
  Piseth(0x0a,0x05);
  Piseth(0x0b,0x03);
}
void Piseth(int address,int data){
  digitalWrite(load,LOW);
  shiftOut(din,clk,MSBFIRST,address);
  shiftOut(din,clk,MSBFIRST,data);
  digitalWrite(load,HIGH);
}
void loop() {
  if (digitalRead(2) == 1){
    i++;
    delay(200);
  }
  else if (digitalRead(3) == 1){
    i--;
    delay(200);
  }
  int d1=i%10;
  int d2=i/10;
  Piseth(1,d1);
  Piseth(2,d2);
}

```

}

របៀបធ្វើឱ្យInput_PULLUP ជាអថេរមួយដែលយើងមិនប្រើស៊ីស្តង់ទេ ដោយសារក្នុង arduino មានស៊ីស្តង់ហើយ វានឹងប្រើ

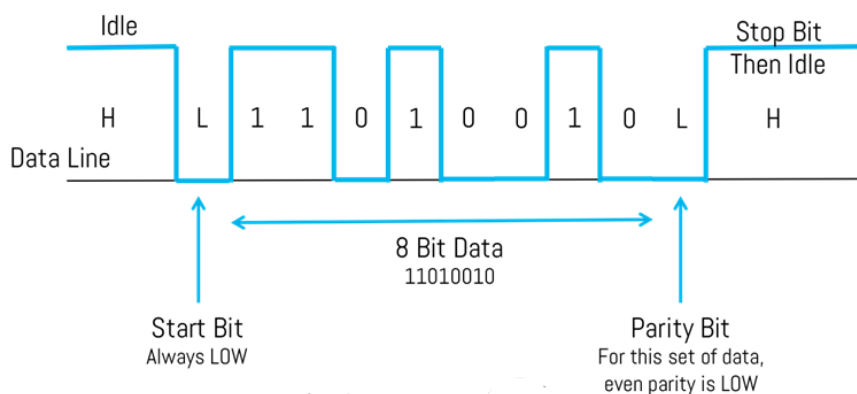
```
int button = 6;
int led = 5;
int state = LOW;
int reading;
int previous = LOW;
long time = 0;
long debounce = 200;
void setup()
{
  pinMode(button, INPUT_PULLUP);
  pinMode(led, OUTPUT);
}
void loop()
{
  reading = digitalRead(button); LOW
  if (reading == HIGH && previous == LOW && millis() - time > debounce) {
    if (state == HIGH)
      state = LOW;
    else
      state = HIGH;
    time = millis();
  }
  digitalWrite(led, state);
  previous = reading;//LOW
}
```

ជំពូកទី៤

Serial Monitor

I. Serial Protocol ជាអ្វី?

វាជារបៀបមួយដែល កញ្ចប់ទិន្នន័យត្រូវបានបោះចេញម្តងមួយៗ ក្នុងរយៈពេលថេរ រយៈពេលថេរនោះត្រូវបានហៅថា Baudrate ។ ស្តង់ដារ Baudrate មានដូចជា 300,600,1200,2400,4800,9600,14400,19200,28800,38400,57600, ឬ 115200 ។ រូបខាងក្រោមបង្ហាញពីកញ្ចប់ទិន្នន័យដែល Protocol Serial ប្រើ



II. Serial of Arduino

1.Function of Syntax និងឧទាហរណ៍

Arduino IDE ដែលយើងប្រើមានភ្ជាប់មកជាមួយនូវ Serial Monitor ដែលអាចឱ្យយើងធ្វើតេស្តជាមួយ Serial code របស់វា ឬក៏ Debug ជាមួយឧបករណ៍ផ្សេងៗបាន ។ ដើម្បីអាចប្រើបានយើងត្រូវយល់ដឹងអំពី Function of Syntax មួយចំនួនរបស់វាដូចតទៅ ៖

- `if(Serial)`
- `available()`
- `begin()`
- `end()`
- `find()`
- `findUntil()`
- `print()`
- `read()`
- `readBytes()`

- `getBytesUntil()`
- `readString()`
- `readStringUntil()`
- `setTimeout()`
- `write()`
- `serialEvent()`
- ❖ `begin()`: បើកឱ្យ Serial port របស់ Arduino ដំណើរការជាមួយ Baudrate ណាមួយ

Syntax**`Serial.begin(Baudrate);`**

Example:

```
void setup(){
  Serial.begin(115200); //Initialize serial with baudrate 115200;
}
```

- ❖ `if(Serial)`: ប្រើសម្រាប់តេស្តថា តើ Serial Port ត្រូវភ្ជាប់ឬនៅ ។

Example:

```
void setup(){
  Serial.begin(9600); //Initialize serial and wait for port to
  open;
  while(!Serial){ //wait for serial port to connect. Needed for
  negative USB
  }
}
void loop(){ //proceed normally
}
```

- ❖ `available()`: សម្រាប់តេស្តមើលថា មានទិន្នន័យនៅក្នុង Buffer ឬនៅមុននឹងអានយក ទិន្នន័យនោះ

```
void setup(){
  Serial.begin(9600);
}
void loop(){
  if(Serial.available() > 0){
    int x = Serial.read();
  }
}
```

- ❖ `end()`: ប្រើសម្រាប់បិទប្រតិបត្តិការណ៍ជើង Serial ហើយឱ្យវាភ្លាយជាជើង Digital In/Out

ធម្មតា

ឧទាហរណ៍

```
void setup(){
  Serial.en();
  pinMode(0, INPUT);
  pinMode(1, INPUT);
}
```

- ❖ `find()`: ប្រើសម្រាប់ស្វែងរកពាក្យ (string) ណាមួយក្នុង Buffer ។ ប្រសិនបើរកឃើញពាក្យនោះវានឹង return true ផ្ទុយទៅវិញ false នៅពេលដល់ timeout

Syntax

```
Serial.find(targetString);
```

ឧទាហរណ៍

```
void setup(){
    Serial.begin(9600);}
void loop(){
    if(Serial.available()>0){
        if(Serial.find("OK")){
            int x=Serial.read();
        }
    }
}
```

- ❖ `findUntil()`: ប្រើប្រាស់សម្រាប់ស្វែងរកពាក្យ (string) ណាមួយក្នុង Buffer រហូតដល់រកពាក្យនោះឃើញ ឬមិនដូច្នោះទេ រហូតដល់រក terminal string ត្រូវបានរកឃើញ ។ ប្រសិនបើរកឃើញពាក្យទាំងនោះវានឹង return true ផ្ទុយមកវិញ false នៅពេលដល់ timeout

Syntax

```
Serial.findUntil(targetString,terminal);
```

ឧទាហរណ៍

```
void setup(){
    Serial.begin(9600);
}
void loop(){
    if(Serial.available()>0){
        if(Serial.findUntil("OK","##")){
            int x=Serial.read();
        }
    }
}
```

- ❖ `print()`: ប្រើសម្រាប់បញ្ជូនទិន្នន័យពី Arduino Serial របស់វាទៅខាងក្រៅ ឬឧបករណ៍ដទៃ

Syntax

```
Serial.print(Val);
Serial.print(Val,format);
```

ឧទាហរណ៍

```
Serial.print(78);//gives"78"
Serial.print(1.23456)// gives"1.23"
Serial.print('N');//gives"N"
Serial.print("Hello Cambodia");//gives "Hello Cambodia"
```

ជាមួយ Format

```
Serial.print(78,BIN) // gives "1001110"
Serial.print(78,DEC)//gives"78"
Serial.print(78,HEX)//gives "4E"
Serial.println(1.23456,0)//gives"1"
Serial.println(1.23456,4)//gives"1.2345"
```

- ❖ `println()`:ប្រើសម្រាប់បញ្ជូនទិន្នន័យពី Arduino Serial របស់វាទៅខាងក្រៅ ឬឧបករណ៍ដទៃ តែមានភ្ជាប់ទៅជាមួយនូវ ASCII (ASCII 10 or "\n") ចុះបន្ទាត់

ឧទាហរណ៍

```
int count;
void setup(){
  Serial.begin(9600);
}
void loop(){
  count++;
  Serial.println(count);
}
```

- ❖ `read()`:សម្រាប់ទាញយកទិន្នន័យ 1byte ពីក្នុង Buffer ។ ទិន្នន័យដែលទទួលបានគឺជាតម្លៃលេខ ។

ឧទាហរណ៍

```
void setup(){
  Serial.begin(9600);
}
void loop(){
  if(Serial.available()>0){
    int x=Serial.read();
  }
}
```


- ❖ `readBytes()`:សម្រាប់ទាញយកច្រើន Bytes ពីក្នុង Buffer ។ រួចដាក់ទៅក្នុង Array of byte[] or char[] ណាមួយដែលអ្នកតាងវា តាមប្រវែងតួអក្សរដែលយើងចង់បាន ។ Return ចំនួនតួអក្សរដែលទទួលបាន ។

Syntax`Serial.readBytes(buf,length);`**ឧទាហរណ៍**

```
char text[50];
void setup(){
    Serial.begin(9600);
}
void loop(){
    if(Serial.available()>0){
        int x=Serial.readBytes(text,10);
    }
}
```

- ❖ `readBytesUntil()`: ប្រើដូច `readBytes` ដែលតែខុសគ្នាត្រង់វាអាចយកទិន្នន័យរហូតដល់ Char Terminator ត្រូវបានរកឃើញ

Syntax`Serial.readBytesUntil(terminator,buf,length);`**ឧទាហរណ៍**

```
char tex[50];
void setup(){
    Serial.begin(9600);
}
void loop(){
    if(Serial.available()>0){
        int x=Serial.readBytes('X',text,10);
    }
}
```

- ❖ `readString()`:សម្រាប់អាចយកត្រប់ទិន្នន័យទាំងអស់ក្នុង Buffer រួចបំប្លែងជា ពាក្យ ។

Syntax`Serial.readString();`

ឧទាហរណ៍

```
String txt;
void setup(){
  Serial.begin(9600);
}
void loop(){
  if(Serial.available()>0){
    txt=Serial.readString();
  }
}
```

- ❖ `readStringUntil()`: សម្រាប់អានយកត្រប់ទិន្នន័យទាំងអស់ក្នុង Buffer រហូតដល់ String Terminator ត្រូវបានរកឃើញ រួចបំប្លែងជាពាក្យ ឬដល់ timeout

Syntax

```
Serial.readString(Terminator);
```

ឧទាហរណ៍

```
String txt;
void setup(){
  Serial.begin(9600);
}
void loop(){
  if(Serial.available()>0){
    txt=Serial.readStringUntil("OK");
  }
}
```

- ❖ `setTimeout()`: កំណត់រយៈពេលសម្រាប់ Arduino ក្នុងការធ្វើប្រតិបត្តិការអានទិន្នន័យ ប្រសិនបើវាមិនអាចអានក្នុងរយៈពេលនោះឱ្យអស់ទេ នោះវានឹងបាត់បង់ទិន្នន័យខ្លះ ។

Syntax

```
Serial.setTimeout(Duration);
```

ឧទាហរណ៍

```
String txt;
void setup(){
  Serial.begin(9600);
  Serial.setTimeout(100);
}
```

```
void loop(){
    if(Serial.available()>0){
        txt=Serial.readString("OK");
    }
}
```

❖ write(): សម្រាប់បញ្ជូនទិន្នន័យពី Arduino ទៅក្រៅ

ឧទាហរណ៍

```
byte x=90;
void setup(){
    Serial.begin(9600);
}
void loop(){
    Serial.write(x);
}
```

❖ serialEvent(): ជា Function មួយដែលវាធ្វើការនៅពេលដែលមាន ទិន្នន័យចូលមកក្នុង Buffer របស់ Arduino ។

ឧទាហរណ៍

```
void serialEvent(){//statements
}
```

លំហាត់

1. ចូរអ្នកសរសេរកូដមួយដែលនៅ ពេលអ្នកចុចប៊ូតុង up/down វាបញ្ជូនពាក្យនោះទៅកាន់ serial monitor ។

2. ចូរសរសេរកូដដែល serial monitor ធ្វើការបញ្ជូនទៅកាន់អំពូលខុសគ្នាឱ្យអាចបិទឬបើកបាន

ក្នុងវាយ command ដើម្បីឱ្យអំពូលភ្លឺពេលចុចលេខ2 ភ្លឺ និង លេខ3ភ្លឺ

```
void setup() {
    Serial.begin(9600);
    pinMode(2,OUTPUT);
    pinMode(3,OUTPUT);
}
void loop() {
    if (Serial.available()>0){
        int x = Serial.read()-'0';
        Serial.print(x);
        if (x==1){
            digitalWrite(3,HIGH);
        }
    }
}
```

```

    else if (x==2){
        digitalWrite(2,HIGH);
    }
    else {
        digitalWrite(2,LOW);
        digitalWrite(3,LOW);
    }
}
}

```

ការប្រើ String ដើម្បីឱ្យអំពូលភ្លឺ

```

void setup() {
    Serial.begin(9600);
    pinMode(2,OUTPUT);
    pinMode(3,OUTPUT);
}

void loop() {
    if (Serial.available()>0){
        String x = Serial.readString();
        Serial.print(x);
        if (x=="LED1"){
            digitalWrite(3,HIGH);
        }
        else if (x=="LED2"){
            digitalWrite(2,HIGH);
        }
        else {
            digitalWrite(2,LOW);
            digitalWrite(3,LOW);
        }
    }
}

```

2.String Objects

String ជាបណ្តុំនៃអក្សរជាច្រើនចូលគ្នាបង្កើតបានជាពាក្យ ។ អក្សរនីមួយៗមានតម្លៃស្មើ 1byte ហើយវាគោរពទៅតាមតារាង ASCII

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Arduino ផ្តល់ឱ្យនូវ Function មួយចំនួនសម្រាប់ប្រើប្រាស់ជាមួយ String នេះមានដូចជា ៖

- String()
- charAt()
- compareTo()
- concat()
- endsWith()
- equals()
- equalsIgnoreCase()
- getBytes()
- indexOf()
- lastIndexOf()
- length()
- remove()
- replace()
- setCharAt()
- startsWith()
- substring()
- toInt()
- toFloat()
- toLowerCase()
- toUpperCase()
- trim()
- ❖ String():សម្រាប់បង្កើតជា String ចេញពីData Type ផ្សេងៗ

Syntax

```
String(val);
String(val,base);
String(val,decimalPlaces);
```

```
String stringOne="Hello String";
String stringOne=String('a');//a"
String stringTwo=String("This is a string");
String stringOne=String(stringTwo+"with more");
String stringOne=String(13);
String stringOne=String(analogRead(0),DEC);
String stringOne=String(45,HEX);
String stringOne=String(255,BIN);
String stringOne=String(millis(),DEC);
String stringOne =String(5.698,2);// become string "5.69"
```

❖ `charAt()`: ថាបំយកតួអក្សរនៅត្រង់ Index ណាមួយ

Syntax

`string.charAt(n);`

❖ `compareTo()`: ប្រើសម្រាប់ប្រៀបធៀប string 2

Syntax

`string.compareTo(string2)`

Parameters

string: a variable of type String

string2: another variable of type String

Returns

a negative number: if string comes before string2

a positive number: if string comes after string2

0: if string equals string2

❖ `concat()`: ប្រើសម្រាប់បញ្ចូល String គ្នា

Syntax

`string.concat(parameter)`

ឧទាហរណ៍

```
String stringTwo=" A long integer:";
stringTwo.concat(123456789);//"A long integer:123456789"
```

❖ `endsWith()`: ត្រូវបានប្រើប្រាស់ដើម្បីពិនិត្យថា String បានបញ្ចប់ដោយពាក្យណាមួយឬអត់ ។

Syntax

`String.endsWith(string2)`

Parameter

string: a variable of type String

string2: another variable of type String

Returns:

true: if string ends with the characters of string2

false: otherwise

❖ equals(): ប្រៀបធៀប String 2 ថា តើស្មើគ្នា ឬអត់ ហើយអក្សរធំនិងតូចគឺខុសគ្នា ។

Syntax

`string.equals(string2)`

parameters:

string, string2: variables of type String

Returns:

true: if string equal string2

false: otherwise

❖ equalsIgnoreCase(): ប្រៀបធៀប String 2 ថា តើស្មើគ្នា ឬអត់ ហើយមិនគិតអក្សរធំ ឬតូចទេ

❖ getBytes(): ចម្លង string ជាកំរិតលក្ខណៈ Array ណាមួយ

Syntax

`string.getBytes(buf, len)`

Parameters:

string: a variable of type String

buf: the buffer to copy the characters into (byte [])

len: the size of the buffer (unsigned int)

❖ indexOf(): រក Index នៃអក្សរក្នុង String ។ រកពីមុខទៅមុន

Syntax

`string.indexOf(val)`
`string.indexOf(val, from)`

parameter

string: a variable of type String

val: the value to search for -char or String

from: the index to start the search from

Returns:

The index of val within the String, or -1 if not found.

❖ lastIndexOf(): រក Index នៃអក្សរក្នុង String ។ រកពីក្រោយទៅមុន

Syntax

```
string.lastIndexOf(val)
string.lastIndexOf(val,from)
```

Parameters

string: a variable of type String

val: the value to search for –char or String

from: the index to start the search from

Returns

The index of val within the String, or -1 if not found.

❖ length(): សម្រាប់វាស់ចំនួនក្នុងអក្សរក្នុងមួយ String

Syntax

```
String.length()
```

Parameter:

string: a variable of type String

Returns:

The length of the String in Characters.

❖ remove(): លុបអក្សរក្នុង String ពីត្រីម Index ណាមួយរហូតដល់ទីបញ្ចប់

Syntax

```
string.remove(index)
string.remove(index,count)
```

parameters:

index : a variable of type unsigned int

count : a variable of type unsigned int

❖ replace(): ជំនួសពាក្យណាមួយក្នុង String ដោយពាក្យថ្មី

Syntax

```
string.replace(substring1,substring2)
```

parameters

string: a variable of type String

substring1: another variable of type String

substring2: another variable of type String

❖ setCharAt(): ដាក់អក្សរមួយត្រឡប់ទៅក្នុង String នៅ index ណាមួយ ។

Syntax**string.setCharAt(index,c)**

parameters:

string: a variable of type String

index: the index to set the character at

c: the character to store to the give location

❖ **startsWith():**តើស្ត្រីមើលថា តើក្នុង String ចាប់ផ្តើមដោយពាក្យណាមួយឬទេ**Syntax****string.startsWith(string2)**

Parameters:

string, string2: variable2 of type String

Returns:

true: if string starts with the characters of string2

false: otherwise

❖ **substring():**កាត់ពាក្យពីត្រីម index ណាមួយទៅកាន់ index មួយទៀត ។**Syntax****string.substring(from)
string.substring(from,to)**

Parameters:

string: a variable of type String

from: the index to start the substring at

to (optional): the index to end the substring before

Returns:

The substring

❖ **toInt():**បំប្លែងពីពាក្យឱ្យទៅជាលេខ Int**Syntax****string.toInt()**

Parameters:

string: a variable of type String

Returns:

-long

-If no valid conversion could be performed because the string doesn't start with an integer number, a zero is returned.

❖ toFloat(): បំលែងពីពាក្យ ឱ្យទៅជាលេខ Float

Syntax

`string.toFloat()`

Parameters:

string: a variable of type String

Returns:

-Float

-If no valid conversion could be performed because the string doesn't start with a integer number, a zero is returned.

❖ toLowerCase(): បំលែងពីពាក្យដែលផ្តល់ដោយអក្សរធំទៅជាអក្សរតូចទាំងអស់

Syntax

`string.toLowerCase()`

Parameters:

string: a variable of type String

❖ toUpperCase(): បំលែងពីពាក្យដែលផ្តល់ដោយអក្សរតូចទៅជាអក្សរធំទាំងអស់ ។

Syntax

`string.toUpperCase()`

Parameters:

string: a variable of type String

❖ trim(): កាត់ចេញនូវ Whitespace ពីមុខនិង ក្រោយពាក្យរបស់យើង

Syntax

`string.trim()`

Parameters:

string: a variable of type String

លំហាត់ ៖ ចូរសរសេរកូដដែល Serial Monitor អាចឱ្យអ្នកបញ្ជូនលេខតាមវ៉ាលីដបង្ហាញនៅ
លើ Max7219 របស់ Arduino

ជំពូកទី៥ | ការបង្ហាញរូបភាពនៅលើអេក្រង់

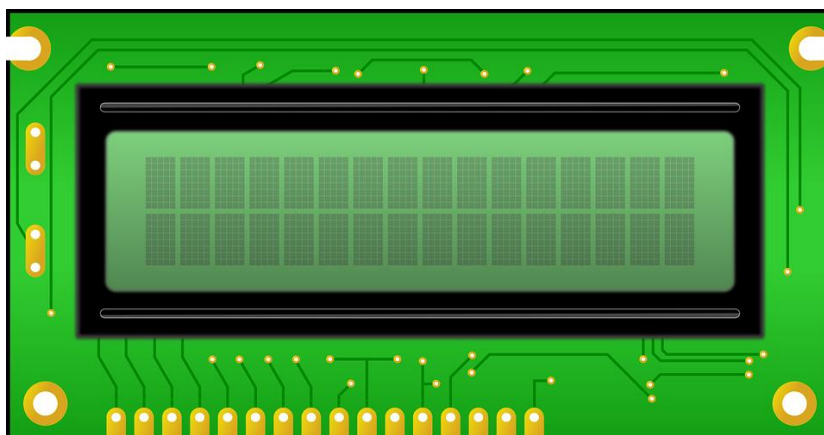
LCD & OLED

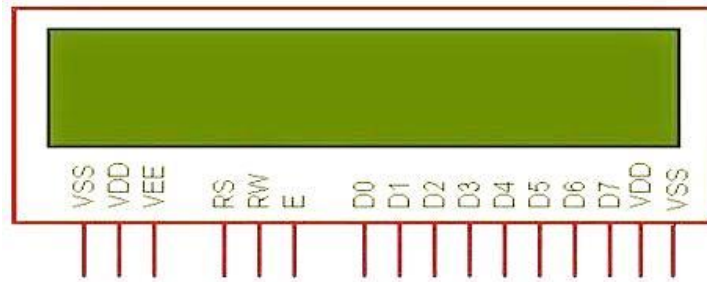
ក្នុងជំពូកនេះអ្នកនឹងបានសិក្សាអំពីការប្រើឧបករណ៍មួយចំនួនទៀតទាក់ទងនឹងការបង្ហាញ ចេញទិន្នន័យឱ្យយើងមើល ជំនួស 7 segment និងចាប់ផ្តើមប្រើប្រាស់ Library ផ្សេងៗរបស់អ្នក Develop ដ៏ទៃទៀត ។

I.Liquid Crystal Display (LCD)

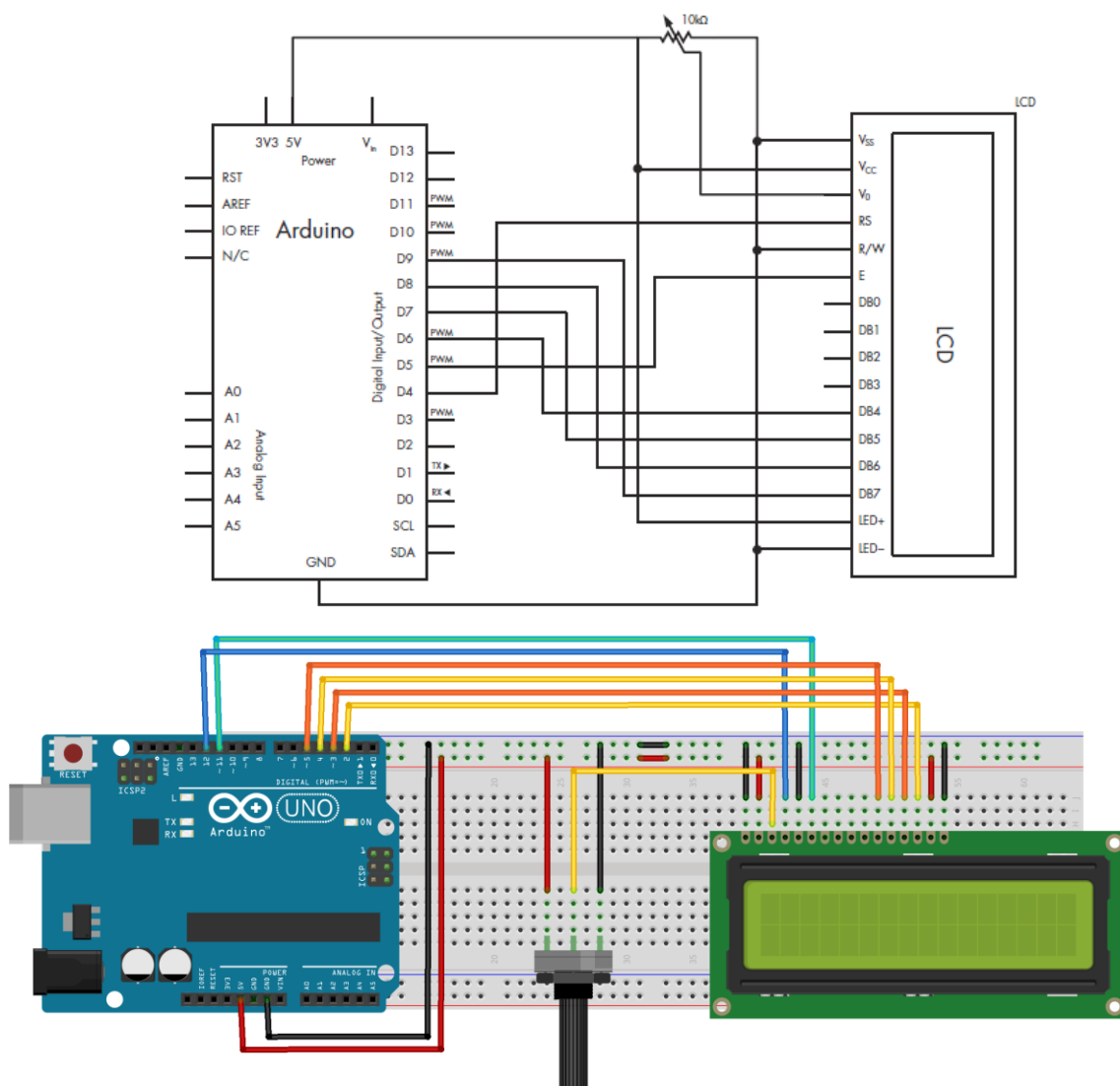
LCD ជាប្រភេទនៃឧបករណ៍អេឡិចត្រូនិចដែលជាទូទៅត្រូវបានគេប្រើសម្រាប់ការ បង្ហាញព័ត៌មានណាមួយពី Microcontroller ហើយវាមានភាពច្បាស់លាស់ និងលំអិតជាងអង្កត់ លេខព្រោះអាចបង្ហាញជាតួអក្សរឡាតាំងស្ទើរគ្រប់អក្សរ និងសញ្ញាផ្សេងៗ ។

ជាដំបូងអ្នកនឹងសិក្សាអំពី LCD 16x2 ដែលមានន័យថាវាជាប្រភេទ LCD ដែលអាចបញ្ចេញ អក្សរបាន 2 ជួរ និងមួយជួរៗមាន 16 អក្សរអតិបរមា ។ ជាធម្មតា LCD 16x2 មាន 16 ជើងដែលវា នីមួយៗមានមុខងារផ្សេងៗ





ដើម្បីអាចប្រើបានយើងត្រូវដាក់បញ្ចូល Library របស់ LCD ទៅក្នុង Sketch របស់យើងតាមរយៈ Sketch>>Include Library>>LiquidCrystal ហើយយើងនឹងឃើញវាចេញនៅក្នុង Sketch របស់យើង៖



1.Function of Syntax និងឧទាហរណ៍

Function របស់ Liquid Crystal រួមមាន៖

- LiquidCrystal()
- begin()
- clear()
- home()
- setCursor()
- write()
- print()
- cursor()
- noCursor()
- blink()
- noBlink()
- display()
- noDisplay
- scrollDisplayLeft()
- scrollDisplayRight()
- autoscroll()
- noAutoscroll()
- leftToRight()
- rightToLeft()
- creatChar()
- ❖ LiquidCrystal(): ប្រើសម្រាប់ប្រកាសបើកដើម Arduino ឱ្យ LCD

Syntax:

LiquidCrystal(rs,enable,d4,d5,d6,d7)

LiquidCrystal(rs,rw,enable,d4,d5,d6,d7)

LiquidCrystal(rs,rw,enable,d0,d1,d2,d3,d4,d5,d6,d7)

ឧទាហរណ៍

```
#include<LiquidCrystal.h>
```

```
LiquidCrystal lcd(12,11,10,5,4,3,2);
```

```
void setup(){
```

```
    lcd.begin(16,1);
```

```
    lcd.print("hello");
```

```
}
```

```
void loop(){ }
```

- ❖ begin(): សម្រាប់ឱ្យ Library ចាប់ផ្តើមដំណើរការ ។

lcd: a variable of type LiquidCrystal

cols: the number of columns that the display has

rows: the number of rows that the display has

- ❖ clear(): លុបអក្សរទាំងអស់នៅលើ Screen
- ❖ home(): ទាញ Cursor ទៅកាន់ Colum=0,Row=0

- ❖ `setCursor()`: ដាក់ Cursor ទៅកាន់ Colum, Row ណាមួយ

Syntax:

```
lcd.setCursor(col,row)
```

- ❖ `write()`: បញ្ជូនអក្សរមួយតួទៅកាន់ LCD

Syntax:

```
lcd.write(data)
```

Example:

```
#include<LiquidCrystal.h>
LiquidCrystal lcd(12,11,10,5,4,3,2);
void setup(){
    lcd.begin(16,1)
    lcd.write("A");
}
void loop(){}
❖ print():បញ្ជូនពាក្យដែលមានច្រើនតួអក្សរទៅ LCD
```

Syntax:

```
lcd.print(data)
```

```
lcd.print(data,BASE)
```

Example:

```
lcd.print("hello")
```

```
lcd.print(73,HEX)
```

- ❖ `cursor()`:បង្ហាញ Cursor នៅលើ Screen
- ❖ `noCursor()`:មិនបង្ហាញ Cursor នៅលើ Screen
- ❖ `blink()`: ឱ្យ Cursor លោតផ្លិបផ្លែត
- ❖ `noBlink()`: បិទមិនឱ្យ Cursor លោតផ្លិបផ្លែត
- ❖ `display()`: ឱ្យ LCD បង្ហាញអក្សរ
- ❖ `noDisplay()`:លាក់អក្សរទាំងអស់ដែលមានលើ LCD តែមិនមានន័យលុបទាំងស្រុងទេគឺអាច បង្ហាញចេញមកវិញតាមរយៈ `display()`
- ❖ `scrollDisplayLeft()`:រុញអក្សរដែលមានលើ Screen ទាំងអស់ទៅឆ្វេងមួយតួអក្សរ ។
- ❖ `scrollDisplayRight()`:រុញអក្សរដែលមានលើ Screen ទាំងអស់ទៅស្តាំមួយតួអក្សរ
- ❖ `autoscroll()`:នៅពេលដែលអក្សរសរសេរលើសចំនួន Colum របស់វាដោះវានឹងមិនរំកិលអក្សរមួយទៅឆ្វេងឬស្តាំ

- ❖ noAutoscroll():នៅពេលដែលអក្សរសរសេរលើសចំនួន Culom របស់វានោះវានឹងមិនរំកិលអក្សរមួយទៅឆ្វេងឬស្តាំ
- ❖ leftToRight():កំណត់យកទិសដៅអក្សរចេញពីឆ្វេងទៅស្តាំ
- ❖ rightToLeft():កំណត់យកទិសដៅអក្សរចេញពីស្តាំទៅឆ្វេង
- ❖ creatChar():សម្រាប់បង្កើតនិមិត្តសញ្ញា ឬតួអក្សរថ្មីណាមួយដែលមិនមែនជាអក្សរឡាតាំង

Syntax:

```
lcd.creatChar(num,data)
```

Parameters:

lcd: a variable of type LiquidCrystal

num: which character to creat (0 to 7)

data: the character's pixel data

ចំណែក៖ mylcd.begin(16,2);មានន័យថា 16x2

```
#include <LiquidCrystal.h>
```

```
LiquidCrystal mylcd(7,6,2,3,4,5); //RS,E,DB4,DB5,DB6,D7
```

```
void setup() {
```

```
    mylcd.begin(16,2);
```

```
    mylcd.print("Chanthorn");
```

```
}
```

```
void loop() {
```

```
}
```

នៅពេលដែលយើងចង់ដាក់ជួរទីពីរគឺ mylcd.setCursor(5,1); mylcd.print("HOW Are YOU");

```
#include <LiquidCrystal.h>
```

```
LiquidCrystal mylcd(7,6,2,3,4,5); //RS,E,DB4,DB5,DB6,D7
```

```
void setup() {
```

```
    mylcd.begin(16,2);
```

```
    mylcd.print("HELLO EVERYONE");
```

```
    mylcd.setCursor(5,1);
```

```
    mylcd.print("HOW Are YOU");
```

```
}
```

```
void loop() {
```

```
}
```

ដើម្បីឱ្យវាវត់គេអាចប្រើ mylcd.scrollDisplayLeft();

```
    delay(500);
```


ចង់មាន blink អាចដាក់ mylcd.blink();

ចង់បានលោតភ្លើងត្រួតដាក់ mylcd.cursor();

របៀបដាក់ឱ្យលោតពាក្យ Hello Cambodia

```
#include <LiquidCrystal.h>
LiquidCrystal mylcd(7,6,2,3,4,5);//RS,E,DB4,DB5,DB6,D7
char myletter[]={"Hello Cambodia"};
void setup() {
  mylcd.begin(16,2);
  mylcd.setCursor(0,1);
  mylcd.blink();
  mylcd.cursor();
}
void loop() {
  for (int i =0;i<=13;i++){
    mylcd.print(myletter[i]);
    delay(500);
  }
  mylcd.clear();
}
```

ចង់វាយពាក្យហើយឱ្យវាចេញដោយមិនបាច់ប្តូរកូដ

```
#include <LiquidCrystal.h>
LiquidCrystal mylcd(7,6,2,3,4,5);//RS,E,DB4,DB5,DB6,D7
void setup() {
  mylcd.begin(16,2);
  Serial.begin(9600);
}
void loop() {
  if (Serial.available()>0){
    String x= Serial.readString();
    mylcd.print(x);
  }
}
```

កូដនេះវាហើយវាយនៅតែរត់ត្រួត ហេតុដូច្នេះហើយយើងត្រូវប្រើ mylcd.clear();

```
#include <LiquidCrystal.h>
LiquidCrystal mylcd(7,6,2,3,4,5);//RS,E,DB4,DB5,DB6,D7
void setup() {
  mylcd.begin(16,2);
  Serial.begin(115200);
}
```

```

}
void loop() {
  if (Serial.available()>0){
    String x= Serial.readString();
    mylcd.clear();
    mylcd.print(x);
  }
}

```

របៀបទី២

```

ដោយប្រើ if (x=="clear"){ mylcd.clear();

#include <LiquidCrystal.h>
LiquidCrystal mylcd(7,6,2,3,4,5); //RS,E,DB4,DB5,DB6,D7
void setup() {
  mylcd.begin(16,2);
  Serial.begin(9600);
}
void loop() {
  if (Serial.available()>0){
    String x = Serial.readString();
    mylcd.print(x);
    Serial.print(x);
    if (x=="clear"){
      mylcd.clear();
    }
  }
}

```

របៀបបង្កើតអក្សរខ្មែរ

ឬ ឬ link ដើម្បីប្តូរកូដ Binary គឺ <https://omerk.github.io/lcdchargen/>

```

#include <LiquidCrystal.h>

LiquidCrystal mylcd(7,6,2,3,4,5); //RS,E,DB4,DB5,DB6,D7

byte display1[]={
  B11111, B00000, B01110, B10001, B10001, B10001, B11001, B10001};

byte display2[]={ B01110, B10001, B00001, B01111, B10000, B10001, B10001,
B01110
};

void setup() {

```

```

mylcd.createChar(0,display1);
mylcd.createChar(1,display2);
mylcd.begin(16,2);
mylcd.write(byte(0));
mylcd.write(byte(1));
}
void loop() {
}

```

លំហាត់

1. ចូរអ្នកសរសេរកូដមួយដែលអ្នកបញ្ជូនពាក្យពី Serial ហើយបង្ហាញនៅលើ LCD ។
2. ចូរអ្នកសរសេរកូដបង្កើតជាតារាងការបង្ហាញនៅលើ LCD ។
3. ចូរអ្នកសរសេរកូដមួយដែលមានប៊ូតុងចុច Up/Down និង Shift Left/Right និងប៊ូតុង Start/rest ។
 Project នេះគឺអ្នកធ្វើប្រព័ន្ធមួយដែលមាន Timer ហើយយើងធ្វើការបញ្ចូលតម្លៃវាតាមរយៈការ Set UP/DOWN រួចបង្ហាញលេខលើ LCD ។ ដើម្បីអាចបញ្ចូលបានល្បឿនយើងអាចប្រើ Shift Left/Right ។ នៅពេលចុចប៊ូតុង Start វាបាច់ផ្ដើមរាប់លេខចុះរហូតដល់សូន្យ នៅពេលដល់សូន្យវាបញ្ចូលទៅកាន់ Relay ដើម្បីភ្ជាប់អ្វីមួយ ។

2. ការដាក់មេរៀន digital

```

#include <LiquidCrystal.h>
LiquidCrystal mylcd(7,6,2,3,4,5); //RS,E,DB4,DB5,DB6,D7
void setup(){
  mylcd.begin(16,2);
}
void loop() {
  for (int h=0;h<=23;h++){
    for (int m=0;m<=59;m++){
      for (int s=0;s<=59;s++){
        mylcd.print(h);
        mylcd.print(":");
        mylcd.print(m);
        mylcd.print(":");
        mylcd.print(s);
        delay(1000);
        mylcd.clear();
      }
    }
  }
}

```

ដើម្បីឱ្យចេញលេខពីខ្ទង់ទៅបីជាមុន ០០ ក៏ដោយគេត្រូវដាក់ print នៅពីមុខ time នោះ

```
#include <LiquidCrystal.h>
LiquidCrystal mylcd(7,6,2,3,4,5); //RS,E,DB4,DB5,DB6,D7
void setup(){
    mylcd.begin(16,2);
}
void loop() {
    for (int h=0;h<=23;h++){
        for (int m=0;m<=59;m++){
            for (int s=0;s<=59;s++){
                if (h<=9){
                    mylcd.print(0);
                }
                mylcd.print(h);
                mylcd.print(":");
                if (m<=9){
                    mylcd.print(0);
                }
                mylcd.print(m);
                mylcd.print(":");
                if (s<=9){
                    mylcd.print(0);
                }
                mylcd.print(s);
                delay(1000);
                mylcd.clear();
            }
        }
    }
}
```

របៀបដាក់ time ឱ្យលោតពីចំណុចណាទៅ គេត្រូវតាំង int k=7;

int l=9; ហើយដាក់ if(h>23){k=0;}និង if(m>59){l=0;}។

```
#include <LiquidCrystal.h>
LiquidCrystal mylcd(7,6,2,3,4,5); //RS,E,DB4,DB5,DB6,D7
int k=7;
int l=9;
void setup(){
    mylcd.begin(16,2);
}
void loop() {
```

```

for (int h=7;h<=23;h++){
  if(h>23){k=0;}
  for (int m=1;m<=59;m++){
    if(m>59){l=0;}
    for (int s=0;s<=59;s++){
      if (h<=9){
        mylcd.print(0);
      }
      mylcd.print(h);
      mylcd.print(":");
      if (m<=9){
        mylcd.print(0);
      }
      mylcd.print(m);
      mylcd.print(":");
      if (s<=9){
        mylcd.print(0);
      }
      mylcd.print(s);
      delay(1000);
      mylcd.clear();
    }
  }
}
}
}

```

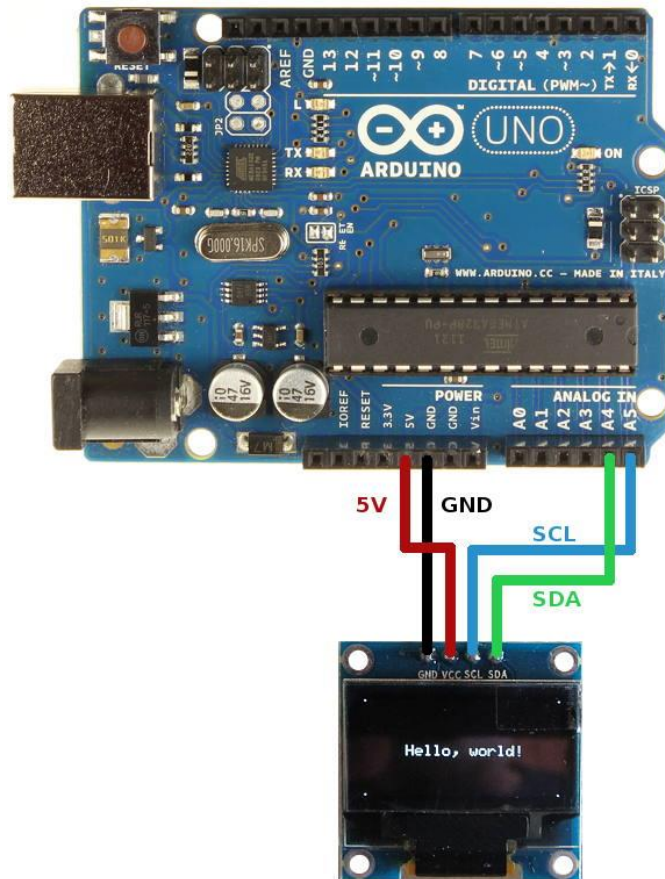
II. Graphic LED OLED

1. ការប្រើប្រាស់

LED OLED ជាប្រភេទ LCD ខ្នាតតូច ល្អសម្រាប់ទីតាំងដែលយើងត្រូវការតូច ។ ម៉្យាងវិញ ទៀត វាអាចបញ្ចេញរូបភាពបានល្អជាង LiquidCrystal ។



គេត្រូវតែស្វែងរកមានទម្រង់ដូចខាងក្រោមទើបអាចដំណើរការបាន



ដើម្បីប្រើប្រាស់វាបានគេត្រូវ Download library របស់ Adafruit តាមរយៈគេហទំព័រ

https://github.com/adafruit/adafruit_SSD1306 ឬគ្រាន់តែចូលទៅក្នុង Skect>include

library>manage libraries>filter you search (វាយ adafruit_SSD1306 រឺធីនេះល្អជាងព្រោះវានឹងស្គាល់ក្នុង library ដោយស្វ័យប្រវត្តិ)



❖ Adafruit_SSD1306():សម្រាប់ប្រកាសអញ្ជាតរបស់ library នេះ

```
Adafruit_SSD1306 OLED(RS_pin)
```

RS_pin: reset pin

❖ Begin():សម្រាប់ផ្ដើមដំណើរការឱ្យ OLED នៅ address របស់វា

```
OLED.begin(address)
```

Address: address រំលស់ OLED ក្នុង Bus 12C

Logo Adafruit

Syntax:

OLED.display()

- ❖ `clearDisplay()`:សម្រាប់បញ្ជាទៅឱ្យ OLED សំអាតអក្សរ ឬរូបចេញ

Syntax:

OLED.clearDisplay()

❖ drawPixel(): សម្រាប់គូសចំណុចមួយ Pixel នៅត្រង់ទីតាំង x,y

Syntax:

OLED.drawPixel(x,y,color)

Parameter

x : x ទីតាំងជួរដេក

y : y ទីតាំងជួរឈរ

color: WHITE/BLACK

❖ drawCircle(): សម្រាប់គូសរង្វង់មានផ្ចិតនៅត្រង់ទីតាំង x,y មានកាំ r

Syntax:

OLED.drawCircle(x,y,r,color)

Parameter

x: x ទីតាំងជួរដេក

y : y ទីតាំងជួរឈរ

r : កាំរង្វង់

color: WHITE/BLACK

❖ drawRect(): សម្រាប់គូសចតុកោណកែង នៅត្រង់ទីតាំង x,y មានបណ្តោយ w និង ទទឹង h

Syntax

OLED.drawRect(x,y,w,h,color)

Parameter

x: x ទីតាំងជួរដេក

y : y ទីតាំងជួរឈរ

w: បណ្តោយ

h: ទទឹង

color: WHITE/BLACK

❖ fillRect(): សម្រាប់គូសចតុកោណកែងពេញសាច់ នៅត្រង់ទីតាំង x,y មានបណ្តោយ និង ទទឹង h

Syntax():

`OLED.fillRect(x,y,w,h,color)`

Parameter

x: x ទីតាំងជួរដេក

y : y ទីតាំងជួរឈរ

w: បណ្តោយ

h: ទទឹង

color: WHITE/BLACK

- ❖ `drawRoundRect()`: សម្រាប់គូសចតុកោណកែងមានលុបជ្រុងកោងនៅត្រង់ទីតាំង x,y មានបណ្តោយ និងទទឹង h

Syntax:

`OLED.drawRoundRect(x,y,w,h,r,color)`

Parameter

x: x ទីតាំងជួរដេក

y : y ទីតាំងជួរឈរ

w: បណ្តោយ

h: ទទឹង

r: radius fillet

color: WHITE/BLACK

- ❖ `fillRoundRect()`:សម្រាប់គូសចតុកោណកែងពេញសាច់មានលុបជ្រុងកោងនៅត្រង់ទីតាំង x,y មានបណ្តោយ w និងទទឹង h

Syntax:

`OLED.fillRoundRect(x,y,w,h,r,color)`

Parameter

x: x ទីតាំងជួរដេក

y : y ទីតាំងជួរឈរ

w: បណ្តោយ

h: ទទឹង

r: radius fillet

color: WHITE/BLACK

- ❖ `drawTriangle()`:សម្រាប់គូសត្រីកោណចេញពីបីចំណុច

Syntax

`OLED.drawTriangle(x1,y1,x2,y2,x3,y3,color)`

❖ drawLine(): សម្រាប់គូសបន្ទាត់

Syntax

```
OLED.drawLine(x1,y1,x2,y2,color)
```

❖ setTextSize():សម្រាប់កំណត់ទំហំអក្សរ

Syntax

```
OLED.setTextSize(size)
```

Parameter

Size: size of text(int)

❖ setTextColor(): សម្រាប់កំណត់ពណ៌អក្សរ

Syntax

```
OLED.setTextColor(color)
```

Parameter

color:WHITE/BLACK

3.ការគូសរូបតិច

គ្រាន់តែ បង្ហាញ adafruit industries នៅលើ LED oled

```
#include <Adafruit_SSD1306.h>
```

```
Adafruit_SSD1306 chanthornoled(13);
```

```
void setup() {
```

```
    chanthornoled.begin(0x3c);
```

```
    chanthornoled.display();
```

```
}
```

```
void loop() {
```

```
}
```

ដើម្បីគូស pixel

```
chanthornoled.drawPixel(32,25,WHITE);
```

គេត្រូវ clearDisplay(); ដើម្បីបង្ហាញរូបនេះ

```
#include <Adafruit_SSD1306.h>
```

```
Adafruit_SSD1306 chanthornoled(13);
```

```
void setup() {
```

```
    chanthornoled.begin(0x3c);
```

```
    chanthornoled.clearDisplay();
```

```
    chanthornoled.drawPixel(32,25,WHITE);
```

```
    chanthornoled.display();
```

```
}
```

```
void loop() {
}
```

គួររង្វង់

chathornoled.drawCircle(64,32,10,WHITE); យើងបានកូដខាងក្រោម

```
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 chathornoled(13);
void setup() {
  chathornoled.begin(0x3c);
  chathornoled.clearDisplay();
  chathornoled.drawPixel(32,25,WHITE);
  chathornoled.drawCircle(64,32,10,WHITE);
  chathornoled.display();
}
void loop() {
}
```

អ្នកអាចសាកល្បងកូដដែលមានទម្រង់ផ្សេងៗដូចខាងក្រោម

របៀបដាក់រូបក្នុង LED OLED គេត្រូវចូលទៅក្នុងវេបសាយ <http://javl.github.io/image2cpp/> ដើម្បី

ប្តូរកូដទៅជា Binary ។ ខាងក្រោមនេះជាកូដធ្វើឱ្យមានរូបកូនសោ

```
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 chathornoled(13);
const unsigned char PROGMEM Mykey[]={
  // 'orange-transistor-512', 128x64px
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x00, 0x00, 0x1f, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x00, 0x00, 0x0f, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xe0, 0x00, 0x00, 0x07, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xc0, 0x00, 0x00, 0x03, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x80, 0x00, 0x00, 0x01, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0x03, 0xff, 0xff, 0xc0, 0xff, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xfe, 0x07, 0xff, 0xff, 0xe0, 0x7f, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x0f, 0xff, 0xff, 0xf0, 0x3f, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x1f, 0xf8, 0x1f, 0xf8, 0x1f, 0xff, 0xff,
  0xff, 0xff, 0xff,
  0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x3f, 0xf0, 0x0f, 0xfc, 0x0f, 0xff, 0xff,
  0xff, 0xff, 0xff,
}
```

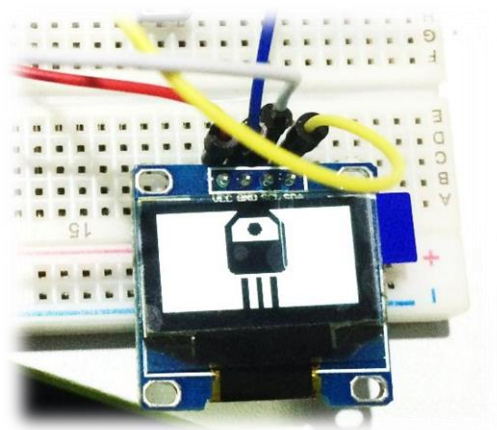
www.devfortech.org

www.devfortech.org

```

0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x7c, 0x3e, 0x0f, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x7c, 0x3e, 0x0f, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x7c, 0x3e, 0x0f, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x7c, 0x3e, 0x0f, 0xff, 0xff, 0xff,
0xff, 0xff, 0xff,
};
void setup() {
  chathornoled.begin(0x3c);
  chathornoled.clearDisplay();
  chathornoled.drawBitmap(0,0,Mykey, 128,64,WHITE);
  chathornoled.display();
}
void loop() {
}

```



កូដដើម្បីសរសេរពាក្យ

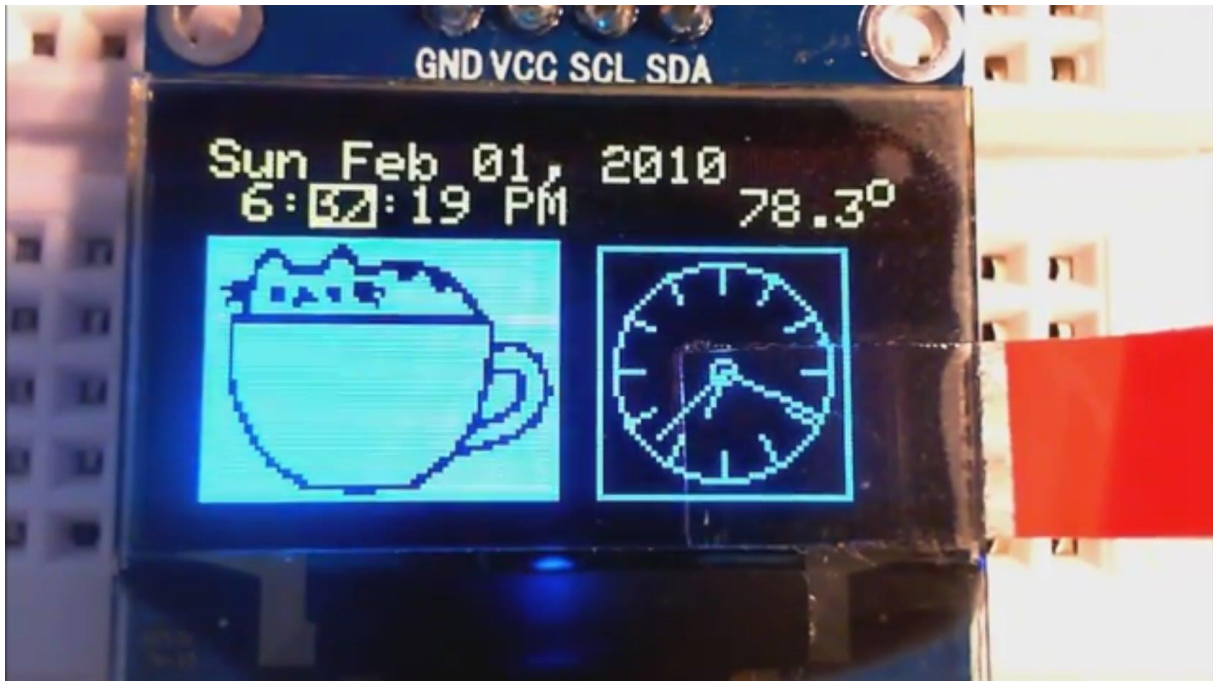
```

#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 chathornoled(13);
void setup() {
  chathornoled.begin(0x3c);
  chathornoled.clearDisplay();
  chathornoled.setTextSize(2);
  chathornoled.setTextColor(WHITE);
  chathornoled.setCursor(20,30);
  chathornoled.print("My time");
  chathornoled.display();
}
void loop() {
}

```

}

ការគូររូបនាឡិកា clock



```
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 oled(LED_BUILTIN);
int xc= 63;
int yc=31;
int r=30;
int r1=25;
void setup() {
  oled.begin(0x3c);
  oled.clearDisplay();
  oled.display();
  initial();
}
void loop() {}
void initial(){
  oled.drawCircle(xc,yc,r,WHITE);
  for(int deg=0;deg<360;deg+=30){
    int x=xc+r*cos(deg*PI/180);
    int y=yc-r*sin(deg*PI/180);
    int x1=xc+r1*cos(deg*PI/180);
    int y1=yc-r1*sin(deg*PI/180);
    oled.drawLine(x,y,x1,y1,WHITE);
  }
  oled.display();
}
```

}

ក្នុងខាងលើ គ្រាន់តែជារង្វង់និងទ្រនិចទេ យើងត្រូវតែសម្រនិចតូចៗបន្ថែម ក្នុងខាងក្រោមជាទ្រនិច

```
#include <Adafruit_SSD1306.h>
```

```
Adafruit_SSD1306 oled(LED_BUILTIN);
```

```
int xc= 63;
```

```
int yc=31;
```

```
int r=30;
```

```
int r1=25;
```

```
float rs=23;
```

```
float rm=20;
```

```
float rh=15;
```

```
void setup() {
```

```
    oled.begin(0x3c);
```

```
    oled.clearDisplay();
```

```
    oled.display();
```

```
    initial();
```

}

```
void loop() {}
```

```
void initial(){
```

```
    oled.drawCircle(xc,yc,r,WHITE);
```

```
    for(int deg=0;deg<360;deg+=30){
```

```
        int x=xc+r*cos(deg*PI/180);
```

```
        int y=yc-r*sin(deg*PI/180);
```

```
        int x1=xc+r1*cos(deg*PI/180);
```

```
        int y1=yc-r1*sin(deg*PI/180);
```

```
        oled.drawLine(x,y,x1,y1,WHITE);
```

```
    }
```

```
    oled.display();
```

```
    drawtick(rs,0,1);
```

```
    drawtick(rm,0,1);
```

```
    drawtick(rh,0,1);
```

}

```
void drawtick(float R,float ang,int color){
```

```
    int X=xc+R*cos(-ang*PI/180+(PI/2));
```

```
    int Y=yc-R*sin(-ang*PI/180+(PI/2));
```

```
    oled.drawLine(xc,yc,X,Y,color);
```

```
    oled.display();
```

}

ដាក់ទ្រនិចទាំងបីឱ្យដើរ

```
#include <Adafruit_SSD1306.h>
```



```

Adafruit_SSD1306 oled(LED_BUILTIN);
int xc= 63;
int yc=31;
int r=30;
int r1=25;
float rs=23;
float rm=20;
float rh=15;
void setup() {
  oled.begin(0x3c);
  oled.clearDisplay();
  oled.display();
  initial();
}
void loop() {
  for(long i=0;i<43200;i++){
    drawtick(rs,(i-1)*6,0);
    drawtick(rm,(i-1)*6,0);
    drawtick(rh,(i-1)*6,0);

    drawtick(rs,i*6,1);
    drawtick(rm,i*6/60,1);
    drawtick(rh,i*6/3600,1);
    delay(1000);
  }
}
void initial(){
  oled.drawCircle(xc,yc,r,WHITE);
  for(int deg=0;deg<360;deg+=30){
    int x=xc+r*cos(deg*PI/180);
    int y=yc-r*sin(deg*PI/180);
    int x1=xc+r1*cos(deg*PI/180);
    int y1=yc-r1*sin(deg*PI/180);
    oled.drawLine(x,y,x1,y1,WHITE);

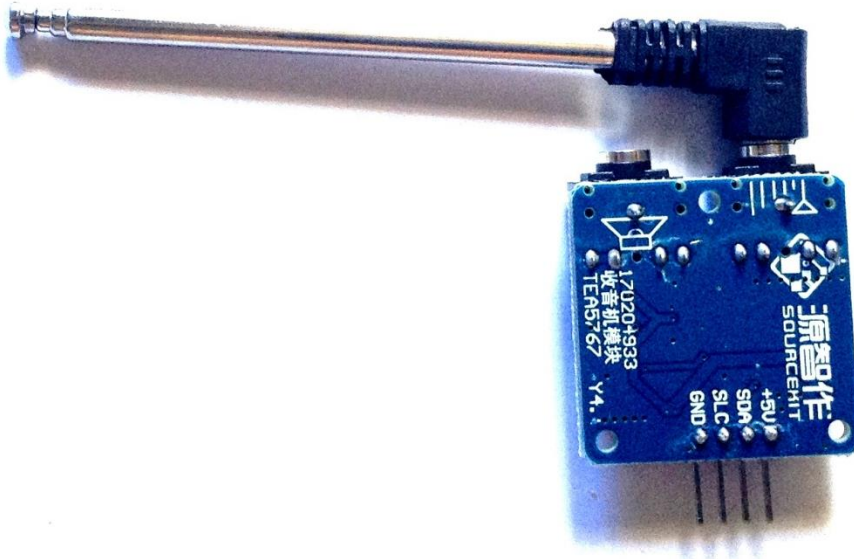
  }
  oled.display();
  drawtick(rs,0,1);
  drawtick(rm,0,1);
  drawtick(rh,0,1);
}

```

```
void drawtick(float R,float ang,int color){
    int X=xc+R*cos(-ang*PI/180+(PI/2));
    int Y=yc-R*sin(-ang*PI/180+(PI/2));
    oled.drawLine(xc,yc,X,Y,color);
    oled.display();
}
```

4.ការបង្កើត វីដេអូ

ដើម្បីបង្កើតវីដេអូជាមួយ Arduino គេត្រូវការ module មួយគឺ TEA5767Radio និង Library របស់វា



ខាងក្រោមនេះជាកូដរបស់វាដែលចាប់យកប៉ូស្តវីទីឡូ 93

```
#include<Wire.h>
#include <TEA5767Radio.h>
TEA5767Radio radio=TEA5767Radio();
void setup(){
    Wire.begin();
    radio.setFrequency(93.0);//pick your own frequency
}
void loop(){ }
```

កូដ OLED time

```
#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 chanthornoled(13);
void setup() {
    chanthornoled.begin(0x3c);
    chanthornoled.clearDisplay();
    chanthornoled.setTextSize(2);
    chanthornoled.setTextColor(WHITE);
```

```

    chanthornoled.setCursor(30 ,10);
}
void loop() {
    for (int h=0;h<=23;h++){
        for (int m=0;m<=59;m++){
            for (int s=0;s<=59;s++){
                chanthornoled.setCursor(30,32);
                chanthornoled.print(h);
                chanthornoled.print(":");
                chanthornoled.print(m);
                chanthornoled.print(":");
                chanthornoled.print(s);
                chanthornoled.display();
                delay(1000);
                chanthornoled.clearDisplay();
            }
        }
    }
}

```

5.Real Time Clock

<http://www.rinkydinkelectronics.com/library.php?id=73>

```

#include <DS1307.h>
DS1307 chanthornrtc(SDA,SCL);
void setup(){
    Serial.begin(115200);
    chanthornrtc.begin();
}
void loop() {
    Serial.println(chanthornrtc.getDOWStr());
    Serial.println(chanthornrtc.getDateStr());
    Serial.println(chanthornrtc.getTimeStr());
    delay(1000);
}

```

កូដខាងលើនេះត្រូវបន្ថែម

```

#include <Adafruit_SSD1306.h>
#include <DS1307.h>
DS1307 rtc(SDA,SCL);
Adafruit_SSD1306 oled(13);
void setup() {
    Serial.begin(115200);

```



```

    rtc.begin();
    oled.begin(0x3c);
    oled.clearDisplay();
    oled.setTextSize(2);
    oled.setTextColor(WHITE);
    // seyhartc.setDOW(WEDNESDAY);
    // seyhartc.setDate(12,3,2018);
    // seyhartc.setTime(7,19,0);
}
void loop() {
    oled.setCursor(30,32);
    oled.println(rtc.getDOWStr());
    oled.println(rtc.getDateStr());
    oled.println(rtc.getTimeStr());
    oled.display();
    delay(1000);
    oled.clearDisplay();
}

```

កូដសម្រាប់ដាក់ម៉ោងរ៉ោ

```

Time t= rtc.getTime();
int H= t.hour;
int M= t.min;
if (H==6&& M==19|M==20){
    digitalWrite(2,HIGH);
    delay(1000);
    digitalWrite(2,LOW);
}
}

```

កូដទាំងអស់

```

#include <Adafruit_SSD1306.h>
Adafruit_SSD1306 oled(13);
#include<DS1307.h>
DS1307 rtc(SDA,SCL);
void setup() {
    oled.begin(0x3c);
    oled.clearDisplay();
    oled.setTextSize(2);
    oled.setTextColor(WHITE);
    // Serial.begin(115200);
    rtc.begin();
}

```

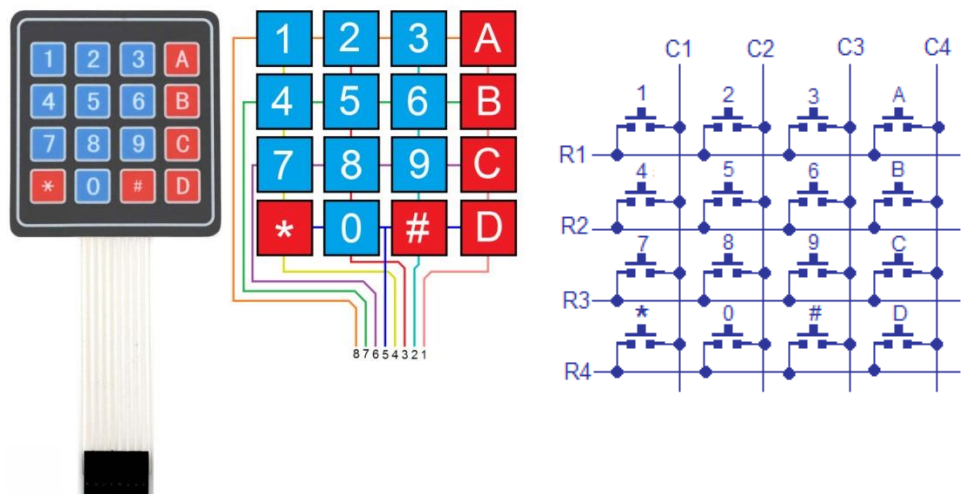
```
// rtc.setDOW(FRIDAY);
// rtc.setDate(16,3,2018);
// rtc.setTime(6,05,0);
pinMode(2,HIGH);
}
void loop() {
  oled.setCursor(0,0);
  oled.print(rtc.getDOWStr());
  oled.setCursor(0,20);
  oled.print(rtc.getDateStr());
  oled.setCursor(0,40);
  oled.print(rtc.getTimeStr());
  // Serial.println(rtc.getDOWStr());;
  // Serial.println(rtc.getDateStr());
  // Serial.println(rtc.getTimeStr());
  oled.display();
  delay(1000);
  oled.clearDisplay();
  Time t = rtc.getTime();
  int H = t.hour;
  int M = t.min;
  if(H==6 && M== 22|| M==23){
    digitalWrite(2,1);
    delay(200);
    digitalWrite(2,0);
    delay(200);
  }
}
```

កូដមួយទៀត

```
#include <Adafruit_SSD1306.h>
#include <DS1307.h>
DS1307 rtc(SDA,SCL);
Adafruit_SSD1306 oled(13);
void setup() {
  Serial.begin(115200);
  rtc.begin();
  oled.begin(0x3c);
  oled.clearDisplay();
  oled.setTextSize(2);
  oled.setTextColor(WHITE);
  // seyhartc.setDOW(WEDNESDAY);
```

```
// seyhartc.setDate(12,3,2018);
// seyhartc.setTime(7,19,0);
pinMode(2,OUTPUT);
}
void loop() {
  oled.setCursor(30,32);
  oled.println(rtc.getDOWStr());
  oled.println(rtc.getDateStr());
  oled.println(rtc.getTimeStr());
  oled.display();
  delay(1000);
  oled.clearDisplay();
  Time t = rtc.getTime();
  int H= t.hour;
  int M = t.min;
  if (H==6 && M==10||M==11){
    digitalWrite(2,HIGH);
    delay(200);
    digitalWrite(2,LOW);
    delay(200);
  }
}
```

6.Keypad



កូដដូចលើ keypad ដើម្បីឱ្យ oled ដំណើរការ

```
#include <Adafruit_SSD1306.h>
#include <Key.h>
#include <Keypad.h>
Adafruit_SSD1306 oled(13);
const byte ROWS = 4; //four rows
```

```

const byte COLS = 4; //four columns
char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};
byte rowPins[ROWS] = {9,8,7,6};
byte colPins[COLS] = {5,4,3,2};
Keypad keypad = Keypad( makeKeymap(keys), rowPins, colPins, ROWS,
COLS );
void setup() {
  Serial.begin(9600);
  oled.begin(0x3C);
  oled.clearDisplay();
}
void loop() {
  char key = keypad.getKey();
  if (key) {
    oled.clearDisplay();
    Serial.println(key);
    oled.print(key);
    oled.display();
  }
}

```

យើងនឹងសរសេរកូដមួយដោយចុច password បើចុចត្រូវនោះអំពូលភ្លឺ បើចុចខុស Buzzer នឹងរ៉ោ

កូដសម្រាប់ set password

1.correct->LED ON

2.incorrect->Buzzer Alarm

*->clear password

#->OK

កូដរបស់វាគឺ

```

#include <Keypad.h>
#include <Key.h>
#include <Adafruit_SSD1306.h>
#include <DS1307.h>
Adafruit_SSD1306 oled(13);
DS1307 rtc(SDA,SCL);

```

```
const byte row = 4;
```

```

const byte col = 4;
char keys[row][col]={
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'},
};
byte rowPins[row]={9,8,7,6};
byte colPins[col]={5,4,3,2};
Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,row,col);
byte ledPin=2;
boolean blink=false,btn1=false;
int password=0;
String passwordText;
void setup() {
    Serial.begin(9600);
    rtc.begin();
    oled.setTextSize(3);
    oled.setTextColor(WHITE);
    oled.clearDisplay();
    pinMode(2,OUTPUT);
    pinMode(11,OUTPUT);
    oled.setCursor(0,0);
}

void loop() {
    char key = keypad.getKey();
    if(key){
        if(key!='#')
            passwordText += key;
        oled.begin(0x3c);
        oled.print(key);
        oled.display();
        Serial.println(passwordText);
    }
    if(passwordText=="2017"){
        password= 1;
    }else{
        password= 2;
    }
    if(key=='#'){

```



```

if(password==1) // Correct password
    digitalWrite(2,1);
else //Wrong password
    digitalWrite(2,0);
if(password==2)
    digitalWrite(11,1);
else
    digitalWrite(11,0);
}
if(key=='*'){
    passwordText = "";
    oled.clearDisplay();
    oled.display();
    oled.setCursor(0,0);
    digitalWrite(11,0);
    digitalWrite(2,0);
}
}

```

III.EEPROM

EEPROM ត្រូវបានប្រើសម្រាប់ការស្តុកទិន្នន័យរយៈពេលយូរដែលអាចបានរហូតដល់ទៅ 20 ឆ្នាំ ជាការពិតណាស់ អញ្ញាតនៅក្នុង Arduino មិនផ្ទុកទិន្នន័យបានទេ នៅពេលដែលវាដាច់ភ្លើងដូច្នេះ ហើយបានជាការដាក់ទិន្នន័យទៅក្នុង EEPROM ជាការចាំបាច់ដើម្បីរក្សាវា ទុកប្រើពេលមានភ្លើងវិញ ។ ក្នុងមួយAddress នៃEEPROM អាចដាក់បាន 8bits data ។

1.Function of Syntax និង ឧទាហរណ៍

- ❖ read() : ជា Function ប្រើសម្រាប់អានយកទិន្នន័យ 1 byte ពី Address ណាមួយរបស់ EEPROM

Syntax :

EEPROM.read(address)

- ❖ write() : ជា Function ប្រើប្រាស់សម្រាប់ដាក់បញ្ចូលទិន្នន័យ 1 byte ពី Address ណាមួយរបស់EEPROM

Syntax:

EEPROM.write(address,value)

- ❖ update() :ជា Function ប្រើសម្រាប់ដាក់បញ្ចូលទិន្នន័យ 1 byte ពី Address ណាមួយរបស់ EEPROM តែការដាក់នេះគឺធ្វើការដាក់បានលុះត្រាណាដែលទិន្នន័យដែលដាក់ខុសពីទិន្នន័យដែលមានស្រាប់នៅក្នុង Address នេះ ។

Syntax:

EEPROM.update(address,value)

- ❖ `get()` : សម្រាប់ទាញយកទិន្នន័យជាប្រភេទណាមួយក៏បានរួមបញ្ចូលទាំងជាប្រភេទ `Float`

Syntax:

```
EEPROM.get(address)
```

- ❖ `put()` : សម្រាប់ប្រើទាញយកទិន្នន័យជាប្រភេទណាមួយក៏បានរួមបញ្ចូលទាំងជាប្រភេទ `Float`

Syntax:

```
EEPROM.put(address,value)
```

- ❖ `EEPROM[]` : សម្រាប់ដាក់ ឬទាញយកទិន្នន័យពី `Address` ជាក់លាក់ណាមួយ

Syntax:

```
EEPROM[address]
```

Example:

```
int x=EEPROM[0]; //read data from address 0
EEPROM[5]=30; //write data from address 5
```

ឧទាហរណ៍ខាងក្រោមបង្ហាញអ្នកពីរបៀបដែល `EEPROM` ប្រើដើម្បីដាក់ និងទាញដាក់ 1byte

```
void setup()
Serial.begin(115200);
int x=EEPROM.read(7);
}
void loop(){
    if(Serial.available()>0){
        String txt =Serial.readString();
        int y=txt.toInt();
        EEPROM.write(7,y);
    }
}
```

ឧទាហរណ៍ខាងក្រោមបង្ហាញអ្នកពីរបៀបដែល `EEPROM` ប្រើដើម្បីដាក់ និងទាញយក `Data`

Object:

```
void setup()
Serial.begin(115200);
int x=EEPROM.get(15);
}
void loop(){
    if(Serial.available()>0){
        String txt =Serial.readString();
        EEPROM.put(15,txt);
    }
}
```

```

#include<EEPROM.h>
void setup() {
    Serial.begin(9600);{
    EEPROM.write(0,100);
}
void loop(){
    int x =EEPROM.read(0);
    Serial.println(x);
    delay(1000);
}
ថ្ងៃ៣៖string
#include<EEPROM.h>
void setup() {
    Serial.begin(9600);{
    EEPROM.write(0,"HELLO");
    pinMode(A0,OUTPUT);
}
void loop(){
    int x = analogRead(A0);
    for (int i=0;;i<=200,i++){
    EEPROM.write(i,x);
    delay(1000);

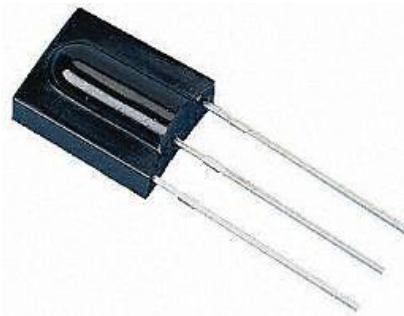
```

ជំពូកទី៦ មូលដ្ឋានគ្រឹះនៃទំនាក់ទំនង តាមប្រព័ន្ធ WIRELESS



I. ការគ្រប់គ្រងការបញ្ជាតាមប្រព័ន្ធ IR

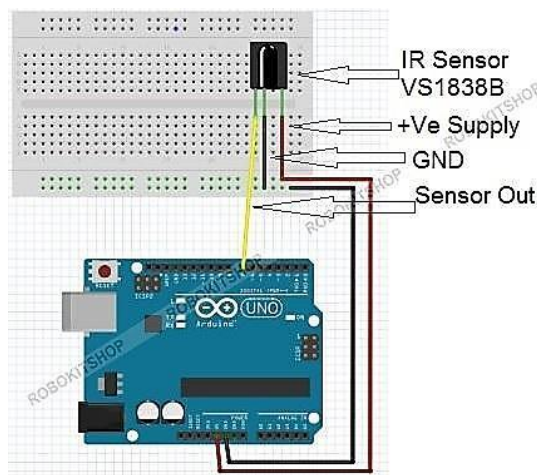
IR វាជាតេលេដែលយើងប្រើរាល់ថ្ងៃជាមួយម៉ាស៊ីនត្រជាក់ ឬទូរទស្សន៍ ។ ក្នុងមេរៀននេះ យើងនឹងសិក្សាអំពីការចម្លង ក៏ដូចជាការបង្កើតតេលេនេះជាមួយ Arduino ។



ដើម្បីអាចប្រើប្រាស់វាបានយើងត្រូវការLibraryមួយដែលរកបានតាមរយៈអាសយដ្ឋាន

<https://github.com/z3t0/Arduino-IRremote>

ir ជាមួយ Arduino



1.Function of Syntax

ចំពោះ syntax ដែលអាចដំណើរការលើវាមានមួយចំនួនដូចខាងក្រោម

- ❖ Receive

- Constructor

Syntax: `IRrecv(IR pin);`

Example: `IRrecv irrecv(10);`

- Decode variable

Syntax: `decode_results results;`

Example:

`IRrecv irrecv(10);`

`decode_results results;`

- Begin

Syntax: `irrecv.enableIRIn();`

- decode

Syntax:

`Irrecv.decode(&results);`

Return:

True : if IR receive data from remote

False : if No data receive

- Get Value

Syntax :

`results.value;`

Return:

Data from remote

- Resume

Syntax:

`irrcecv.decode(&reults);`

Return:

True: if IR receive data from remote

False: if No data receive

- Get Value

Syntax:

results.value;

Return:

Data from remote

- Resume

Syntax:

irrecv.resume();

Example:

irrecv.resume();

- ❖ Transmitter

- Constructor

Syntax:

IRsend irsend;

- Send

Syntax:

Irsend.sendSony(data,bits);

Parameter:

data: ទិន្នន័យដែលត្រូវផ្ញើ

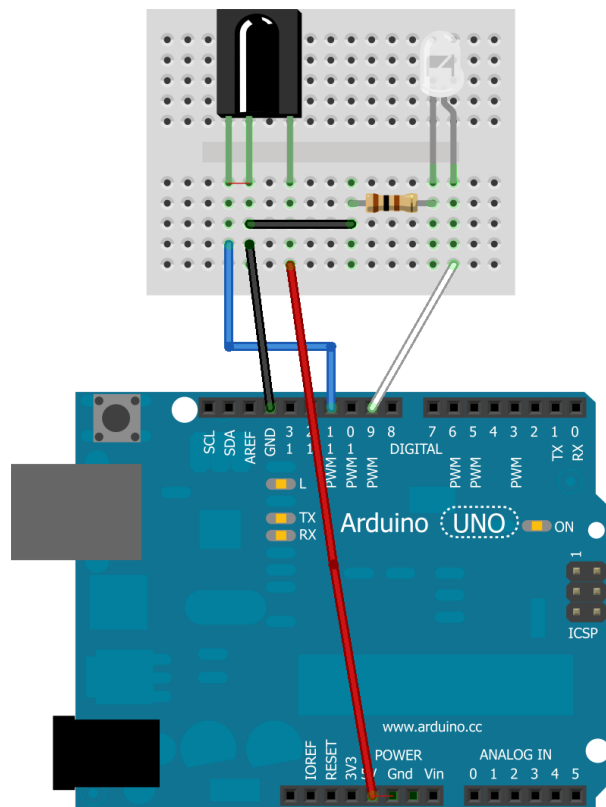
bits : ចំនួន bit ដែលត្រូវផ្ញើ

ខាងក្រោមជាសៀគ្វី និងគំរូកូដសម្រាប់ការបញ្ជូនពី Arduino ទៅកាន់ឧបករណ៍ណាមួយរបស់អ្នក៖

Hardware specifications

Board/CPU	Send Pin	Timers
ATtiny84	6	1
ATtiny85	1	TINYO
ATmega8	9	1
Atmega32u4	5,9,13	1,3,4
ATmega48, ATmega88, ATmega168, ATmega328	3,9	1,2
ATmega1284	13,14,6	1,2,3
ATmega164, ATmega324, ATmega644	13,14	1,2
ATmega8535, ATmega16, ATmega32	13	1
ATmega64, ATmega168	13	1

ATmega1280, ATmega2560	5,6,9,11,46	1,2,3,4,5
ESP32	N/A(not supported)	1
Teensy 1.0	17	1
Teensy 2.0	9,10,14	1,3,4_HS
Teensy++ 1.0/2.0	1,16,25	1,2,3
Teensy 3.0/3.1	5	CMT
Teensy-LC	16	TPM1



2.ការបង្ហាញតាមរយៈ Serial Monitor

កូដសម្រាប់ការធ្វើតេស្តទៅលើ IR

កូដ BIN សម្រាប់Transimt

```
#include <boarddefs.h>
#include <IRremote.h>
#include <IRremoteInt.h>
#include <ir_Lego_PF_BitStreamEncoder.h>
IRrecv myIR(2);
```

```

decode_results results;
void setup(){
    Serial.begin(9600);
    myIR.enableIRIn();
}
void loop(){
    if (myIR.decode(&results)){
        Serial.println(results.value,BIN);
        myIR.resume();
    }
}

```

គេអាចដាច់កូដ ដោយចុចលេខ 1 អំពូលទី១ភ្លឺ និងចុចលេខ២ អំពូលទីពីរភ្លឺ

```

#include <boarddefs.h>
#include <IRremote.h>
#include <IRremoteInt.h>
#include <ir_Lego_PF_BitStreamEncoder.h>
IRrecv myIR(12);
decode_results results;
void setup() {
    pinMode(13,OUTPUT);
    pinMode(3,OUTPUT);
    digitalWrite(13,HIGH);
    Serial.begin(9600);
    myIR.enableIRIn();
}
void loop() {
    if (myIR.decode(&results)) {
        Serial.println(results.value,HEX);
        long x = results.value;
        if (x==0x6496C054){
            digitalWrite(3,HIGH);
        }
        else if (x==0x1E05F7A0){
            digitalWrite(3,LOW);
        }
        myIR.resume();
    }
}

```


Ir ខាងលើនេះប្រើជើងលេខ 12

Codeដើម្បីចុចមួយរលត់ចុចម្តងទៀតភ្លឺ

```
#include <boarddefs.h>
#include <IRremote.h>
#include <IRremoteInt.h>
#include <ir_Lego_PF_BitStreamEncoder.h>
boolean=i;
IRrecv myIR(12);
decode_results results;
void setup() {
    pinMode(13,OUTPUT);
    pinMode(3,OUTPUT);
    digitalWrite(13,HIGH);
    Serial.begin(9600);
    myIR.enableIRIn();
}
void loop() {
    if (myIR.decode(&results)) {
        Serial.println(results.value,HEX);
        long x = results.value;
        if (x==0x6496C054){
            digitalWrite(3,i);
            i=!i;
            delay(200);
        }
        myIR.resume();
    }
}
```

ប្រភេទIR មួយប្រភេទដែលគេប្រើសម្រាប់ឡានបញ្ជា

Recive code

```
#include <RH_ASK.h>
RH_ASK driver; //Rx pin 11
char buf[RH_ASK_MAX_MESSAGE_LEN];
char buflen = sizeof(buf);
void setup(){
    Serial.begin(9600);
    driver.init();
}
void loop(){
```

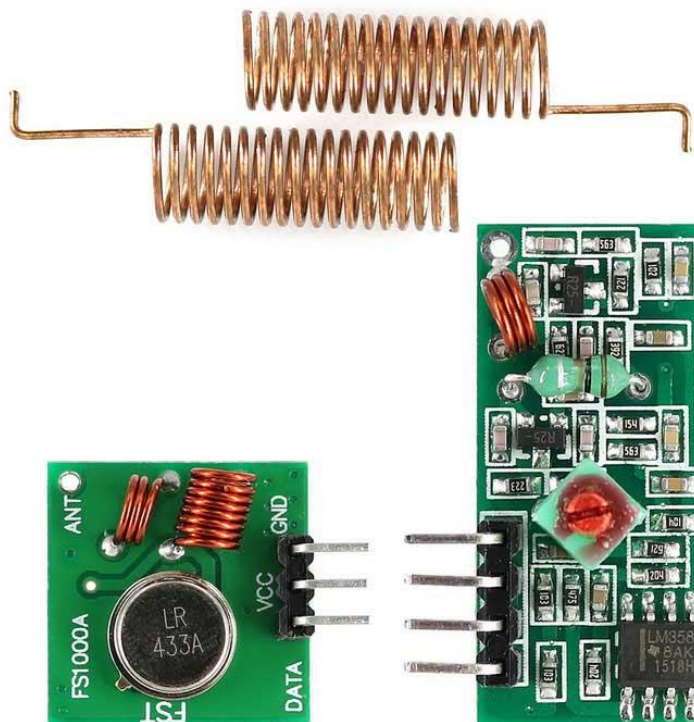
```

if (driver.recv(buf, &buflen)){
String txt(buf);
Serial.println(txt);
    }
}

```

II. RF Tx/Rx module

វាជាប្រភេទ module ដែលអនុញ្ញាតឱ្យយើងធ្វើទំនាក់ទំនងដោយអត់ខ្សែក្រោមក្រោយកង់ វិទ្យុ 433MHz ក្នុងចម្ងាយជិតជាង 20m-90m រវាង Arduino ពីរ ឬច្រើន ។ Module នេះមានពីរ ប្រភេទផ្សេងគ្នាដែលមួយប្រើសម្រាប់ការបញ្ជូនទិន្នន័យ និងមួយម្ខាងទៀតសម្រាប់ការទទួល ទិន្នន័យ ។ វាមានសណ្ឋានដូចខាងក្រោម ៖



ទោះបីយ៉ាងណាក៏ដោយការបញ្ជូនជាមួយវាមានការយឺតដែលសម្រាប់ Application មួយចំនួន ប្រហែលមិនគ្រប់គ្រាន់ទេ ។ ដើម្បីងាយស្រួលប្រើការ យើងត្រូវមាន Library មួយដែលអាចរក បាននៅក្នុងអាសយដ្ឋាន

<http://randomnerdtutorials.com/rf-433mhz-transmitter-receiver-module-with-arduino/>

1..Function of Syntax

សម្រាប់ Function របស់វាមាន ៖

❖ Contructor

Syntax :

RH-ASK Variable;

❖ Initial

Variable.init();

❖ Send

Syntax:

Variable.send(Char array, char length);

❖ Wait Data to send completely before send new data

Syntax:

Variable.waitPacketSent();

❖ Receive

Syntax:

Variable.recv(Char array,&char length);

❖ **Transmitter**

```
#include<RH_ASK.h>
RH_ASK driver;//connect to pin 12 of Arduino
void setup(){
  Serial.begin(9600);//
  Driver.init();
}
void loop(){
  char msg[60];
  String txt "hello Cambodia";
  txt.toCharArray(msg,txt.length()+1);
  driver.send(msg,strlen(msg));
  driver.waitPacketSent();
  delay(200);
}
```

❖ **Receiver**

```
#include<RH_ASK.h>
RH_ASK driver;//Rx pin 11
char buf[RH_ASK_MAX_MESSAGE_LEN];
char buflen=sizeof(buf);
void setup(){
  Serial.begin(9600);
```

```

driver.init();
void loop(){
  if(driver.recv(buf,&buflen)){
    String txt(buf);
    Serial.println(txt);
  }
}

```

III.UART Tx/Rx Tranceiver module(HC-12)

វាជាម៉ូដែលដែលក្រោយរបស់ HC-11 ដែលអាចធ្វើទំនាក់ទំនងបានរហូតដល់ទៅ 1800m នៅក្នុងលំហទទេ។ វាមានភាពងាយស្រួលក្នុងការប្រើប្រាស់ជាង Module មុនៗដែលមានតែ Transmitter&Receiver ផ្សេងៗ ម៉្យាងទៀតដោយវាប្រើ protocol UART ហេតុដូច្នេះហើយវាងាយស្រួលក្នុងការប្រើប្រាស់ជាមួយ Serial របស់ Arduino ជា module ដ៏ទៃទៀតឆ្លាយណាស់ ។



HC-12 មានភ្ជាប់មកជាមួយនូវ AT ដែលអាចប្រើសម្រាប់ការ Register វា ។ AT Command លំអិតមាននៅ ។

<http://trainelectronics.com/Arduino/Hc-12-SerialRadio/images/Hc12%20v2.3A.pdf>

ដើម្បីអាចចូលទៅកាន់ AT Command យើងត្រូវផ្ដាច់វាពីភ្លើងសិន រួចផ្ដាច់ជើងSet របស់វាទៅកាន់តង់ស្យុង 0v រួចភ្ជាប់ភ្លើងម្តងទៀត នោះវានឹងចូលទៅកាន់ AT Command ជាមួយ Baudrate 9600 ។ យើងអាចប្រើប្រាស់ Arduino ដើម្បីចូលទៅកាន់ AT command នេះបានតាមរយៈការបញ្ចូលកូដ Blank ទៅវារួចប្រើប្រាស់ Serial Monitor

1.Function of Syntax

❖ AT

សម្រាប់តេស្តថាតើ Communication រវាងនឹង HC-12 module

Return :

OK មានន័យថាការភ្ជាប់គឺជោគជ័យ ផ្ទុយទៅវិញគឺគ្មានអ្វី return

❖ AT+Bxxxx

សម្រាប់កំណត់ Baudrate

Syntax :

AT+Bxxxx

Example:

AT+B115200

Return:

OK+ B115200

❖ AT+Cxxxx

សម្រាប់កំណត់ channel ឱ្យវាដែលមានពី 001-127

Syntax:

AT+Cxxxx

Example:

AT+C002

Return:

OK+C002

❖ AT+Px

សម្រាប់កំណត់អនុភាពបញ្ជូនចេញឱ្យវា ដែលមានតម្លៃពី1-8= -1dBm to 20dBm

Syntax:

AT+Px

Example:

AT+P5

Return:

OK+P5

❖ AT+Uxxx

សម្រាប់កំណត់ចំនួន bit, Parity និង Stop bit ដែល 0 មានន័យថា Odd, E មានន័យថា Even ,
1 stop bit 1 , 2 stop bit 2 និង 3 stop bit 1.5

Syntax:

AT+Uxxx

Example:

At+U8E1

Return:

OK+U8E1

❖ AT+FUx

សម្រាប់កំណត់ Mode ឱ្យ Protocol Serial ។ វាមាន 4 ម៉ូតគឺ FU1, FU2 , FU3 និង FU4 ។

- FU1 :ប្រើសម្រាប់សន្សំថាមពល វាស៊ីត្រឹម 3.6mA ប្រើបានគ្រប់ Baud rate
- FU2: ប្រើសម្រាប់សន្សំថាមពលបានកាន់តែច្រើនជាងមុនគឺ 80uA មានតែ Baud rate 1200bps , 2400bps និង 4800bqs ទេដែលប្រើបាន ។ ដើម្បីជៀសវាងការបាត់បង់ទិន្នន័យតាមផ្លូវ វាទាមទារឱ្យការបញ្ជូនត្រូវចំណាយពេលយូរជាង ១វិនាទី១ ។
- FU3 : វាស៊ីថាមពល 16mA និងប្រើបានគ្រប់ Baud rate
- FU4 : សម្រាប់បញ្ជូនរយៈចម្ងាយរហូតដល់ 1.8km ។ ដើម្បីជៀសវាងការបាត់បង់ទិន្នន័យតាមផ្លូវ វាទាមទារឱ្យការបញ្ជូនត្រូវចំណាយពេល យូរជាង 4 វិនាទី ។

Syntax:

At+FUx

Example:

AT+FU4

Return:

AT+OK

❖ AT+SLEEP

សម្រាប់កំណត់ឱ្យ HC - 12 ចូលទៅ Sleep mode ដូចនេះវានឹងស៊ីថាមពលតិច ។ ភាគច្រើនយើងប្រើវានៅដែលគ្មានប្រតិបត្តិការទិន្នន័យដើម្បីសន្សំ ។

Syntax:

AT+SLEEP

Example:

AT+SLEEP

Return:

OK+SLEEP

❖ AT+DEFAULT

សម្រាប់កំណត់ឱ្យវាទៅកាន់ Default value ដើមពីរោចក្រវារីញ

Syntax:

AT+DEFAULT

Example:

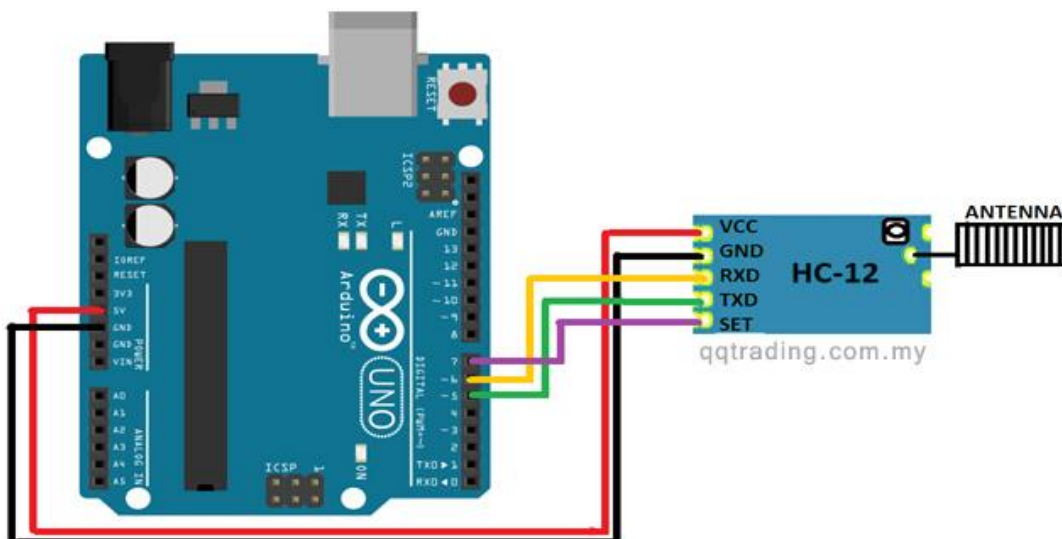
AT+DEFAULT

Return:

OK+DEFAULT

2. របៀប Transmite and Receive Data

ជាដំបូងត្រូវភ្ជាប់ជើង HC-12 ជាមួយជើង Arduino



គេត្រូវអនុវត្តតាមជំហានដូចខាងក្រោម

a/ upload blink code ចូលទៅក្នុងArduino

b/ ចូលទៅ Serial monitor ដើម្បីចុះឈ្មោះ channel (receiver ក៏មានchannel ដូចគ្នាដែរ) យកកូដ Blink មក register channel

Example

AT

AT+DEFAULT

AT+C002

ក្រោយពីបានកំណត់នូវchannel រួចគេupload ចូលក្នុងArduino

ដាក់កូដសម្រាប់Transmit និង receive

- កូដសម្រាប់ Transmit(1)

```
void setup() {
  Serial.begin(9600);
  Serial.setTimeout(100);
}
void loop() {
  Serial.println("HELLO TEACHER");
  delay(1000);
}
```

- កូដសម្រាប់ Receive (2)

```
void setup() {
  Serial.begin(9600);
  Serial.setTimeout(100);
}
void loop() {
  if (Serial.available() > 0){
    String x=Serial.readString();
    Serial.println(x);
  }
}
```

c/ ដកជើង set ចេញពីGND ហើយដួរជើងTX ទៅ RX និង RX មក TX វិញ

d/យកfile ចាស់ blink នេះមកសរសេរកូដបន្ថែមទៀតសម្រាប់ដាក់ transmit ហើយនិង receive

e/នៅពេលupload កូដ ត្រូវដកជើងទី 0 ឬ ទី1 ឬដកទាំង TX និង RX ជាមុនសិនព្រោះជើងទាំងនេះពិបាក upload កូដណាស់ ។

គេអាចប្រើប្រាស់នូវ Syntax ដូចខាងក្រោមដើម្បីបញ្ជាឱ្យអំពូល ឬម៉ូទ័រ ដំណើរការក្នុងទម្រង់ផ្សេងគ្នា ។ យកកូដ(1) & (2) មកដាក់

សំណត់

ចូរអ្នក និងមិត្តរបស់អ្នកសរសេរកូដម្នាក់មួយដែលអាចឆ្លងឆ្លើយសារជាមួយគ្នាបានតាមរយៈ HC-12 ដោយប្រើ Serial Monitor ។

កូដសម្រាប់ recivce សម្រាប់ទទួល signal ផ្សេងៗ

Transmit data

```
void setup() {
  Serial.begin(9600);
  Serial.setTimeout(100);
}
void loop() {
  Serial.println("HELLO");
  delay(1000);
}
```

3.របៀបបញ្ជូនសញ្ញាដល់ឬធាតុអេឡិចត្រូនិចផ្សេងៗ

Transmite

```
void setup() {
  Serial.begin(9600);
  Serial.setTimeout(100);
}
void loop() {
  if(Serial.available()>0){
    String x= Serial.readString();
    Serial.print(x);
  }
}
```

Receiver

ខាងក្រោមនេះជាកូដដែលទទួលយកសញ្ញា ON និង OFF ដើម្បីឱ្យអំពូលដំណើរការ

```
void setup() {
  Serial.begin(9600);
  Serial.setTimeout(100);
  pinMode(2,1);
}
void loop() {
  if(Serial.available()>0){
    String x= Serial.readString();
    // Serial.print(x);
    if(x=="ON"){
      digitalWrite(2,1);
    }
  }
}
```

```

    else if (x=="OFF"){
        digitalWrite(2,0);
    }
}

```

Transmite Data

```

int b = 7;
//const int br=b;
void setup() {
    Serial.begin(9600);
    Serial.setTimeout(100);
    pinMode(7,INPUT);
}
void loop() {
    // if(Serial.available()>0)
    if(digitalRead (b)==1)
    {
        // String x= Serial.readString();
        String x= "ON";
        Serial.print(x);
        // if(digitalRead (b)==1){
        //     x="ON";
        //     delay(100);
        // }
    }
}
////////////////////////////////Transmitter////////////////////////////////
Reveiver code
boolean b=false;
void setup() {
    Serial.begin(9600);
    Serial.setTimeout(100);
    pinMode(2,OUTPUT);
}
void loop() {
    if(Serial.available()>0){
        String x= Serial.readString();
        //Serial.print(x);
        if(x=="ON"){

```

```

        digitalWrite(2,b);
        b=!b;
    }
//    else if (x=="OFF"){
//        digitalWrite(2,0);
//    }
}
}

```

4.HC12 with Button

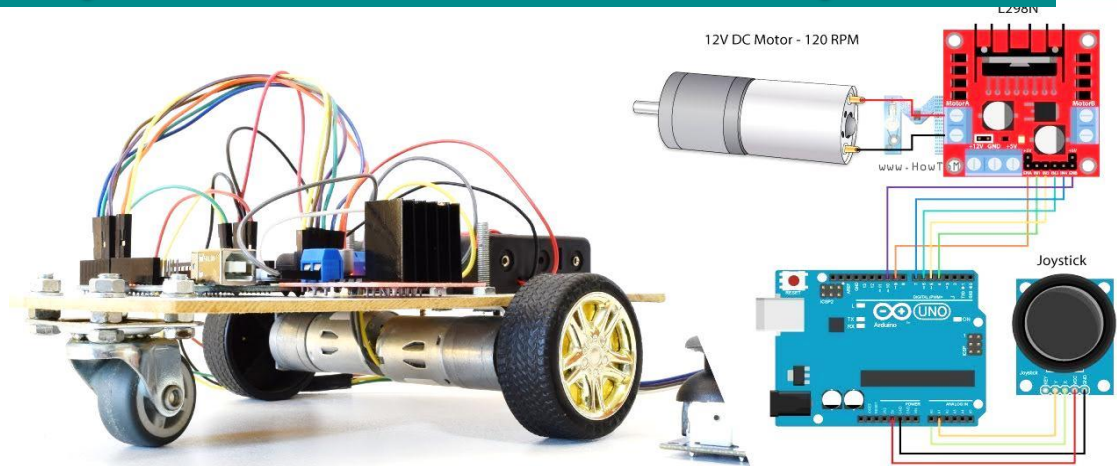
```

void setup() {
    Serial.begin(9600);
    Serial.setTimeout(100);
    pinMode(2,INPUT_PULLUP);
    pinMode(3,INPUT_PULLUP);
    pinMode(4,INPUT_PULLUP);
    pinMode(5,INPUT_PULLUP);
}
void loop() {
    String x=Serial.readString();
    if(digitalRead(2)==LOW){
        x='1';
        Serial.print(x);
        delay(200);
    }
    else if(digitalRead(3)==LOW){
        x='2';
        Serial.print(x);
        delay(200);
    }
    else if(digitalRead(4)==LOW){
        x='3';
        Serial.print(x);
        delay(200);
    }
    else if(digitalRead(5)==LOW){
        x='4';
        Serial.print(x);
        delay(200);
    }
}

```

ជំពូកទី៧

មូលដ្ឋានគ្រឹះនៃការបញ្ជាម៉ូទ័រ DC



I. Analog Input / Output

1. Analog Read and Syntax

យើងប្រើវាសម្រាប់ការអានយក និងបញ្ចេញសញ្ញា Analog ក្នុង Arduino ។ ទោះបី Arduino ជា Microcontroller ក៏ដោយក៏វាមិនប្រាកដថាប្រើបានតែជាមួយ Digital ដែរ ។ ក្នុងមេរៀននេះយើងនឹងសិក្សាពីការប្រើប្រាស់ Analog pin របស់វា ។ Arduino Uno មាន 6 channel ហើយ Mega មាន 8 channels ។ Analog reference ប្រើសម្រាប់កំណត់តម្លៃតង់ស្យុង ធៀប ដែលត្រូវយកទៅធៀបនឹងតង់ស្យុងនៃជើង Analog Input របស់វា ។ តម្លៃតង់ស្យុងដែល យើងកំណត់នេះជាតម្លៃតង់ស្យុងអតិបរមាដែលជើង Analog Input អាចទទួលបាន មានន័យ ថាតម្លៃនោះគឺបើគិតថា Digital គឺស្មើ 1023 ។

Syntax:

`analogReference(type)`

Parameter:

type: ជ្រើសរើសប្រភេទដែលត្រូវប្រើ(DEFAULT, INTERNAL, INTERNAL1V1, INTERNAL2V56, EXTERNAL)

-DEFAULT : តង់ស្យុងធៀបមានតម្លៃ 5 volts (on 5v Arduino boards) ឬ 3.3 volts (on 3.3v Arduino boards)

- INTERNAL : តង់ស្យុងធៀបស្មើ 1.1 volts ចំពោះ ATmega168 ឬ ATmega328 និង 2.56 volts នៅលើ ATmega8 (សម្រាប់ Arduino Mega ពុំមានជម្រើសនេះទេ)
- INTERNAL1V1: តង់ស្យុងធៀបស្មើ 1.1v សម្រាប់ Arduino Mega តែប៉ុណ្ណោះ
- INTERNAL2V56: តង់ស្យុងធៀបស្មើ 2.56V សម្រាប់ Arduino Mega តែប៉ុណ្ណោះ
- EXTERNAL : តង់ស្យុងដាក់ទៅកាន់ជើង AREF របស់វាពីខាងក្រៅ (0-5V only) ត្រូវប្រើជាតង់ស្យុងធៀប Analog Read ប្រើសម្រាប់ទាញយកតម្លៃ Analog ហើយបំប្លែងវាជាតម្លៃ Digital ។ Resolution Analog to Digital មាន 10 bits ដែលមានន័យថាវាបំប្លែងតង់ស្យុងពី 0V-5V ឱ្យទៅជាតម្លៃលេខចន្លោះពី 0-1023 ស្មើនឹង 4.9mV/unit ។

analogRead(pin)

parameter:

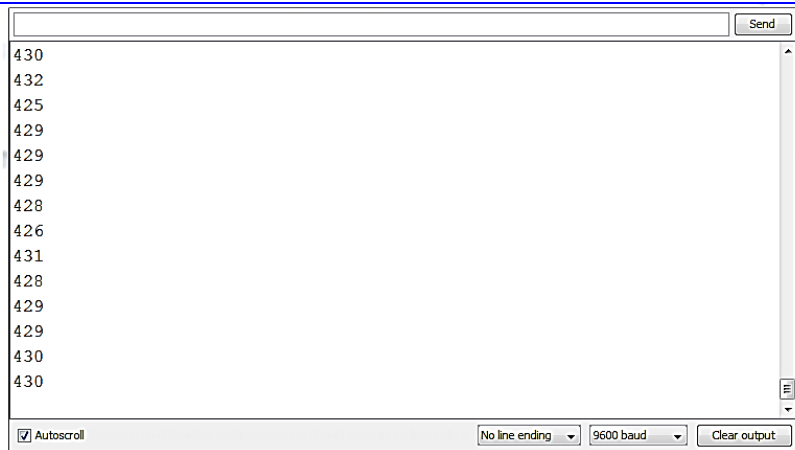
pin: លេខជើង Analog របស់ Arduino

Return

0-1023

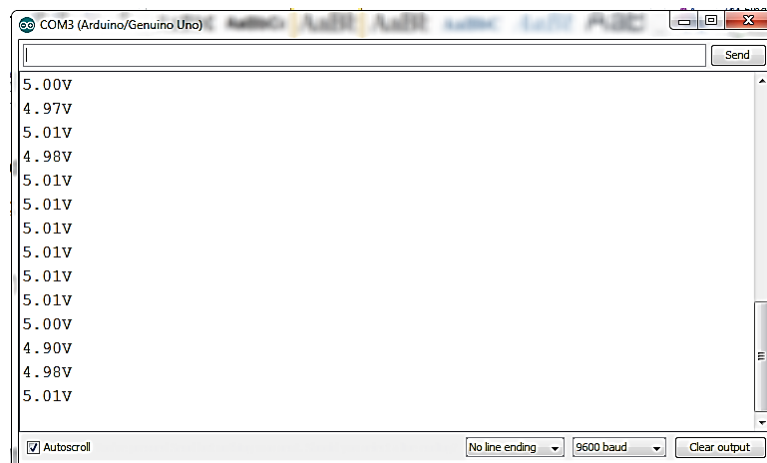
លំហាត់ ៖ ចូរអ្នកប្រើ Potentiometer សម្រាប់តេស្តជាមួយ Analog read តាមរយៈការភ្ជាប់វាដូចសៀគ្វីខាងក្រោម រួចបង្ហាញតម្លៃដែលអានបាននៅក្នុង Serial monitor

```
void setup() {
  Serial.begin(9600);
  pinMode(A0, INPUT); //optional
}
void loop() {
  int x = analogRead(A0);
  Serial.println(x);
  delay(500);
}
```



ចង់ឱ្យរក្សាលំនឹងបញ្ចេញក្បែរ

```
void setup() {
  Serial.begin(9600);
  pinMode(A0,INPUT);//optional
}
void loop() {
  int x=analogRead(A0);
  float y=x*0.0049;
  Serial.print(y);
  Serial.println("V");
  delay(500);
}
```



2. រៀបចំបញ្ជាអំពូល និង ម៉ូដុលឱ្យដំណើរការ

```
void setup() {
  Serial.begin(9600);
  pinMode(3,INPUT);
}
void loop() {
```

```

    analogWrite(3,125);
    delay(500);
    analogWrite(3,255);
    delay(500);
}
ករណីដាក់អំពូលឱ្យវាតែម្តង
void setup() {
    Serial.begin(9600);
    pinMode(3,OUTPUT);
}
void loop() {
    for (int i=0;i<=255;i++){
        analogWrite(3,i);
        delay(100);
    }
}

```

3.ការប្រើប្រាស់ RGB LED

RGB = Red , Green , Blue

គ្រប់ពណ៌ទាំងអស់កើតតែពីពណ៌ បីប៉ុណ្ណោះគឺ ក្រហម បៃតង និង ខៀវ ។ ពណ៌ទាំងបីនេះផ្សំចូលគ្នាបង្កើតបានពណ៌ទៀតអាស្រ័យទៅលើការកម្រិតពណ៌ដែលត្រូវបានដាក់ ហើយពណ៌នីមួយៗមានតម្លៃខ្ពស់បំផុត 255 ។ ដើម្បីមានភាពងាយស្រួលក្នុងការជ្រើសពណ៌គេតែងតែជ្រើសពណ៌តាម color pick up ដែលត្រូវ Browser ។ ខាងក្រោមនេះគឺជាឧទាហរណ៍នៃការតម្លៃគឺមានអំពូល RGB ដោយប្រើជើង Analog (PWM ~) ក្នុងករណីនេះគេត្រូវប្រើវេស៊ីស្តង់មួយដើម្បីជៀសវាងការឆេះអំពូល ។

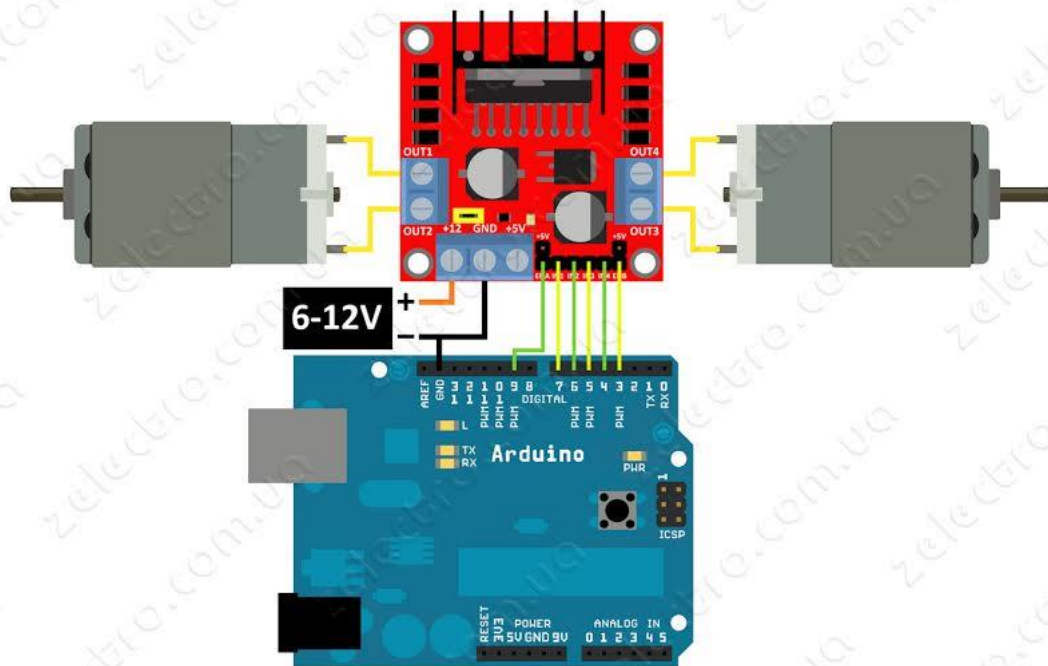
```

void setup() {
    Serial.begin(9600);
    pinMode(9,OUTPUT);
    pinMode(10,OUTPUT);
    pinMode(11,OUTPUT);
}
void loop() {
    analogWrite(9,255);
    analogWrite(10,255);
    analogWrite(11,255);
}

```

ដើម្បីចង់ដឹងពីពិធីការបស់វាគេត្រូវដាក់ជើងពីរមានតម្លៃ ០ ហើយជើងមួយទៀតដាក់ 255 ដែលជាកម្រិតភ្លឺខ្ពស់បំផុត

ការបញ្ជាម៉ូទ័រ



```
void setup() {
  Serial.begin(9600);
  pinMode(2,OUTPUT);//IN1
  pinMode(3,OUTPUT);//IN2
  pinMode(4,OUTPUT);//IN3
  pinMode(5,OUTPUT);//IN4
}

void loop() {
  digitalWrite(2,1);
  digitalWrite(3,0);
  digitalWrite(4,1);
  digitalWrite(5,0);
}
```

សម្រាប់ឱ្យម៉ូទ័រដើរទៅមុខទៅក្រោយ

```
void setup() {
  Serial.begin(9600);
  pinMode(2,OUTPUT);//IN1
  pinMode(3,OUTPUT);//IN2
```



```

pinMode(4,OUTPUT);//IN3
pinMode(5,OUTPUT);//IN4
pinMode(9,OUTPUT);
pinMode(10,OUTPUT);
}
void loop() {
  digitalWrite(2,0);
  digitalWrite(3,1);
  analogWrite(9,255);
  digitalWrite(4,0);
  digitalWrite(5,1);
  analogWrite(10,255);
}

```

4.ការបញ្ជា car តាម IR remote

```

#include <boarddefs.h>
#include <IRremote.h>
#include <IRremoteInt.h>
#include <ir_Lego_PF_BitStreamEncoder.h>
IRrecv irrecv(12);
decode_results results;
void setup() {
  pinMode(13,OUTPUT);
  digitalWrite(13,HIGH);
  Serial.begin(9600);
  irrecv.enableIRIn();
  pinMode(2,OUTPUT); //IN1
  pinMode(3,OUTPUT); //IN2
  pinMode(4,OUTPUT); //IN3
  pinMode(5,OUTPUT); //IN4
  pinMode(9,OUTPUT); //ENA
  pinMode(10,OUTPUT); //ENB
}
void loop() {
  if (irrecv.decode(&results)) {
    Serial.println(results.value,HEX);
    long x = results.value;
    if (x==0x1FE649B){ //Forward
      digitalWrite(2,1);
      digitalWrite(3,0);
      analogWrite(9,200);
      digitalWrite(4,0);
    }
  }
}

```

```

        digitalWrite(5,1);
        analogWrite(10,200);
    }
    else if (x==0xFFFFFFFF){ //Backward
        digitalWrite(2,0);
        digitalWrite(3,1);
        analogWrite(9,200);
        digitalWrite(4,1);
        digitalWrite(5,0);
        analogWrite(10,200);
    }
    else if (x==0x1FEE41B){// turn right
        digitalWrite(2,1);
        digitalWrite(3,0);
        analogWrite(9,200);
        digitalWrite(4,0);
        digitalWrite(5,1);
        analogWrite(10,127);
    }
    else if (x==0x1FEA45B){// turn left
        digitalWrite(2,1);
        digitalWrite(3,0);
        analogWrite(9,127);
        digitalWrite(4,0);
        digitalWrite(5,1);
        analogWrite(10,200);
    }
    else if (x==0x1FEB04F){// stop
        digitalWrite(2,0);
        digitalWrite(3,0);
        analogWrite(9,127);
        digitalWrite(4,0);
        digitalWrite(5,0);
        analogWrite(10,200);
        irrecv.resume();
    }
}
}
}

```

5.ការបញ្ជា car តាម HC-12

```
//Receiver HC-12
```

```
void setup() {
    Serial.begin(9600);
    Serial.setTimeout(100);
    pinMode(2,1); //IN1
    pinMode(3,1); //IN2
    pinMode(4,1); //IN3
    pinMode(5,1); //IN4
    pinMode(9,1); //ENA
    pinMode(10,1); //ENB
}

void loop() {

    if (Serial.available()>0){
        String x = Serial.readString();
//        Serial.println(x);
        if (x=="forword"){
            digitalWrite(2,1);
            digitalWrite(3,0);
            analogWrite(9,200);
            digitalWrite(4,1);
            digitalWrite(5,0);
            analogWrite(10,255);
//            delay(200);
        }
        else if (x=="backward"){
            digitalWrite(2,0);
            digitalWrite(3,1);
            analogWrite(9,255);
            digitalWrite(4,0);
            digitalWrite(5,1);
            analogWrite(10,200);
//            delay(200);
        }
        else if (x=="left"){
            digitalWrite(2,1);
            digitalWrite(3,0);
            analogWrite(9,255);
            digitalWrite(4,1);
            digitalWrite(5,0);
```

```

        analogWrite(10,100);
//        delay(200);
    }

    else    if (x=="right"){
        digitalWrite(2,1);
        digitalWrite(3,0);
        analogWrite(9,100);
        digitalWrite(4,1);
        digitalWrite(5,0);
        analogWrite(10,255);
//        delay(200);
    }

    else {
        digitalWrite(4,0);
        digitalWrite(5,0);
        digitalWrite(2,0);
        digitalWrite(3,0);
        analogWrite(10,0);
        analogWrite(9,0);
    }
}
}

```

Transmitter HC-12 បញ្ជាម៉ូទ័រ ជាមួយប៊ូតុង

```

int b1 = 7;// CHANGE PIN HERE
int b2 = 8;
int b3 = 9;
int b4 = 10;
int b5 = 11;
void setup() {
    Serial.begin(9600);
    Serial.setTimeout(100);
    pinMode(7,INPUT_PULLUP);
    pinMode(8,INPUT_PULLUP);
    pinMode(9,INPUT_PULLUP);
    pinMode(10,INPUT_PULLUP);
    pinMode(11,INPUT_PULLUP);
}
void loop() {
    if(digitalRead (b1)==0){
        String x= "forword";

```

```

    Serial.print(x);
    delay(200);
  }
  else if(digitalRead (b2)==0){
String x= "backward";
Serial.print(x);
    delay(200);
  }
  else if(digitalRead (b3)==0){
String x= "right";
Serial.print(x);
    delay(200);
  }
  else if(digitalRead (b4)==0){
String x="left";
Serial.print(x);
    delay(200);
  }
  else if(digitalRead (b5)==0){
String x= "stop";
Serial.print(x);
    delay(200);
  }
}

```

ក្នុងខាងលើនេះមានប៊ូតុង 5 គឺ បត់ឆ្វេង បត់ស្តាំ ទៅមុខ ថយក្រោយ និង ឈប់ ។ ក្នុងករណីជាក់ស្តែងការធ្វើបែបនេះវាពិបាកបញ្ជាកូនឡាន ហេតុដូច្នេះនេះ យើងនឹងប្រើកូដមួយទៀតដែលពេលដែលចុចប៊ូតុងទើបម៉ូទ័រដើរ បត់ឆ្វេង បត់ស្តាំ ទៅមុខថយក្រោយ ហើយវាមានតែប៊ូតុង ៤ ទេ ការប្រើកូដសម្រាប់បញ្ជាម៉ូទ័រដោយចុចជាប់ទើបដើរបើកដៃឈប់ ។

```

int lastforward; int lastbackward; int lastright ;int lastleft ;int
forward;int backward;int right;int left;
void setup() {
  pinMode(2, INPUT_PULLUP);
  pinMode(3, INPUT_PULLUP);
  pinMode(4,INPUT_PULLUP);
  pinMode(5,INPUT_PULLUP);
  Serial.begin(9600);
}
void loop() {

```

```

forward = digitalRead(2);
backward = digitalRead(3);
right=digitalRead(4);
left=digitalRead(5);

if (forward != lastforward) {
    if (forward == 0) {
        Serial.println("forward");
    }
    else {
        Serial.println("stop");
    }
    delay(50);
}
else if (backward != lastbackward) {
    if (backward == 0) {
        Serial.println("backward");
    }
    else {
        Serial.println("stop");
    }
    delay(50);
}
//right and left
if (right!= lastright) {
    if (right == 0) {
        Serial.println("right");
    }
    else {
        Serial.println("stop");
    }
    delay(50);
}
else if (left != lastleft) {
    if (left == 0) {
        Serial.println("left");
    }
    else {
        Serial.println("stop");
    }
    delay(50);
}

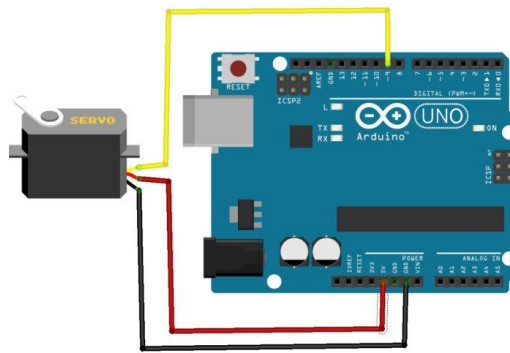
```

```

}
lastforward = forward;
lastbackward = backward;
lastright=right;
lastleft=left;
}

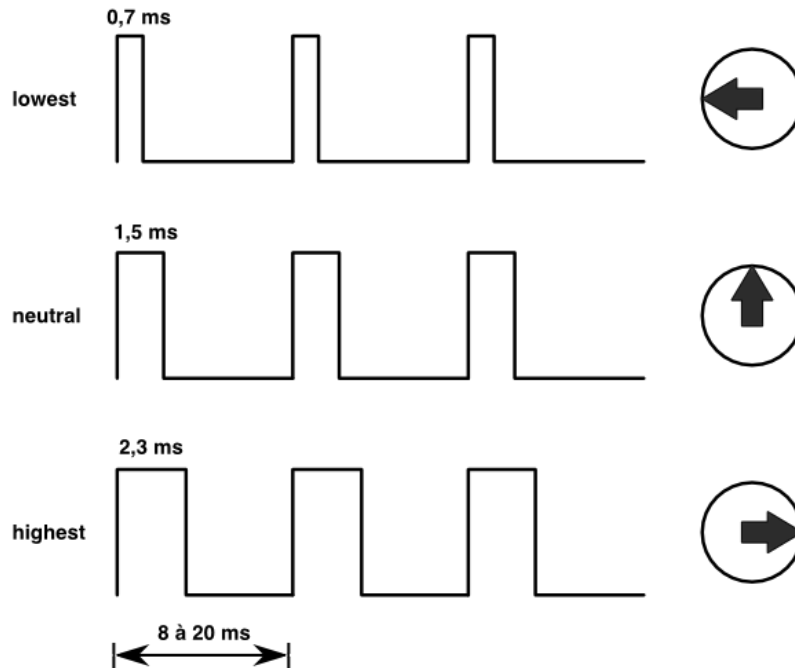
```

II. ការបញ្ជា servo motor



Servo Motor ជាប្រភេទនៃម៉ូទ័រដែលគេអាចបញ្ជាវាបានតាមរយៈសញ្ញាលំដាប់ខ្លីៗ ។ ការបញ្ជា នោះគឺគេធ្វើការបញ្ជាទៅកាន់កម្រិតមុំណាមួយដែលគេចង់បាន ។ ជាធម្មតាមុំនៃអ្វីលរបស់វា មាន ចន្លោះពី -60° ទៅ 60° និង ពី -90° ទៅ 90° អាស្រ័យលើប្រភេទម៉ូទ័រដែលអ្នកទិញ ។

Arduino បញ្ជូនសញ្ញាលំដាប់ PWM ដើម្បីបញ្ជាប្រភេទម៉ូទ័រនោះដែល PWM នោះមាន Duty cycle ប្រើ ហែលពី $0.5ms - 2.3ms$ និងប្រេកង់ $50Hz$ ។ ខាងក្រោមនេះជាក្រាបបង្ហាញពីសញ្ញា ៖



Arduino IDE មាន Library សម្រាប់ប្រើប្រាស់ជាមួយម៉ូទ័រប្រភេទនេះស្រាប់ ដោយគ្រាន់តែអ្នកដាក់វាចូលទៅកាន់ Sketch>>Include Library>> Servo ដែលមានស្រាប់ ។ ជាទូទៅវាអាចបញ្ជាម៉ូទ័រ Servo motor ចំនួន ១២ សម្រាប់ Arduino Board ធម្មតា និង ៤៨ ម៉ូទ័រសម្រាប់ Arduino Mega ។ ចំពោះ Arduino board ធម្មតា ការប្រើប្រាស់នូវ Library នេះនាំឱ្យមានការបិទនូវ PWM សញ្ញាដែលត្រូវចេញនៅជើង ៩ និង ១០ ទោះបីជើងទាំងនោះត្រូវប្រើ ឬមិនប្រើជាមួយ Servo ក៏ដោយ ។ សម្រាប់ Arduino Mega ក្នុងករណីប្រើ Servo Motor ក្នុងចំនួនតិចជាង ១២ ម៉ូទ័រជើង PWM ទាំងអស់នៅដំណើរការដូចធម្មតា លើកលែងតែចាប់ពីចន្លោះ ១២ ទៅ ២៣ម៉ូទ័រឡើងជើង PWM នៅជើងទី១១ និង ១២ ត្រូវបានបិទមិនដំណើរការ ។

1.Function of Syntax និង ឧទាហរណ៍

Function របស់វាមាន៖

❖ attach()

Syntax:

`servo.attach(pin)`

`servo.attach(pin,min,max)`

Parameters

pin: ជើងដែល Servo ភ្ជាប់ទៅកាន់

min(optional): the pulse width , in microseconds, corresponding to the minimum(0-degree) angle on the servo (defaults to 544)

max(optional): the pulse width, in microseconds , corresponding to the maximum(180-degree) angle on the servo(defaults to 2400)

❖ write()

Syntax:

```
servo.write(angle)
```

Parameters

angle: មុំដែលត្រូវវិលពី 0-180

❖ writeMicroseconds()

Syntax:

```
servo.writeMicroseconds(uS)
```

Parameters

uS: គិតជាមីក្រូនាទី

❖ read()

Syntax :

```
servo.read()
```

Return

មុំដែលម៉ូទ័រស្ថិតនៅ

❖ attached()

Syntax

```
servo.attached()
```

Parameters

សម្រាប់ពិនិត្យមើលថាតើ Servo pin ត្រូវបានប្រកាសឱ្យប្រើឬនៅ

❖ detach()

Syntax:

```
servo.detach()
```

Parameters

ផ្តាច់ជើង Servo ហើយក្លាយជា Digital ធម្មតាវិញ

```
#include <Servo.h>
```

```
Servo m1;
```

```
void setup() {
```

```
    m1.attach(3);
```

```
}
```

```
void loop() {
```

```
    m1.write(0);
```

```
    delay(3000);
```

```
    m1.write(90);
```

```
    delay(3000);
```

```
    m1.write(180);
```

```
}
```

កូដនេះដើរហើយវាមិនទាន់ល្អ ដោយត្រូវដើរចុះឡើងបន្តិចគេត្រូវបន្ថយនាទី min និង max

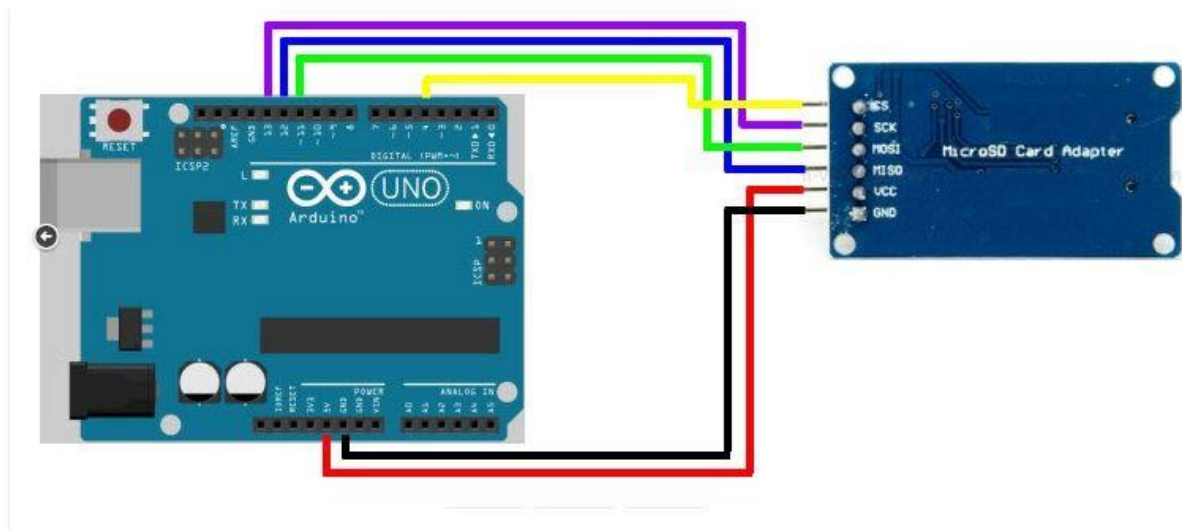
```
#include <Servo.h>
Servo m1;
void setup() {
  m1.attach(3,511,2100);
}
void loop() {
  m1.write(0);
  delay(3000);
  m1.write(90);
  delay(3000);
  m1.write(180);
  delay(3000);
```

```
}
```

ការប្រើ servo ជាមួយអេស៊ីស្តង់ត៍រ៉ាដ្យាត

```
#include <Servo.h>
Servo m1;
void setup(){
  m1.attach(3,544,2400);
}
void loop() {
  int an=analogRead(A0);//read 0-1023
  int ang=map(an,0,1023,544,2400);
  m1.writeMicroseconds(ang);
}
```

III.SD Card



នៅពេលដែលយើងដោត memory ទើបវាដំណើរការ

```
#include <SD.h>
void setup() {
  Serial.begin(9600);
  while(!SD.begin(4)){
    Serial.print(".");
    delay(1000);
  }
  Serial.println("\r\nSuccessfull");
}
void loop() {
}
```

ការបង្កើត name មេម៉ូរី

```
#include <SD.h>
void setup() {
  Serial.begin(9600);
  while(!SD.begin(4)){
    Serial.print(".");
    delay(1000);
  }
  Serial.println("\r\nSuccessfull");
  SD.mkdir("myfile");
}
void loop() {
}
```

បើចង់បង្កើត subfile

```
#include <SD.h>
void setup() {
  Serial.begin(9600);
  while(!SD.begin(4)){
    Serial.print(".");
    delay(1000);
  }
  Serial.println("\r\nSuccessfull");
  SD.mkdir("myfile/document");
}
void loop() {
}
```

ដើម្បីលុប subfile នៅ

```
#include <SD.h>
void setup() {
  Serial.begin(9600);
  while(!SD.begin(4)){
    Serial.print(".");
    delay(1000);
  }
  Serial.println("\r\nSuccessfull");
  SD.mkdir("myfile/document");
  SD.rmdir("myfile/document");
}
void loop() {
}
```

ការបង្កើត file

```
#include <SD.h>
File f;
void setup() {
  Serial.begin(9600);
  while(!SD.begin(4)){
    Serial.print(".");
    delay(1000);
  }
  Serial.println("\r\nSuccessfull");
  SD.mkdir("myfile/document");
  f=SD.open("myfile/document/test.txt",FILE_WRITE);
}
```

```

    f.close();
}
void loop() {
}

```

ការដាក់ទិន្នន័យក្នុងfile

```

#include <SD.h>
File f;
void setup() {
    Serial.begin(9600);
    while(!SD.begin(4)){
        Serial.print(".");
        delay(1000);
    }
    Serial.println("\r\nSuccessfull");
    SD.mkdir("myfile/document");
    f=SD.open("myfile/document/test.txt",FILE_WRITE);
    f.close();
    if(SD.exists("myfile/document/test.txt")){
        f=SD.open("myfile/document/test.txt",FILE_WRITE);
        if(f){
            f.println("HELLO");
            f.println("dog");
            f.println("the moon");
        }
        f.close();
    }
}
void loop() {
}

```

ការ Read អក្សរ

```

#include <SD.h>
File f;
void setup() {
    Serial.begin(9600);
    while(!SD.begin(4)){
        Serial.print(".");
        delay(1000);
    }
    Serial.println("\r\nSuccessfull");
    SD.mkdir("myfile/document");

```

```

    f=SD.open("myfile/document/test.txt",FILE_WRITE);
    f.close();
    // if(SD.exists("myfile/document/test.txt")){
    //   f=SD.open("myfile/document/test.txt",FILE_WRITE);
    //   if(f){
    //     f.println("HELLO");
    //     f.println("dog");
    //     f.println("the moon");
    //   }
    //   f.close();
    // }
    if(SD.exists("myfile/document/test.txt")){
    f=SD.open("myfile/document/test.txt",FILE_READ);
    if(f){
        while(f.available()){
            Serial.write(f.read());
        }
    }
    f.close();
    }
}
void loop() {
}

```

ជំពូកទី៨

SENSORS



នៅក្នុងផ្នែកនេះអ្នកនឹងរៀនពីប្រភេទ sensor ផ្សេងៗដែល sensor ទាំងនោះរួមមាន sensor សម្រាប់វាស់ សីតុណ្ហភាព ពន្លឺ ចម្ងាយ ចលនា អណ្តាតភ្លឺ អាកុល សំណើម sensor សម្រាប់វាស់ ទឹកភ្លៀង ជាដើម ។

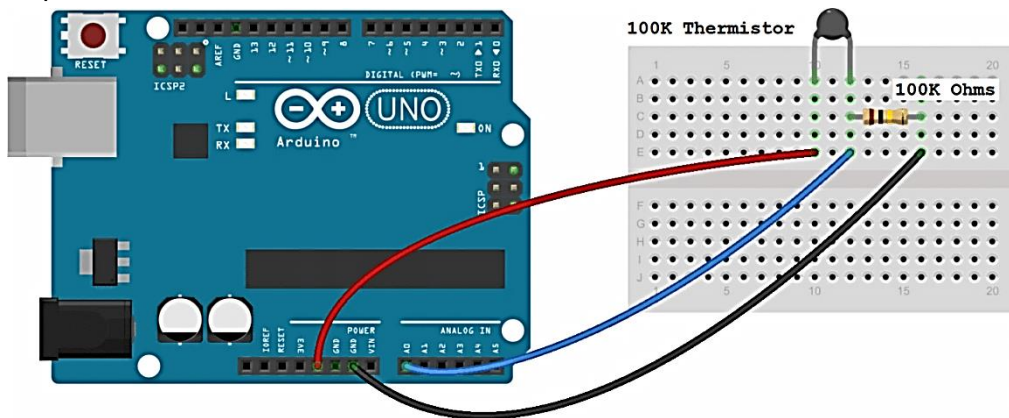
1.Thermistor sensor

Thermistor ជាស៊ីស្តង់អគ្គិសនីដែលតម្លៃរេស្តង់របស់វាប្រែប្រួលខ្លាំងតាមរយៈកំដៅពីខាងក្រៅ ។ វាត្រូវបានគេប្រើសម្រាប់រង្វាស់សីតុណ្ហភាពក្នុងកម្រិតសីតុណ្ហភាពខ្ពស់ល្មមល្បឿនតិចចាប់ពី ចន្លោះ -90°C – 300°C Thermistor មានភាពខុសគ្នាពី Resistance Temperature Detector (RTDs) ដោយសារធាតុផ្សំប្រើក្នុងវាជាធម្មតាធ្វើពីសេរ៉ាមិច ឬ Polymer ខណៈដែលធាតុផ្សំ RTDs ធ្វើពីលោហៈសុទ្ធហើយអាចរាស់រហូតដល់រាបរយពាន់អង្សាសេ ។



រូប thermistor មានច្រើន

បង្កើតសៀគ្វីនៃ thermistor sensor



ខាងនេះជាសណ្ឋានមួយចំនួននៃ Thermistor ។ ដោយសារលក្ខណៈពិសេសខាងលើរបស់វា វាត្រូវបានគេចាត់ទុកដូចជា Sensor មួយដូច្នោះដែរ ។ នៅក្នុងមេរៀននេះយើងនឹងធ្វើការសិក្សាជាមួយ Thermistor NTC 3950 ។ Thermistor ជាប្រភេទ Sensor ដែលធ្វើឱ្យប្រែប្រួលតង់ស្យុង No-linear ។ ដូច្នេះហើយក្នុងការគណនាសីតុណ្ហភាពរបស់វាក្នុងករណី thermistor NTC នេះគេប្រើសមីការ Steinhart-Hart ដែលមានរូបមន្ត ៖

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{\beta} \ln \left(\frac{R}{R_0} \right)$$

ដែល T :

T : សីតុណ្ហភាពដែលត្រូវវាស់គិតជា កែវរិន (K)

T_0 : សីតុណ្ហភាពនៅបន្ទប់ពិសោធន៍ $25^{\circ}C = 298.15K$

β : ចំនួនថេរ NTC 3950 មានន័យថា $\beta = 3950$

R : រេស៊ីស្តង់នៅសីតុណ្ហភាពកំពុងវាស់

R_0 : រេស៊ីស្តង់នៅក្នុងបន្ទប់ពិសោធន៍ក្នុងសីតុណ្ហភាព $25^{\circ}C = 10Kohm$

```
void setup() {
  Serial.begin(9600);
}
void loop() {
  int x =analogRead(A0);
  Serial.println(x);
  delay(500);
}
```

កូដខាងលើនេះជាកូដដែលបង្ហាញថាសីតុណ្ហភាពកើនឡើង នោះវាកើនលើងដែរ បើវាថយចុះវា នឹងថយចុះដែរ ។ ប្រើសិនជាយើងចង់ឱ្យបង្ហាញជាអង់ស្យូសេនោះគេ សរសេរកូដ

```
const float T0 = 298.15;
const float R0 = 10000;
const float belta = 3950;
const float r = 10000;
float val;
float R,vol,T;
void setup(){
  Serial.begin(9600);
}
void loop(){
  val = analogRead(A0);
  vol = (val * 5) / 1023;
  R = (r*(5-vol)) / vol;
  T= (T0 * belta) / (belta + (T0 * log(R/R0)))-273;
  Serial.println(String("Temperatur= ")+String(T));
  delay(500);
}
```

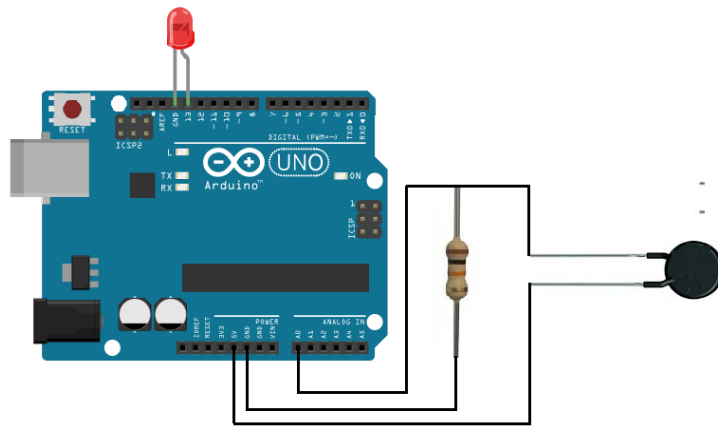
របៀបដាក់សីតុណ្ហភាពដើម្បីឱ្យអំពូល ឬ buzzer ផ្តល់សញ្ញានៅពេលសីតុណ្ហភាពកើនឡើង

```
const float T0 = 298.15;
const float R0 = 10000;
const float belta = 3950;
const float r = 10000;
float val;
float R, vol, T;
void setup() {
  Serial.begin(9600);
}
void loop() {
```

```

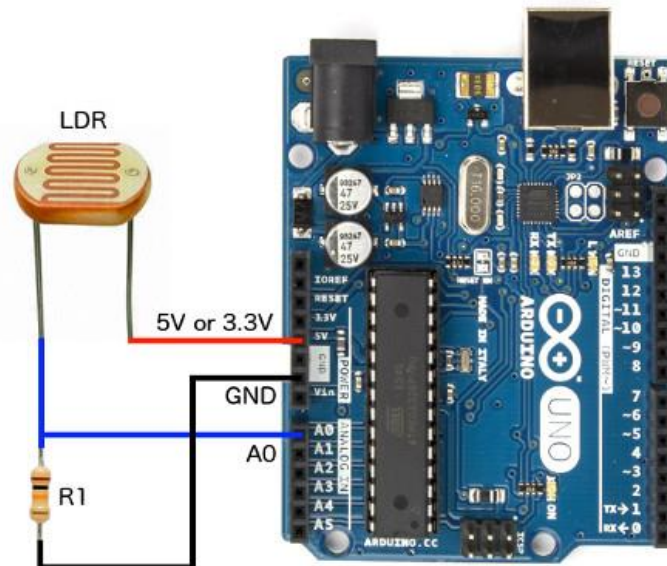
val = analogRead(A0);
vol = (val * 5) / 1023;
R = (r * (5 - vol)) / vol;
T = (T0 * belta) / (belta + (T0 * log(R / R0))) ;
int TC = T - 273;
pinMode(2,OUTPUT);
Serial.println(TC);
delay(500);
if(TC>37){
    digitalWrite(2,HIGH);
}
else{
    digitalWrite(2,LOW);
}
}
}

```



2.Light Sensor ត្រូវគេប្រើគេស្គាល់ជាទូទៅថាហ្វូតូរេស្តង់(LDR) ដែលប្រែប្រួលតម្លៃរេស៊ីស្តង់ របស់វាអាស្រ័យពន្លឺ យើងអាចប្រើ Analog Read របស់ Arduino ដើម្បីអានយកតម្លៃនៃ Lux ពន្លឺ តាមរយៈវាហើយធ្វើជាប្រព័ន្ធក្នុងត្រួលស្វ័យប្រវត្តិអ្វីមួយបាន ។ ខាងក្រោមនេះជាកំណត់តាងនៃ

សៀគ្វី យើងអាចវាស់កម្រិតពន្លឺតាមរូបមន្ត
$$Lux = \frac{500}{R_{LDR}}$$
 ដែល
$$R_{LDR} = \frac{5 - Vol}{I}$$



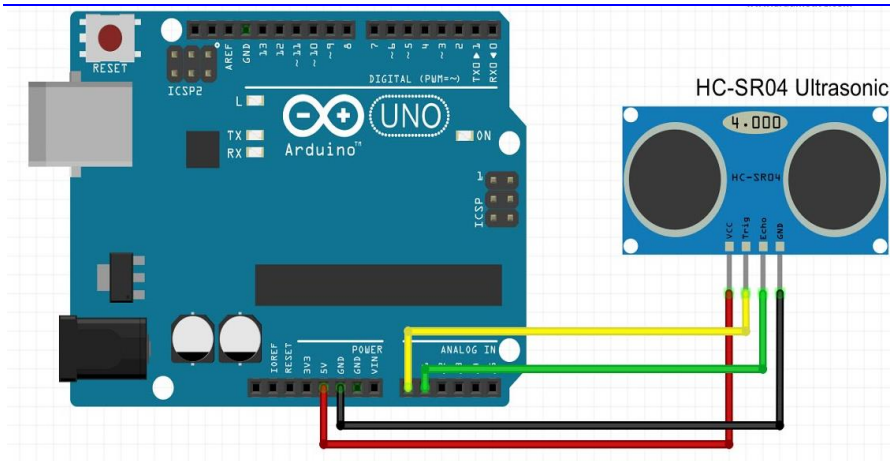
ខាងក្រោមនេះជាកូដសម្រាប់វាស់កម្រិតពន្លឺដែលបានប្រើស៊ីស្តង់ 100kΩ

```
const float r=100000;
float val;
float Rldr,vol,Lux;
void setup(){
    Serial.begin(9600);
}
void loop(){
    vol=analogRead(A0);
    vol=(val*5)/1023;
    Rldr=(r*(5-vol)/vol)/1000;
    Lux=500/Rldr;
    Serial.println(String("Lux=")+String(Lux));
    Delay(500);
}
```

3.Distance Sensor

យើងប្រើប្រាស់ Ultrasonic sensor module HC-SR04 សម្រាប់វាស់ចម្ងាយ ។ Ultra sound ដែលបានផ្លាស់ទី 340m/s និងទៅប៉ះជាមួយវត្ថុ ហើយឡុងត្រលប់មកវិញក្នុងរយៈពេលមួយ ។ យើងធ្វើការវាស់រយៈពេលនោះរួចគណនាជាចម្ងាយរបស់វាជៀបទៅនឹងវត្ថុ ឬឧបសគ្គនោះ ។ ខាងក្រោមនេះជា Datasheet របស់វា ៖

<http://www.micropik.com/PDF/HCSR04.pdf>



ឧទាហរណ៍៖

```
const int trigPin=2;
const int echoPin=4;
long duration,cm;
void setup(){
    Serial.begin(9600);
    pinMode(trigPin,OUTPUT);
    pinMode(echoPin,INPUT);
}
Void loop(){
    digitalWrite(triPin,LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin,HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin,LOW);
    duration=pulseIn(echoPin,HIGH);
    cm=microsecondsToCentimeters(duration);
    Serial.print(cm);
    Serial.println("cm");
    delay(100);
}
long microsecondsToCentimeters(long microseconds){
    return microseconds/58;
}
```

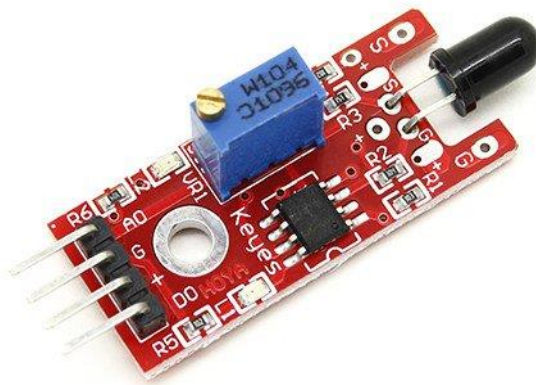
4.Security Sensor

- **Motion sensor** : ជាប្រភេទ sensor ដែលធ្វើការនៅពេលដែលមានវត្ថុ ឬភារៈមានជីវិតផ្លាស់ទី មានចលនាពីមុខវា ។ ជាធម្មតាវាឱ្យ OUTPUT ជាឌីជីថល ។ ចំពោះកូដរបស់វាគឺប្រើ DigitalRead ដើម្បីចាប់យកសញ្ញា ។

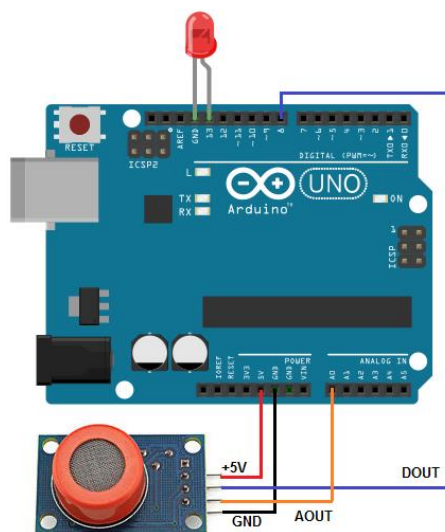


-Flame Sensor

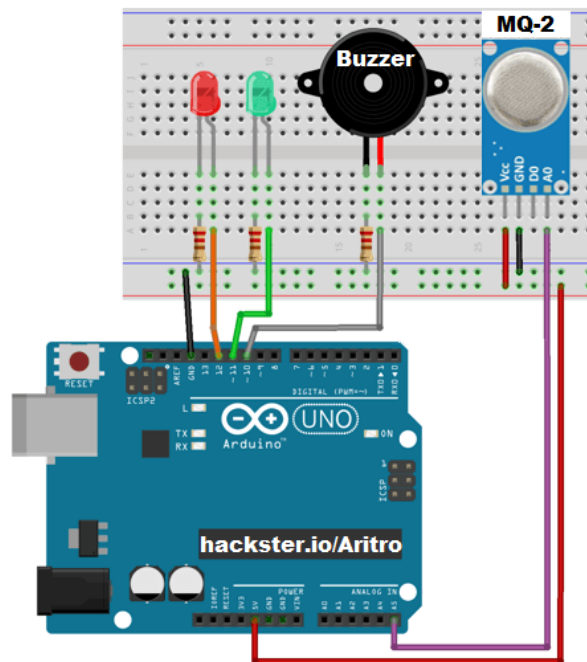
ជាប្រភេទ Sensor ប្រើសម្រាប់វាស់ ក៏ដូចជាចាប់យកសញ្ញាលំដាប់នៅពេលមានអគ្គិសនីកើតឡើង ។ វាចាប់យកតែពន្លឺអណ្តាតភ្លើង ហើយមិនចាប់យកផ្សែងនោះទេ ។ វាមាន OUTPUT ពីរប្រភេទគឺ Digital and Analog ដែលមានន័យថាយើងអាចប្រើ DigitalRead and AnalogRead ។



-Acohol Sensor : សម្រាប់វាស់កម្រិតជាតិអាល់កុល ។ វាមាន OUTPUT ពីរគឺ Digital and Analog ។

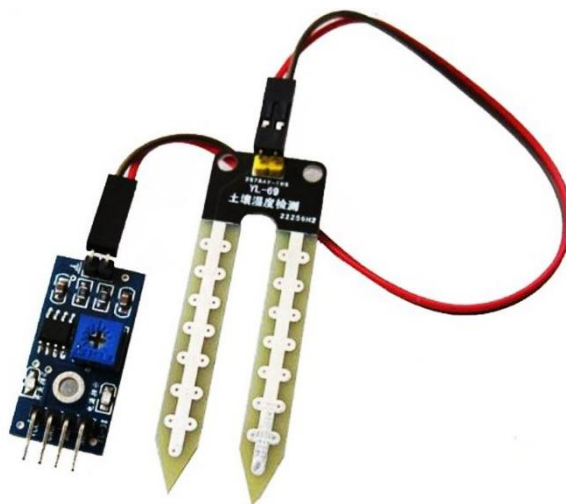


-**Gas Methan Senso** : សម្រាប់វាស់ ឬកំណត់កម្រិតនៃ Gas ដែលអាចនឹងឆ្លាយនៅក្នុងផ្ទះបាយ របស់យើង ។ វាឱ្យ OUTPUT Digital and Analog ។



5.Agriculture Sensor

-**Soil Sensor** :ជាប្រភេទ sensor ដែលត្រូវបានគេប្រើសម្រាប់វាស់កម្រិតសំនើមនៅក្នុងដីបាន ។ OUTPUT របស់វាឱ្យជា Digital and Analog ដែលតម្លៃរបស់វាស្មើ 1023 នៅពេលគ្មានសំនើម និង ស្មើ 0 នៅពេលដីសើម 100% ។



-Rain Sensor : ប្រើសម្រាប់ចាប់យកសញ្ញាលំដាប់ទៅពេលមានតំណក់ទឹកភ្លៀងស្រក់ចំណី ។ វាឱ្យ
OUTPUT ជា Digital ។ រូប



6.Switch sensor

-Capacitive touch switch: ជាប្រភេទ កុងតាក់ដែលគ្មានមេកានិច តែវាប្រើ Capacitive នៃដៃរបស់
យើងសម្រាប់ភ្ជាប់កុងតាក់ ។



ឯកសារពិគ្រោះ

-www.arduino.cc

-ADUINO PROGRAMMING part 1(2018) by ARROWDOT

-www.github